

Verbindung eines Roboterarms mit einem Regelungssystem für den Einsatz im Lehrbetrieb

MASTERTHESIS

PAUL FRIESEN

MATRIKELNUMMER: 1327520

Eidesstattliche Erklärung

Ich versichere hiermit, dass ich meine Masterarbeit mit dem Titel *Verbindung eines Roboterarms mit einem Regelungssystem für den Einsatz im Lehrbetrieb* selbstständig und ohne fremde Hilfe angefertigt habe, und dass ich alle von anderen Autoren wörtlich übernommenen Stellen wie auch die sich an die Gedankengänge anderer Autoren eng anliegenden Ausführungen meiner Arbeit besonders gekennzeichnet und die Quellen zitiert habe. Ich versichere zudem, dass die eingereichte elektronische Fassung mit der gedruckten Fassung übereinstimmt.

Mühlheim a.M., 06.11.2021

Unterschrift: 

Inhaltsverzeichnis

Abbildungsverzeichnis.....	V
Abkürzungsverzeichnis	VIII
1. Einleitung.....	1
1.1. Problemstellung	1
1.2. Zielsetzung.....	2
2. Grundlagen.....	4
2.1. Kinematik.....	4
2.2. Manipulator.....	9
2.3. Greifer.....	11
2.4. Steuerung	12
2.5. SmartPAD	14
2.6. KRL.....	17
3. Anbindungsmöglichkeiten.....	22
3.1. Vergleich der Feldbusse:	22
3.2. Weitere Anbindungen über TCP/IP oder UDP.....	28
3.3. Existierende Anbindungsimplementierungen.....	34
4. Eigene Implementierung	37
4.1. Bahnfahrt.....	38
4.2. KRL-Skript Teil 1.....	43
4.3. KRL-Skript Teil 2.....	44
4.4. Manuelle Bewegung.....	46
4.5. Generieren der Nachrichten.....	50
4.6. Laufzeittest	52
5. Applikation	57
5.1. Beispielapplikation 1	57
5.2. Beispielapplikation 2	58
5.3. Singularitäten	60
6. Modell	64
6.1. Modellierung des KR3	64
6.2. Erstellung der Trajektorie.....	68
7. Fazit	73
7.1. Bewertung	73
7.2. Ausblick.....	74

Literaturverzeichnis	76
Anhang 1: Ordnerstruktur der Abgabedatei	80
Anhang 2: Installation des KVP-Servers.....	82

Abbildungsverzeichnis

Abbildung 1: Anzahl der weltweit eingesetzten Industrieroboter von 2009 bis 2019 [1].	1
Abbildung 2: Darstellung einer Z-Y-X Rotation [4].	7
Abbildung 3: Darstellung der Denavit-Hartenberg-Transformation zwischen zwei lokalen Koordinatensystemen [5].	8
Abbildung 4: Schaubild für den Aufbau einer Robotersteuerung [3].	9
Abbildung 5: Bewegungsbereich der einzelnen Achsen des KR3 R540 [6].	10
Abbildung 6: (Links) Seitliche Aufnahme des Manipulators. (Rechts) Drehrichtungen der einzelnen Gelenkachsen [6].	10
Abbildung 7: (links) Draufansicht des Greifers mit dem dafür definierten TCP [4]. (rechts) Im ready2_educate enthaltener Würfel.	11
Abbildung 8: Hinteransicht der KR C4 Compact mit Anschlussbezeichnungen [9].	12
Abbildung 9: Aufbau der Software-Hierarchie des KR C4 [11].	13
Abbildung 10: (Links) Vorderansicht des SmartPADs. (Rechts) Hinteransicht des SmartPADs.	15
Abbildung 11: Beziehungen der in KUKA-Systemen verwendeten Koordinatensysteme [11].	16
Abbildung 12: Inline-Formular für die PTP-Bewegung.	19
Abbildung 13: Inline-Formular für die LIN-Bewegung.	20
Abbildung 14: Inline-Formular für die CIRC-Bewegung.	20
Abbildung 15: Darstellung der Bahnen für die Bewegungsarten PTP (l.o.), LIN (r.o.) und CIRC (unten) [4].	21
Abbildung 16: Aufbau eines Profinet-Frames bei der Echtzeit-Übertragung [16].	23
Abbildung 17: Darstellung der Verknüpfungen von CIP-Objekten [18].	24
Abbildung 18: (Oben) Ein Slave liest Daten aus einem EtherCAT-Frame. (Unten) Ein anderer Slave schreibt während der Laufzeit Daten in den Frame [27].	26
Abbildung 19: Darstellung der Kommunikation eines OPC UA Clients und Servers [37].	29
Abbildung 20: Beispiel für ein RSI-Kontext für die Kommunikation über Ethernet [39].	30
Abbildung 21: (L. o.) Darstellung einer Absoluten Bahnkorrektur. (R. o.) Darstellung der absoluten sensorgeführten Bewegung. (Unten) Darstellung der relativen sensorgeführten Bewegung [39].	31
Abbildung 22: Schema der Kommunikation des KUKA.Ethernet KRL Optionspaketes [40].	32
Abbildung 23: (Oben) Aufbau einer Leseanfrage an den KVP-Server für die Variable BSPVAR. (Mitte) Aufbau einer Schreibanfrage für BSPVAR auf den Wert 1. (Unten) Aufbau einer Antwort auf eine der beiden oberen Nachrichten.	33
Abbildung 24: Latenzzeiten zwischen senden einer Anfrage und Empfangen einer Antwort mit KukavarProxy [42].	34

Abbildung 25: Schema der Verwendung einer externen Bahnplanung mit JOpenShowVar [42].	36
Abbildung 26: Schaubild für die Interaktion zwischen den verschiedenen Teilprogrammen.	37
Abbildung 27: Flussdiagramm für das Abfahren einer Bahn.	39
Abbildung 28: Darstellung des Kommunikationsverlaufs beim Initialisierungsvorgang in Bahnskript.m.	40
Abbildung 29: Darstellung des Kommunikationsverlaufs beim Programmablauf in Bahnskript.m.	41
Abbildung 30: Veranschaulichung des Vorgangs zum Auswählen eines X-Koordinatenwertes von 24.4 mm.	42
Abbildung 31: Ablaufdiagramm des KRL-Programms driver während einer Bahnfahrt.	44
Abbildung 32: Ablaufdiagramm für die manuelle Bewegung in driver.	45
Abbildung 33: Schaubild für das Funktionsprinzip von Teachskript.m.	47
Abbildung 34: Ablaufdiagramm für Teachskript_Spacemaus.m.	49
Abbildung 35: 3D-Modell des Manipulators mit eingezeichneten Wegpunkten der Testbahn.	53
Abbildung 36: Zykluszeiten für jede Polling-Iteration während des Tests von fread/fwrite (1) und read/write (2). Es wurde im ersten Durchlauf nur FINISH (a) und danach zusätzlich die Ist-Position (b) angefragt.	54
Abbildung 37: Zykluszeiten für jede Polling-Iteration während des Tests der Java-(3) und .NET-Klassen (4). Es wurde im ersten Durchlauf nur FINISH (a) und danach zusätzlich die Ist-Position (b) angefragt.	56
Abbildung 38: Darstellung der Sequenz der angefahrenen Punkte in der Beispielapplikation 1.	57
Abbildung 39: (Links) Ist-Positionsverlauf des Endeffektors während der Würfelapplikation. (Rechts) Zykluszeiten des Pollings während der Würfelapplikation.	58
Abbildung 40: (Links) CAD-Modell der Greiferzangen. (Rechts) Aufbau im Arbeitsraum für die Flaschenapplikation.	59
Abbildung 41: (Links) Modell des Laboraufbaus und Ist-Positionsverlaufs. (Rechts) Zykluszeiten während des Pollings bei Ausführung der Applikation.	60
Abbildung 42: Manipulator in der Stellung der Überkopf- (o.l.), Strecklagen- (o.r.) und Handachsen-Singularität (unten) [11].	61
Abbildung 43: Bedeutung der einzelnen Bits eines Status- und Turn Wertes [48].	63
Abbildung 44: Darstellung der mit RST erstellten kinematischen Kette ohne (links) und mit (rechts) Visualisierung.	66
Abbildung 45: (Links) Ausschnitt aus dem Modellierungsteil des Simulink-Modells. (Rechts) Darstellung des Modells mit allen verwendeten Koordinatensystemen.	67
Abbildung 46: Darstellung des Lösungsbereichs für den Mittelpunkt des Kreises. Die orangene Ebene beinhaltet P1 und P2 und die violette Ebene enthält alle Punkte mit gleichem Abstand zu P1 und P2. Die weiße Linie zeigt die Lösungen für einen festgelegten X-Wert für den Mittelpunkt.	71
Abbildung 47: Ausschnitt aus der Animation der Bahnfahrt des Robotermodells.	72

Abbildung 48: Screenshot des Netzwerkkonfigurationsmenüs mit fest zugeteilter IP-Adresse.	82
Abbildung 49: Desktop des Windows-Betriebssystems auf dem SmartPAD und laufender KVP-Server.	83

Abkürzungsverzeichnis

AUT	Automatikbetrieb
AUT EXT	Automatikbetrieb extern
BFGS	Broyden-Fletcher-Goldfarb-Shanno
CAD	Computer-aided Design
CAN	Controller Area Network
CIP	Common Industrial Protocol
CIRC	kreisförmige Bewegung
DAT	Data File
DLL	Dynamic Link Library
DP	Dezentrale Peripherie
E/A	Ein-/Ausgabe
EDS	electronic data sheet
EtherCAT	Ethernet for Control Automation Technology
Ethernet/IP	EtherNet Industrial Protocol
FPS	Bildfrequenz, Frames per Second
GSD	General Station Description
GSDML	General Station Description Markup Language
GUI	Graphical User Interface
HOV	Hand-Override
ICT	Instrument Control Toolbox
IO	Input Output
KCB	KUKA Controller Bus
KEB	KUKA Extension Bus
KLI	KUKA Line Interface
KONI	KUKA Optional Network Interface
KPC	KUKA PC
KRL	KUKA Robot Language
KSB	KUKA System Bus
KSI	KUKA Service Interface
KVP	KukavarProxy
LIN	lineare Bewegung
ODVA	Open DeviceNet Vendor Association
OPC	Open Platform Communications
OSI	Open Systems Interconnection
PID	proportional-integrierend-differenzierend
PLA	Polyactid
POV	Program-Override
PTP	Point-to-Point
RSI	Robot Sensor Interface
RST	Robotic Systems Toolbox
SAK	Satzkoinzidenzfahrt
SM	Simscape Multibody
SPL	Spline-Bewegung
SPS	Speicherprogrammierbare Steuerung
SRC	Source Code
STL	Standard Triangle Language
STP	Standard for the Exchange of Product Model Data
T1	Testbetrieb 1

T2	Testbetrieb 2
TCP	Tool Center Point
TCP/IP	Transmission Control Protocol / Internet Protocol
UDP	User Datagram Protocol
WVS	WorkVisual Solution File
XML	Extensible Markup Language

1. Einleitung

1.1. Problemstellung

Roboter haben in der heutigen Welt eine wesentliche Bedeutung und sind aus unserem Leben nicht mehr wegzudenken. Durch den Einsatz z.B. in Produktionsprozessen garantieren sie eine zuverlässige und effiziente Herstellung von verschiedenen Gütern und sind somit eine Triebfeder der heutigen Wirtschaft. In den letzten Jahren wuchs deshalb stetig die Zahl der eingesetzten Industrieroboter wie in Abbildung 1 gezeigt.

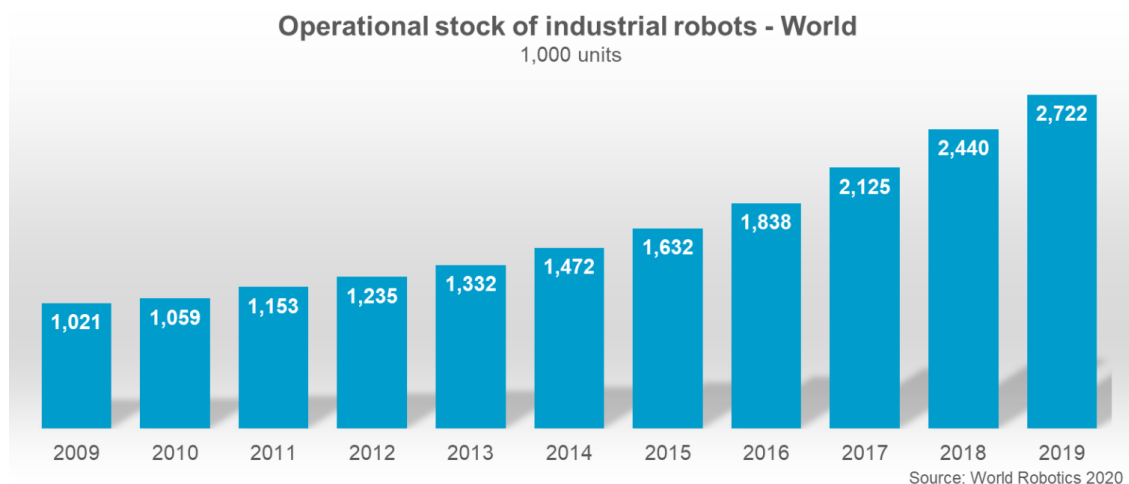


Abbildung 1: Anzahl der weltweit eingesetzten Industrieroboter von 2009 bis 2019 [1].

Aus diesen Gründen ist es wichtig, dass zukünftige Generationen auf die Arbeit und den Umgang mit Robotern und autonomen Systemen vorbereitet werden. An der Frankfurt University of Applied Sciences (UAS) wurde deshalb eine Anbindung eines Roboterarms der Firma KUKA AG an einen Labor-PC mit dem Programm MATLAB erstellt. Der PC ist in dem Laboraufbau an eine KUKA-Schulungszelle *ready2_educate pro* angebunden und es ist die MATLAB Version 2020a installiert. Es ist dann möglich, über MATLAB die Bewegung des in der Schulungszelle befindlichen Roboterarms zu steuern. Dies geschieht über eine *Transmission Control Protocol / Internet Protocol* (TCP/IP)-Kommunikation, welche in Kapitel 4 erklärt wird. Weiterhin wurde mit MATLAB der Roboterarm modelliert und ein Programm zur Darstellung der Armbewegung entworfen. Studierende haben dann so die Möglichkeit, im Labor erarbeitete Bewegungsabläufe später nachzuvollziehen oder auch schon Laboranwendungen vorzubereiten und zu testen.

1.2. Zielsetzung

Ziel dieser Thesis war eine Anbindung eines Roboterarms an einen Labor-PC mit dem Programm MATLAB zu erstellen. Über MATLAB-Skripte und -Funktionen soll die Bewegung des Roboterarms gesteuert werden können, um somit mehr Möglichkeiten für die Integration der Schulungszelle in den Lehrbetrieb an der Frankfurt UAS zu haben. Damit das System auch sinnvoll und in einem ausreichenden Umfang verwendet werden kann, ergeben sich die in der Tabelle 1 vorgestellten Anforderungen.

Feature-Nr.	Beschreibung	Wichtigkeit
1	Bewegungssteuerung	
1.1	Vorgabe von Sollpositionen, zu denen sich der Arm bewegt	++
1.2	Bewegungsführung von Hand	++
1.3	Speichern von Positionen & Bahnen	+
1.4	Anpassung von Bewegungen	+
1.5	Steuerung des Greifers	++
2	Rückgabewerte	
2.1	Rückgabe der Ist-Position	++
2.2	Rückgabe der Ist-Geschwindigkeit	-
2.3	Rückgabe der Ist-Beschleunigung	-
3	Weitere Eigenschaften	
3.1	Robuste Ausführung	++
3.2	Echtzeitverhalten	+
3.3	Benutzerfreundlichkeit	+
3.4	Installationsaufwand	-
4	Modellanforderungen	
4.1	Darstellung der Armbewegung	+
4.2	Simulation des Greiferverhaltens	-

Tabelle 1: Anforderungstabelle an die Lösungsimplementierung. „++“ steht für sehr wichtig, „+“ für wichtig und „-“ für weniger wichtig.

Die Anforderungen und deren Gewichtung wurden hinsichtlich des Laborbetriebes mit Studierenden und dem Entwurf von zukünftigen Regelungssystemen gestellt, in welche der Roboterarm dann integriert werden kann. Um die Anforderungen zu erfüllen, wurde deshalb in dieser Thesis wie folgt vorgegangen.

1. Recherche zu den Komponenten und der Funktionsweise des KUKA-Systems
2. Vergleich der Anbindungsmöglichkeiten
3. Implementierung der Lösung
4. Erstellung von Applikationen zum Testen der Lösung

Die Ergebnisse von Punkt 1 werden in Kapitel 2 vorgestellt. Die meisten Informationen wurden aus Datenblättern und Bedienungsanleitungen entnommen, die von KUKA kostenfrei zur Verfügung gestellt werden. In Kapitel 3 wird Punkt 2 behandelt, wo die Funktionsweisen der Feldbusse vorgestellt und der Aufwand der Anbindungen an den Labor-PC verglichen, sowie vorhandene Anbindungsimplementierungen vorgestellt werden. Diese dienen als Referenz für die Bearbeitung von Punkt 3, dessen Ergebnisse in Kapitel 4 vorgestellt werden. In Kapitel 5 sind die unter Punkt 4 genannten Applikationen gezeigt. Mit ihnen sind Beispiele für die Erarbeitung eigener Anwendungen gegeben und es wurde so die Funktionalität der in dieser Thesis erarbeiteten Lösung bestätigt. Die Erarbeitung des Programms zur Darstellung der Armbewegungen wird in Kapitel 6 vorgestellt. Die Dateien mit dem in der Thesis erarbeiteten Code sind Teil der Abgabe und bei Nennung einer Code-Datei im weiteren Bericht wird deshalb auf den Anhang in Form einer WinRAR-Datei verwiesen.

2. Grundlagen

Um eine Lösung für die Anbindung des Roboters zu implementieren, ist ein gewisses Verständnis von den Komponenten und deren Funktionsweise nötig. In diesem Kapitel werden deshalb die Theorie hinter der Roboterbewegung und die verschiedenen Bestandteile der Roboterzelle im Labor vorgestellt. Es wird eine allgemeine Beschreibung gegeben und nur bestimmte Details beschrieben, die für die spätere Lösungsimplementierung auch relevant sind.

2.1. Kinematik

Bevor der eigentliche Roboterarm vorgestellt wird, sind in diesem Abschnitt die theoretischen Grundlagen eines Manipulators vorgestellt. Ein Manipulator ist dabei ein von Hand geführtes Hebelgestänge, welcher sich nicht von selbst bewegen kann [2]. Durch das Verwenden eines Steuerungs- oder Regelungssystems wird dann der Manipulator zu einem Roboter. In dieser Thesis wird der Begriff Roboterarm und Manipulator synonym verwendet. Ein Roboterarm lässt sich generell als eine Anordnung von Gelenken und starren Körpern, welche nicht verformbar sind, beschreiben. Bei so einer kinematischen Kette existieren verschiedene Gelenktypen, meistens werden jedoch rotatorische oder translatorische Gelenke verwendet. Der Roboterarm im Labor hat z.B. sechs Rotationsgelenke, die nacheinander verbunden sind, weshalb diese Konfiguration auch als serielle Kinematik bezeichnet wird. Die Untersuchung der möglichen Anordnungen von Gelenken und starren Körpern, sowie deren Bewegung wird dem Begriff Kinematik zugeordnet. Werden Energie und Kräfte dabei noch mitberücksichtigt, ist dann die Dynamik eines Körpers gemeint, welche jedoch in dieser Thesis nicht weiter behandelt wird, da durch die Regelung der Robotersteuerung im Labor kein gesonderter Umgang damit notwendig ist.

Die Stellung eines Manipulators wird durch die Positionswerte seiner Gelenke oder durch die Lage seines Endeffektors, also des letzten starren Körpers in der kinematischen Kette, in einem Koordinatensystem, z.B. dem kartesischen beschrieben. Die Umrechnung der Gelenkwerte auf die Endeffektorposition wird als direkte oder Vorwärtskinematik bezeichnet und die umgekehrte Berechnung als inverse Kinematik. Bei der Vorwärtskinematik geschieht dies durch die schrittweise Transformation der Position der Basis, also des ersten starren Körpers einer Kette, entlang den Gelenkpositionen. Diese werden meist mit der Position des vorheri-

gen Gelenks als Bezugssystem ausgedrückt und mithilfe von homogenen Transformationsmatrizen dargestellt, welche wie folgt aufgebaut sind.

$${}^{(A)}T(q)_{(B)} = \begin{bmatrix} {}^{(A)}R(q)_{(B)} & {}^{(A)}t(q)_{(B)} \\ 0 & 1 \end{bmatrix} \quad (1)$$

${}^{(A)}R_{(B)}$ ist hier die 3 x 3 Rotationsmatrix vom Koordinatensystem B zum System A und ${}^{(A)}t_{(B)}$ ist der entsprechende Translationsvektor. Die homogene Matrix ist hier in Abhängigkeit vom Wert eines Gelenks q angegeben. Um die Endeffektorposition in Bezug auf das Ursprungskoordinatensystems $x_{TCP(0)}$ anzugeben, werden die homogenen Transformationsmatrizen der Gelenke 1-6 eines Manipulators miteinander wie folgt multipliziert.

$${}^{(0)}T(\bar{q})_{(6)} * x_{TCP(6)} = ({}^{(0)}T(q_1)_{(1)} * {}^{(1)}T(q_2)_{(2)} * \dots * {}^{(5)}T(q_6)_{(6)}) * x_{TCP(6)} = f(\bar{q}) \quad (2)$$

Mit $\bar{q}$ ist die Gesamtkonfiguration der Gelenke ausgedrückt. Für die Steuerung eines Roboterarms wird jedoch meistens eine Sollposition für $x_{TCP(0)}$ vorgegeben, die es mit den richtigen Gelenkwerten zu erreichen gilt. Dafür wird die Inverse der direkten Kinematikfunktion $\bar{q} = f^{-1}(x_{TCP(0)})$ gebildet. Die Lösung dafür ist meist schwieriger zu berechnen und es können folgende Ansätze verwendet werden.

- analytisch
- geometrisch
- numerisch

Der analytische und geometrische Ansatz funktionieren jedoch oft nur bei einfachen kinematischen Ketten mit wenig Freiheitsgraden und außerdem ist solch eine Berechnung speziell nur für einen Manipulator ausgelegt. Das Ergebnis ist dann wie bei der direkten Kinematik eindeutig, bei mehreren Freiheitsgraden kann aber eine Sollposition $x_{TCP(0)}$ durch mehrere Gelenkstellungen $\bar{q}$ erreicht werden, welche bei dem analytischen Ansatz unberücksichtigt bleiben. Deshalb wird i.d.R. die Lösung numerisch berechnet, denn ein Algorithmus dafür kann, u. a. auf verschiedene kinematische Ketten angewendet werden und mehrere Lösungen ausgeben, aus denen sich die beste auswählen lässt. Wie bei allen numerischen Berech-

nungen sind die Lösungen jedoch nur Annäherungen an den wahren Wert, und eine zu große Abweichung davon kann je nach Anwendung zu Problemen führen.

Zur Berechnung der numerischen Lösung werden häufig die Differenz der Positionen und Gelenkwerte zwischen dem Soll- und Ist-Wert einer Bewegung $\Delta \bar{q} = (f^{-1})'(\Delta x_{TCP(0)})$ betrachtet. Dadurch ergibt sich folgende Gleichung zur Berechnung der Sollgelenkwerte [3].

$$\bar{q} = \bar{q}_0 + J^{-1}(\bar{q}_0) * (x_{TCP(0)} - f(\bar{q}_0)) \quad (3)$$

$\bar{q}_0$ und $f(\bar{q}_0)$ sind die Ist-Werte für Gelenke und Position und J ist die Jacobi-Matrix von f , welche alle Gleichungen von f nach allen Variablen, in diesem Fall die Freiheitsgrade des Manipulators, ableitet. Die Anzahl der numerischen Lösungen hängt dadurch von der Konfiguration des Manipulators ab. In einem dreidimensionalen Raum ist bei weniger als sechs Freiheitsgraden oft keine Lösung möglich, da dann die Jacobi-Matrix nicht vollen Rang hat und somit nicht invertierbar ist. Bei genau sechs Freiheitsgraden existieren meist eine oder mehrere Lösungen und bei mehr als sechs Freiheitsgraden existieren unendlich viele Gelenkstellungen. Solche Manipulatoren werden als redundant bezeichnet und es sind dann sogenannte Nullraumbewegungen möglich, wo bei einer Bewegung die Endeffektorposition unverändert bleibt. Dies ist hilfreich, um Singularitäten zu vermeiden. Singularitäten sind dabei bestimmte Gelenkkonfigurationen, bei denen die Bewegungsachsen von zwei Gelenken übereinander liegen. Dadurch fällt ein Freiheitsgrad bei dem Manipulator weg. Im Falle eines Roboterarms mit sechs Drehgelenken existieren drei singuläre Stellungen welche in Abschnitt 5.3 behandelt werden, bei denen dann keine eindeutige Lösung für die Jacobi-Matrix berechnet werden kann.

Darstellung von Transformationen

Wie vorhin erwähnt lässt sich eine Rotation im dreidimensionalen Raum mit einer 3x3 Rotationsmatrix ausdrücken. Da aber dafür neun Parameter bestimmt werden müssen, wird stattdessen eine Reihenfolge von Elementarrotationen um die Basisachsen des Koordinatensystems festgelegt und die Winkel dieser Rotationen zur Angabe verwendet. Die Matrizen dieser Elementarrotationen hängen nur von einem Wert ab, weshalb eine beliebige Rotation mit nur drei Winkeln ausgedrückt werden kann. Bei KUKA-Systemen werden z.B. eine Varia-

tion der Euler-Winkel verwendet, wo zuerst um die Z-Achse, dann um die neue Y-Achse und schließlich um die wiederum neue X-Achse rotiert wird, wie in Abbildung 2 gezeigt.

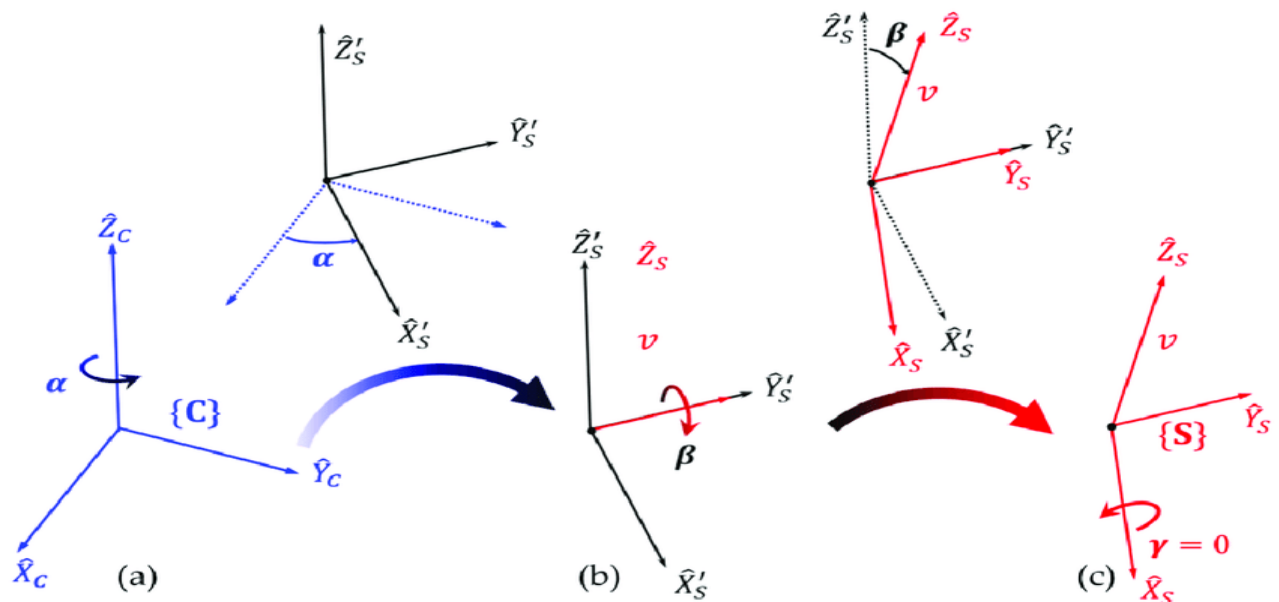


Abbildung 2: Darstellung einer Z-Y-X Rotation [4].

Alternativ lässt sich eine Rotation mit Quaternionen darstellen. Diese bestehen aus vier reellen Werten, welche als beliebiger dreidimensionaler Vektor gesehen werden können, um den mit dem vierten Wert ein Winkel für eine Rotation angegeben wird. Wegen ihrer numerischen Stabilität, d.h. ihren geringen Einfluss auf die numerische Lösung bei Wertungenauigkeiten, werden sie vor allem bei Robotersteuerungen eingesetzt.

Eine weitere Reduzierung der Transformationsparameter von sechs auf vier ist mit den Denavit-Hartenberg-Parametern möglich. Diese legen eine bestimmte Reihenfolge von Schritten vor, mit welchen die lokalen Koordinatensysteme einer kinematischen Kette transformiert werden, und lauten wie folgt [3].

1. Rotation um die Basis-Z-Achse mit θ , sodass die X-Achse vom Basis- und Folgesystem parallel verlaufen.
2. Bewegung entlang der Basis-Z-Achse mit dem Wert d.
3. Bewegung entlang der Basis-X-Achse mit dem Wert a.

4. Rotation um die Folge-X-Achse mit α , sodass die Folge-Z-Achse parallel zur Bewegungsachse des Gelenks ist.

In Abbildung 3 sind diese Schritte dargestellt. Damit ist eine einfachere Darstellung von Manipulatorkonfigurationen möglich, was z.B. die Berechnung der direkten Kinematik erleichtert. Bei ausschließlich vorliegenden Rotationsgelenken sind zudem die Parameter a , d und α bei Bewegungen des Manipulators konstant. In Abbildung ist DH-Transformation gezeigt.

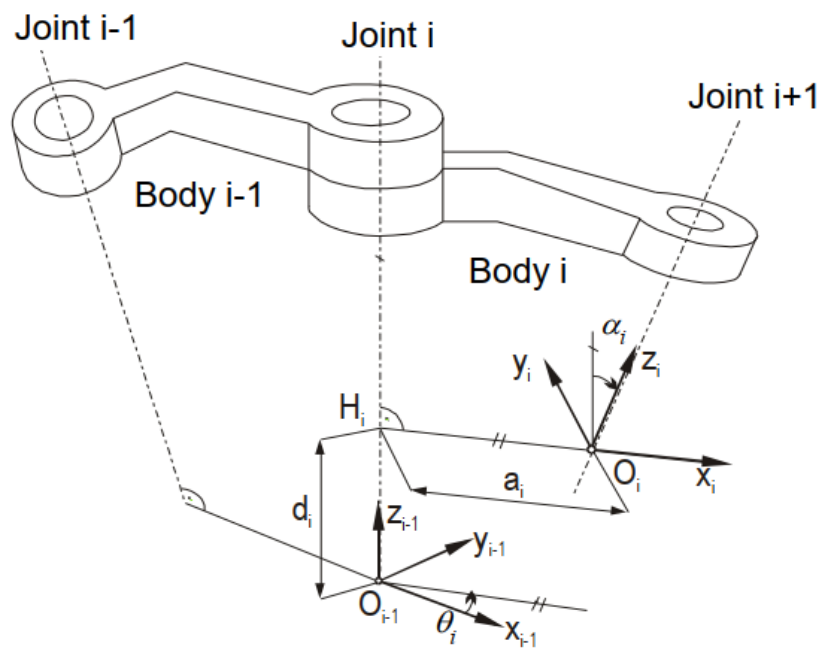


Abbildung 3: Darstellung der Denavit-Hartenberg-Transformation zwischen zwei lokalen Koordinatensystemen [5].

Steuerung

Wie vorher erwähnt, bildet der Manipulator den mechanischen Teil des Roboters. Für eine Bewegung ist das richtige Setzen von Signalen notwendig, was eine Robotersteuerung übernimmt. Diese ist einerseits für die Regelung der einzelnen Aktoren zuständig, sowie für die zentrale Planung der Bewegung. Ein Schaubild dafür ist in Abbildung 4 gezeigt. Auf der unteren Ebene bilden die elektromechanischen Komponenten der einzelnen Gelenkmotoren verzögernde Elemente, die zu einem klassischen Regelkreis mit z.B. PID- (proportional-integrierend-differenzierend) Reglern geschlossen werden. Hinzu kommen noch Messglieder in der Rückkopplung und weitere Komponenten zur Signalaufbereitung wie z.B. Filter. Da Manipulatoren aus mehreren Gelenken bestehen, werden die Soll- und Ist-Werte dieser Ein-

zelachsenregelungen in einem Bahnplanungsalgorithmus zusammengeführt, der wie vorhin beschrieben die direkte und damit dann die inverse Kinematik berechnet. Programme und auch Eingaben vom Benutzer wirken dann auf diese Planung ein. Da beim Arbeiten mit Robotern eine Schadensgefahr ausgeht, besitzt eine Robotersteuerung i.d.R. sicherheitsrelevante Funktionen, welche sowohl auf die zentrale Planung als auch auf die untergeordnete Regelung zugreifen können, um im Ernstfall die Aktoren rechtzeitig zu stoppen. Für eine funktionierende Steuerung ist es notwendig, dass diese in Echtzeit arbeitet. Ein System arbeitet dann in Echtzeit, wenn es die Verarbeitung von Daten innerhalb einer bestimmten Zeitspanne garantieren kann. Je nach Anwendung unterscheidet sich dieser Wert, beim KUKA-System im Labor ist er 12 ms.

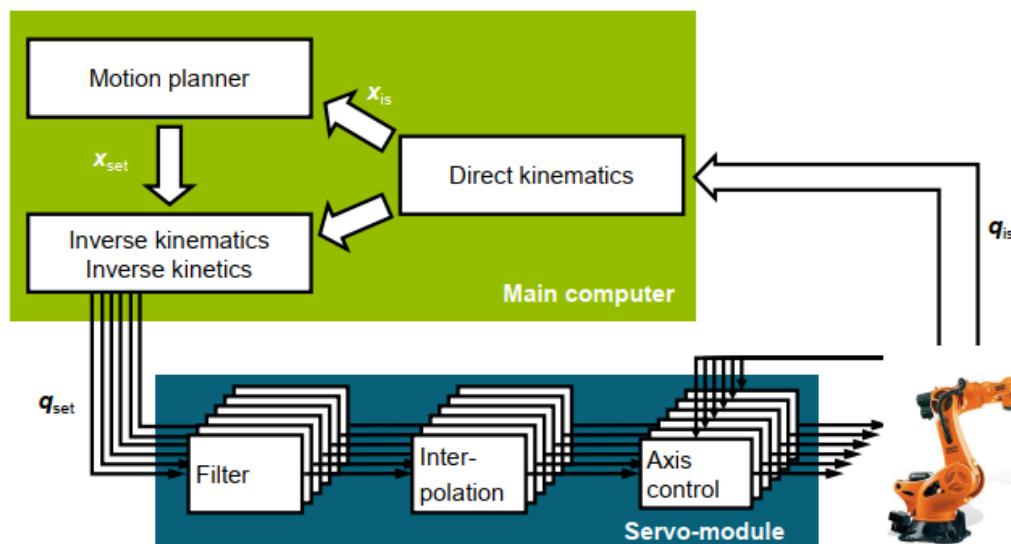


Abbildung 4: Schaubild für den Aufbau einer Robotersteuerung [3].

2.2. Manipulator

In dieser Arbeit wurde ein Roboterarm des Modells AGILUS KR 3 R540 von KUKA verwendet [6]. Aus der Modellbezeichnung gehen eine Nennt Tragfähigkeit von 3 kg und eine maximale Reichweite von 540 mm hervor. Wie in Abschnitt 2.1 erwähnt hat der Arm sechs rotierende Gelenke. Der Bewegungsbereich der Achsen ist in Abbildung 5 gegeben. Durch seine Bauform kommt der AGILUS auf ein Gewicht von 26,5 kg. An den Anbauflansch lassen sich verschiedene Endeffektoren anbringen, je nach Werkzeug ist aber zu beachten, dass durch die Verschiebung des Schwerpunktes sich eine reduzierte Traglast ergibt. In [6] ist ein entsprechendes Traglastdiagramm als Referenz gegeben.

Bewegungsbereich	
A1	$\pm 170^\circ$
A2	$-170^\circ / 50^\circ$
A3	$-110^\circ / 155^\circ$
A4	$\pm 175^\circ$
A5	$\pm 120^\circ$
A6	$\pm 350^\circ$

Abbildung 5: Bewegungsbereich der einzelnen Achsen des KR3 R540 [6].

Der KR3 besitzt eine Positionswiederholungsgenauigkeit von 0,02 mm. Dieser Wert beschreibt die Streuung der Abweichung der Ist-Position von einem programmierten Pfad, welcher mehrmals hintereinander durchfahren wird. Somit bietet der KR 3 bei zyklischen Arbeitsabläufen eine hohe Präzision. Die Positionsabsolutgenauigkeit, also die durchschnittliche Abweichung der Ist-Position beim Anfahren zufälliger Punkte, beträgt ca. 0,3 mm [7]. Durch die abgerundete und geschlossene Form des Gehäuses ergeben sich wenige Störkonturen, was den Arm beim Arbeiten auf engem Raum sicherer und deshalb für den Lehrbetrieb im Labor interessant macht. In Abbildung 6 ist ein Bild von dem Manipulator selbst gegeben.

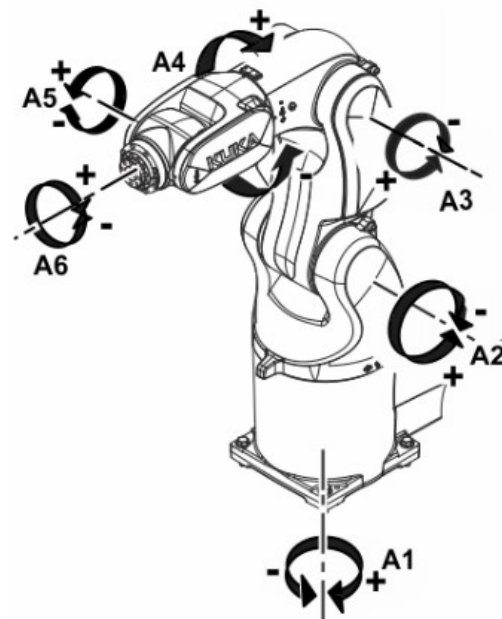


Abbildung 6: (Links) Seitliche Aufnahme des Manipulators. (Rechts) Drehrichtungen der einzelnen Gelenkachsen [6].

2.3. Greifer

Als Endeffektor ist an dem Manipulator zurzeit ein Parallelgreifer montiert, welcher als Teil des ready2_educate-Pakets mitgeliefert [8]. Dieser wird pneumatisch mit einem Druck von 6-8 bar betrieben und hat nur zwei Schaltzustände, nämlich offen und geschlossen. Weitere Sensorik z.B. zur Erfassung der Greifkraft besitzt er nicht. Alternativ lassen sich auch die Befehle des Optionspakets KUKA.GripperSpotTech verwenden. Damit ist es auch möglich, den Greifer verzögert zu betätigen und auf Fehlstellungen zu prüfen. In Abbildung 6 ist der Greifer mit dem von KUKA festgelegten *Tool Center Point* (TCP) gezeigt. Dieser wird über folgende homogene Transformation erreicht:

$$T_{07} = T_{06} * \begin{pmatrix} 0.482 & 0.017 & -0.876 & 31.67 \\ -0.026 & 1 & 0.005 & -26.94 \\ 0.876 & 0.021 & 0.482 & 117.13 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

Die Angaben für die Translation stehen in mm. Im geöffneten Zustand haben die Backen einen Abstand von 3,1 cm und im geschlossenen Zustand 1,9 cm. Weiterhin stehen die Backen nur 1,8 cm hervor. Es ist ersichtlich, dass der Greifer für das Greifen der im ready2_educate-Paket mitgelieferten 2,5cm großen Würfel ausgelegt wurde und deshalb nur eingeschränkt Anwendungen mit anderen Gegenständen möglich sind. In Abbildung 7 ist der Greifer und solch ein Würfel gezeigt.



Abbildung 7: (links) Draufsicht des Greifers mit dem dafür definierten TCP [4]. (rechts) Im ready2_educate enthaltener Würfel.

2.4. Steuerung

Zur Steuerung des Manipulators wird das Steuerungssystem KR C4 Compact von KUKA verwendet [9]. Diese ist mit seinen geringen Abmessungen und einer 19-Zoll-Bauform für den Einbau Schaltschränke ausgelegt. Dementsprechend ist die Steuerung im Inneren der Roboterzelle positioniert. Die KR C4 ist für die Trajektorienberechnung, die Ansteuerung der Gelenkaktoren, sowie für die Überwachung und Ausführung sicherheitsrelevanter Aufgaben, wie z.B. Not-Halt, zuständig. Zudem bietet sie verschiedene Schnittstellen für die Anbindung von weiteren externen Komponenten wie zusätzliche Robotersteuerungen, speicherprogrammierbaren Steuerungen (SPS) oder Sensoren. Die Steuerung arbeitet mit der KUKA-internen Sprache KRL, kann aber auch PLC-Sprachen und den bei CNC-Maschinen verwendeten G-Code interpretieren. Die KR C4 ist in Abbildung 8 dargestellt.

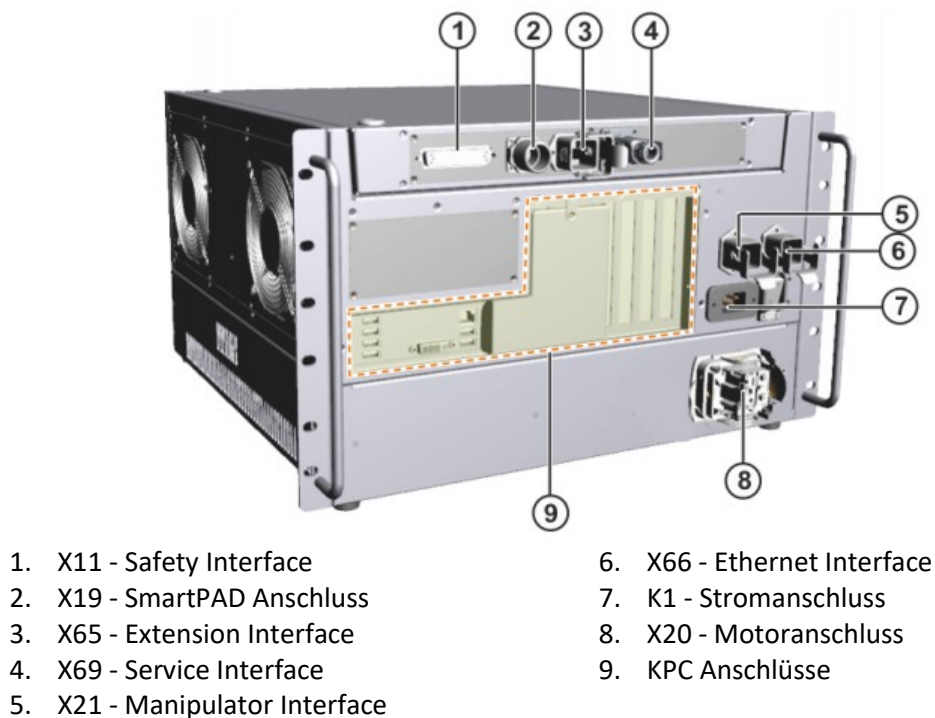


Abbildung 8: Hinteransicht der KR C4 Compact mit Anschlussbezeichnungen [9].

Die Software-Architektur der KR C4 ist modular aufgebaut. Es laufen dabei verschiedene Applikationen auf der Steuerung wie in Abbildung 9 dargestellt. Die systemrelevanten Applikationen *Robot Control*, welches die Bahnplanung übernimmt und *Safety*, welches für die Sicherheitssteuerung zuständig ist, werden ständig ausgeführt. Die Ausführung der Applikati-

onen wird vom *KUKA PC* (KPC) übernommen, welcher Teil der Steuerung ist. Es lassen sich dazu noch weitere optionale Applikationen ergänzen, wie z.B. das vorher erwähnte Paket *KUKA.GripperSpotTech* oder *KUKA.RobotSensorInterface*, womit Daten in Echtzeit mit externen Sensoren ausgetauscht werden können und in Abschnitt 3.2 vorgestellt wird. Die KR C4 regelt dann den Datenaustausch zwischen diesen Apps und vorhandenen Schnittstellen. Es existieren sechs interne Bussysteme, über die weitere Hardware an die Steuerung angeschlossen werden kann. Diese sind alle Ethernet-basiert und in Tabelle 2 vorgestellt. Die Anschlussbezeichnungen beziehen sich auf die physikalischen Schnittstellen, die sich auf der Hinterseite der Steuerung befinden, wie in Abbildung 8 zu sehen.

Bussystem	KLI (KUKA Line Interface)	KSI (KUKA Service Interface)	KONI (KUKA Optional Network Interface)	KEB (KUKA Extension Bus)	KCB (KUKA Controller Bus)	KSB (KUKA System Bus)
Anbindung von:	-SPS -Feldbusse -Ethernet-Netzwerke	WorkVisual-Notebook zur Diagnose	Kameras (nur mit Kuka.VisionTech)	EtherCAT-Geräte	Antriebskomponenten	Kuka-eigene Hardware
Anschluss	X66/X67	X69/USB	X66	X44/X65	X13/X11	X13/X11

Tabelle 2: Vorstellung der vom KR C4 verwendeten internen Bussysteme [10].

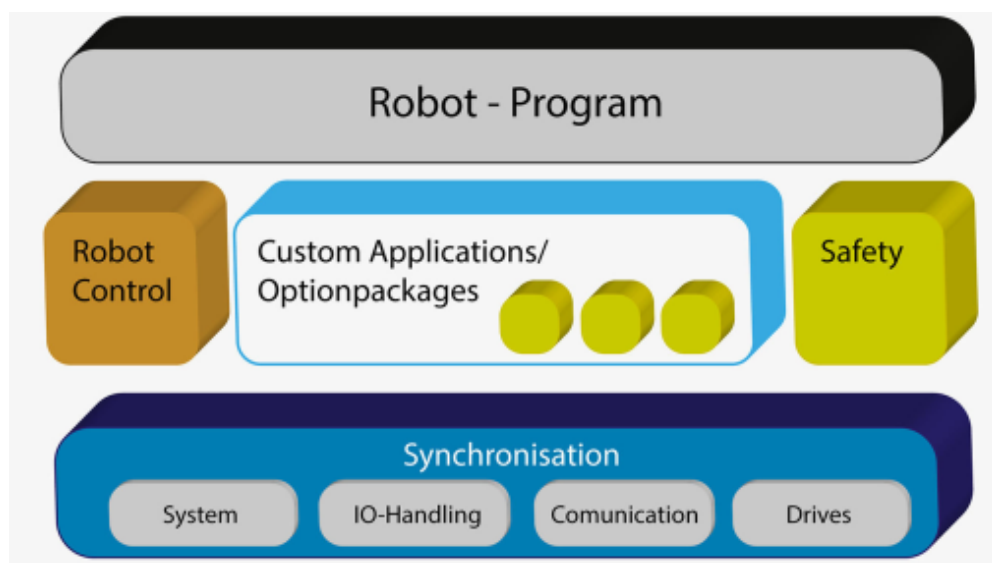


Abbildung 9: Aufbau der Software-Hierarchie des KR C4 [11].

2.5. SmartPAD

Die Bedienung und Programmierung der Steuerung erfolgt i.d.R. durch das KUKA SmartPAD [11]. Dieses ist ein Bedienhandgerät, welches Tastenelemente, ein Touch-Screen und eine 6D-Maus zur Bewegung des Manipulators bereitstellt. Es können dabei entweder jeweils die einzelnen Achsen angesteuert werden, oder die Position des Endeffektors in kartesischen Koordinaten geändert werden. Die KR C4 berechnet dann die dafür nötigen Achswinkel. Zudem befindet sich ein Not-Halt-Knopf auf dem SmartPAD und mehrere Zustimmungstasten, die als zusätzliches Sicherheitsfeature dienen. So muss für die manuelle Bedienung des Arms eine der Tasten in Mittelstellung durchgängig gedrückt werden, damit dieser sich bewegt. Weiterhin sind noch Tasten zum Starten und Stoppen von Programmabläufen und frei konfigurierbare Tasten für z.B. die Steuerung von Werkzeugen vorhanden. Eine Vorder- und Rückansicht des SmartPADs ist in Abbildung 10 dargestellt.

Auf der Steuerung läuft einerseits das Echtzeit-Betriebssystem *VxWorks* und andererseits eine embedded-Version von Windows 7. Diese Kombination wird von KUKA auch als *VxWin* bezeichnet verbindet so die Vorteile von beiden Systemen [12]. Die GUI auf dem SmartPAD wird dabei als Windows-Programm ausgeführt. Die Zugriffsrechte auf Dateien und Einstellungen werden durch fünf Benutzergruppen geregelt. So kann die unterste Gruppe „Bediener“ nur KRL-Skripte auswählen und starten, während die Gruppe „Administrator“ auch die Zugriffsrechte für die anderen Gruppen ändern kann. Der Zugang zu den Gruppen erfolgt außer für „Bediener“ durch ein Passwort und wenn eine Gruppe eine Zeit lang inaktiv ist, wird automatisch zu „Bediener“ gewechselt. Dies verhindert einen Missbrauch des Systems durch Unbefugte und eignet sich somit auch zur Anwendung in der Lehre.



Abbildung 10: (Links) Vorderansicht des SmartPADs. (Rechts) Hinteransicht des SmartPADs.

Im *Graphical User Interface* (GUI)-Programm lässt sich aus einem von verschiedenen Profilen für Werkzeug und Basis wählen. Dies ist für die Ausgabe der Ist-Position relevant, da diese als TCP-Position des Werkzeugs in Bezug zum Koordinatensystem der gewählten Basis berechnet wird. Es können Profile von jeweils 16 Basen und Werkzeugen gespeichert werden. Darin werden dann die Werte für die jeweiligen Koordinatensysteme in Bezug zum Weltkoordinatensystem bzw. dem Flanschkoordinatensystem angegeben. In Abbildung 11 ist ein Schaubild mit den von KUKA verwendeten Koordinatensystemen gegeben. Im Falle der im Labor verwendeten Zelle wird als Basis das Weltkoordinatensystem \$WORLD verwendet, welches gleich dem Fußpunkt des Manipulators \$ROBROOT ist. Für den TCP wird jedoch das Koordinatensystem des Endeffektors \$NULLFRAME verwendet, da die in Abschnitt 2.3 gezeigte Transformation eine leichte Seitwärtsverschiebung ergibt, was das Arbeiten mit dem Werkzeug erschwert.

Weiterhin lässt sich im GUI-Programm die Verfahrensgeschwindigkeit beim manuellen Steuern und bei Skriptausführung einstellen. Dies wird als *Hand-Override* (HOV) bzw. *Program-Override* (POV) bezeichnet und wird zusätzlich zu eventuell vorprogrammierten Geschwindigkeiten im Skript angewendet. Die Angaben werden in Prozent gemacht und beziehen sich bei *Point-to-Point* (PTP)-Bewegungen auf die maximalen Rotationsgeschwindigkeiten der

einzelnen Achsen. Bei Bahnbewegungen beziehen sie sich auf einen Maximalwert von 3 m/s. Es existieren zudem vier verschiedene Betriebsarten, welche in Tabelle 3 vorgestellt sind.

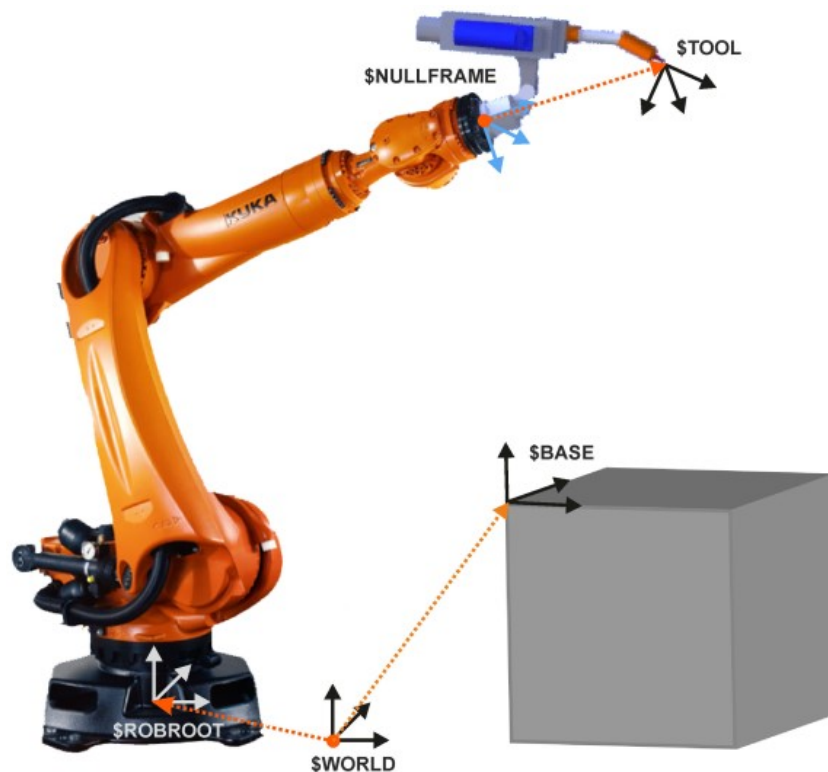


Abbildung 11: Beziehungen der in KUKA-Systemen verwendeten Koordinatensysteme [11].

Betriebsart	T1	T2	AUT	AUT EXT
Bezeichnung	Testbetrieb 1	Testbetrieb 2	Automatik	Automatik Extern
Programm-ausführung	Zustimm- + Starttaste durchgedrückt		Starttaste nur einmal drücken	Starttaste einmal drücken oder Externes Signal
Eigenschaften	• offener Arbeitsraum möglich • max. 0,25 m/s	• Arbeitsraum muss geschlossen sein • max. 3 m/s • keine manuelle Bewegung möglich		
Verwendung	• sicherem Testen von Programmen • Einteachen von Bahnen	• sicherem Testen von Programmen	• Abarbeitung von Programmen	

Tabelle 3: Vorstellung der vier Betriebsarten auf dem KRC4.

Bestimmte Systemeinstellungen wie z.B. die derzeit installierten Optionspakete oder Parameter für den Anschluss von Feldbussen werden in einer Konfigurationsdatei im *WorkVisual Solution File* (WVS)-Format gespeichert, die dann im GUI-Programm des SmartPADs geladen wird. Diese Dateien werden mit dem Programm WorkVisual erstellt und bearbeitet, welches kostenfrei von KUKA angeboten wird [13].

2.6. KRL

Die *KUKA Robot Language* (KRL) ist eine proprietäre prozedurale Programmiersprache, die auf PASCAL basiert [14]. Programme und Bewegungsabläufe, welche von der Steuerung ausgeführt werden, sind mit KRL geschrieben. Dabei besteht ein KRL-Programm aus zwei Dateien in den Formaten *Source Code* (SRC) und *Data File* (DAT). Die SRC-Datei enthält die eigentlichen Befehle, welche meist Bewegungen zu einer Zielkoordinate sind, oder auch weitere typische Logikbefehle wie Schleifen oder Variablenzuweisungen sein können. In der DAT-Datei werden Variablen deklariert und ggf. initialisiert. Die Werte werden dann auch zwischen Programmaufrufen erhalten. So werden z.B. Zielpunkte, welche im Teach-in-Verfahren erstellt werden, in dieser Datei gespeichert. Es können zwar auch Variablen in der SRC-Datei deklariert werden, diese sind aber dann nur zur Laufzeit verfügbar. Die SRC und DAT-Dateien, die zum selben Programm gehören, müssen denselben Namen haben. In Quelltext 1 ist ein Beispiel für die Syntax eines Programmcodes gegeben.

```
DEFDAT BspProg
  REAL C
  POS P1 = {X 200,Y 0,Z 600,A 20,B 20,C 0,S 2,T 8}
ENDDAT
```

```
DEF BspProg ( )
  INT A,B
  A=1
  B=2
  C=A/B
  PTP P1
END
```

Quelltext 1: (Links) Beispiel für eine DAT -Datei. (Rechts) Beispiel für eine zugehörige SRC-Datei.

In KRL werden vier Grunddatentypen unterstützt, nämlich INT, REAL, BOOL und CHAR. Aus diesen lassen sich noch weitere Datentypen mithilfe von *Strucs* und *Enums* erstellen. So gibt es z.B. für die Angabe von Positionen die vordefinierten *Strucs* AXIS und POS, welche wie folgt definiert sind:

STRUC AXIS REAL A1, A2, A3, A4, A5, A6

STRUC POS REAL X, Y, Z, A, B, C, INT S, T

Die Felder A1-A6 stehen für die Rotationswinkel der Robotergelenke in Grad. Bei der POS-*Struc* bilden X, Y, Z die kartesischen Koordinaten des Endeffektors in mm und A, B, C die Winkel der Z-, Y- und schließlich X- Rotation des Endeffektor-Frames in Grad. Diese müssen nicht zwangsläufig Absolutwerte sein und können auch je nach gewähltem Bezugssystem und Programmbefehl relative Angaben sein. S und T stehen für Status und Turn und legen eine eindeutige Stellung der Achswinkel fest, falls eine Position durch mehrere Stellungen erreicht werden kann. Weiterhin lassen sich in KRL noch Unterprogramme und Funktionen definieren. Eine Funktion unterscheidet sich dabei von einem Unterprogramm, indem diese einen Rückgabewert hat. Der Geltungsbereich von Programmstrukturen, also Variablen und Unterprogramme, beschränkt sich auf das Programm, in dem sie deklariert wurden, mit dem Schlüsselwort GLOBAL aber lassen diese sich auch von anderen KRL-Programmen aus erreichen. Durch Unterprogramme lässt sich Code besser organisieren und auch eine Routine für Interrupts festlegen.

Es gibt in KRL Funktionen für vier verschiedene Bewegungsarten, nämlich *Point-to-Point* (PTP), lineare (LIN), kreisförmige (CIRC) und Spline-Bewegungen (SPL). Da die Funktionen unterschiedliche Parameter je nach Bewegungsart erwarten, werden im Folgenden zum besseren Verständnis des in Kapitel 4 beschriebenen Codes die Eigenschaften der Bewegung an sich und der Aufbau des Bewegungsbefehls in KRL beschrieben. Die Befehle werden dabei als Inline-Formular gezeigt, welches eine vereinfachte Darstellung im GUI-Programm ist. Diese Formulare ermöglichen eine graphische Eingabe der Befehlsparameter und es werden zudem nicht eingetragene Parameter automatisch in dem DAT-Teil des Skripts initialisiert, womit sich deren Benutzung besonders für Anfänger eignet.

PTP:

Die PTP-Bewegung fährt den Zielpunkt über die schnellste Strecke an. Diese ist aber nicht unbedingt die kürzeste und auch nicht wirklich voraussehbar, weshalb bei einer PTP-Fahrt ein größeres Kollisionsrisiko besteht. Wie in Abbildung 15 zu sehen, ergeben sich meist geschwungene Bahnen aufgrund der rotierenden Achsen des Roboters. Der Aufbau des KRL-Befehls für diese Bewegung ist in Abbildung 12 gezeigt.

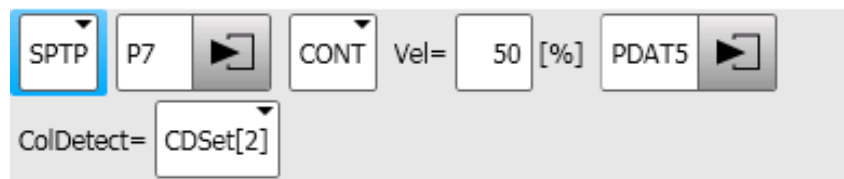


Abbildung 12: Inline-Formular für die PTP-Bewegung.

Als Argumente erwartet der Befehl einen Zielpunkt, hier P7, und einen Bewegungsdatensatz, hier PDAT5. Mit dem Datensatz werden weitere Bewegungsoptionen wie maximale Beschleunigung und Orientierung des TCP während der Fahrt festgelegt. Weiterhin wird für den Befehl die Geschwindigkeit gesetzt. Im Falle einer PTP-Bewegung wird wie schon in Abschnitt 2.5 erwähnt diese als relative Prozentangabe zur maximalen Achsgeschwindigkeit angegeben. Weitere Befehlsparameter sind CONT, mit welchem ein Überschleifen des Zielpunktes eingestellt werden kann, und ColDetect, welcher die Kollisionserkennung während der Bewegung steuert. Das Überschleifen bewirkt, dass der Manipulator den Zielpunkt nicht genau anfährt, sondern schon vorher die Bewegung als abgeschlossen registriert und sich zum nächsten Zielpunkt bewegt. Dadurch ist keine starke Reduzierung der Geschwindigkeit notwendig und die Bewegung ist flüssiger.

LIN:

Die LIN-Bewegung fährt eine Gerade vom Start- zum Zielpunkt. Im Vergleich zur PTP-Bewegung ist TCP-Position voraussehbar, jedoch gilt dies nicht für die restlichen Teile des Manipulators. Die LIN und die CIRC-Bewegung sind im Gegensatz zu PTP Bahnbewegungen, da der TCP sich mit konstanter Geschwindigkeit bewegt. Der Befehlsaufbau der Bewegung ist in Abbildung 13 gezeigt. Es werden dieselben Parameter wie bei PTP erwartet, die Geschwindigkeit wird jedoch als absolute Geschwindigkeit des TCP in m/s angegeben.

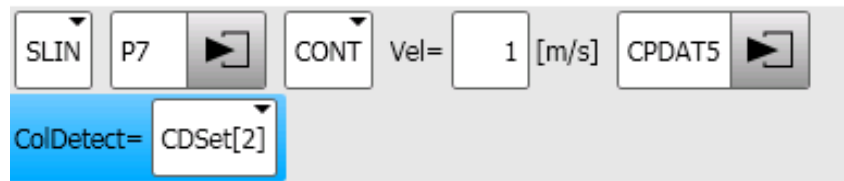


Abbildung 13: Inline-Formular für die LIN-Bewegung.

CIRC:

Die CIRC-Bewegung fährt eine Kreisbahn zum Zielpunkt über einen Hilfspunkt, hier P8. Radius und Winkel der Kreisbahn werden dann so berechnet, dass die Bahn durch den Hilfspunkt passiert. Der Befehl ist in Abbildung 14 gezeigt. Es werden die selben weiteren Parameter wie bei LIN erwartet, es aber noch zusätzlich möglich einen Winkel festzulegen, bis zu dem der Roboter nach dem Erreichen des Zielpunkts auf der Kreisbahn noch weiterabfährt.

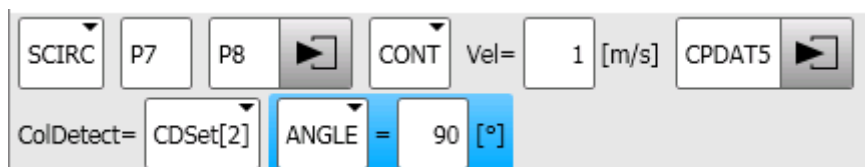


Abbildung 14: Inline-Formular für die CIRC-Bewegung.

Spline-Bewegungen werden mit dem Befehl SPL definiert, jedoch kann er nur innerhalb einer Spline-Blockstruktur im Skript verwendet werden und da der Befehl deshalb nicht verwendet wird, wird nicht weiter darauf eingegangen.

Wie schon in Abschnitt 2.5 erwähnt existieren in der KUKA Softwarearchitektur sogenannte Systemvariablen und Systemdateien, die den Zustand der Steuerung und des Arms beschreiben. Diese sind an dem \$-Zeichen am Anfang erkenntlich. In der Datei \$CONFIG.DAT können dann z.B. globale Variablen definiert werden, die für alle Programme zugänglich sind oder über die Variable \$ORI_TYPE die Orientierung des TCP während einer Bewegung festgelegt werden. Eine zentrale Rolle spielen dabei die Ein-/Ausgabe (E/A)-Arrays \$IN/\$OUT und \$AN_IN/\$AN_OUT. Damit ist es möglich, binäre bzw. analoge Signale von externen Geräten in KRL-Code zu verwenden. Über WorkVisual lassen sich Datenbytes von Feldbussen zu beliebigen Feldern der E/A-Arrays zuordnen und es ist somit möglich, laufenden KRL-Code von

außen zu beeinflussen. Für die Umsetzung der Steuerung in dieser Thesis aber wurde eine andere Methode verwendet, welche in Kapitel 4 vorgestellt wird.

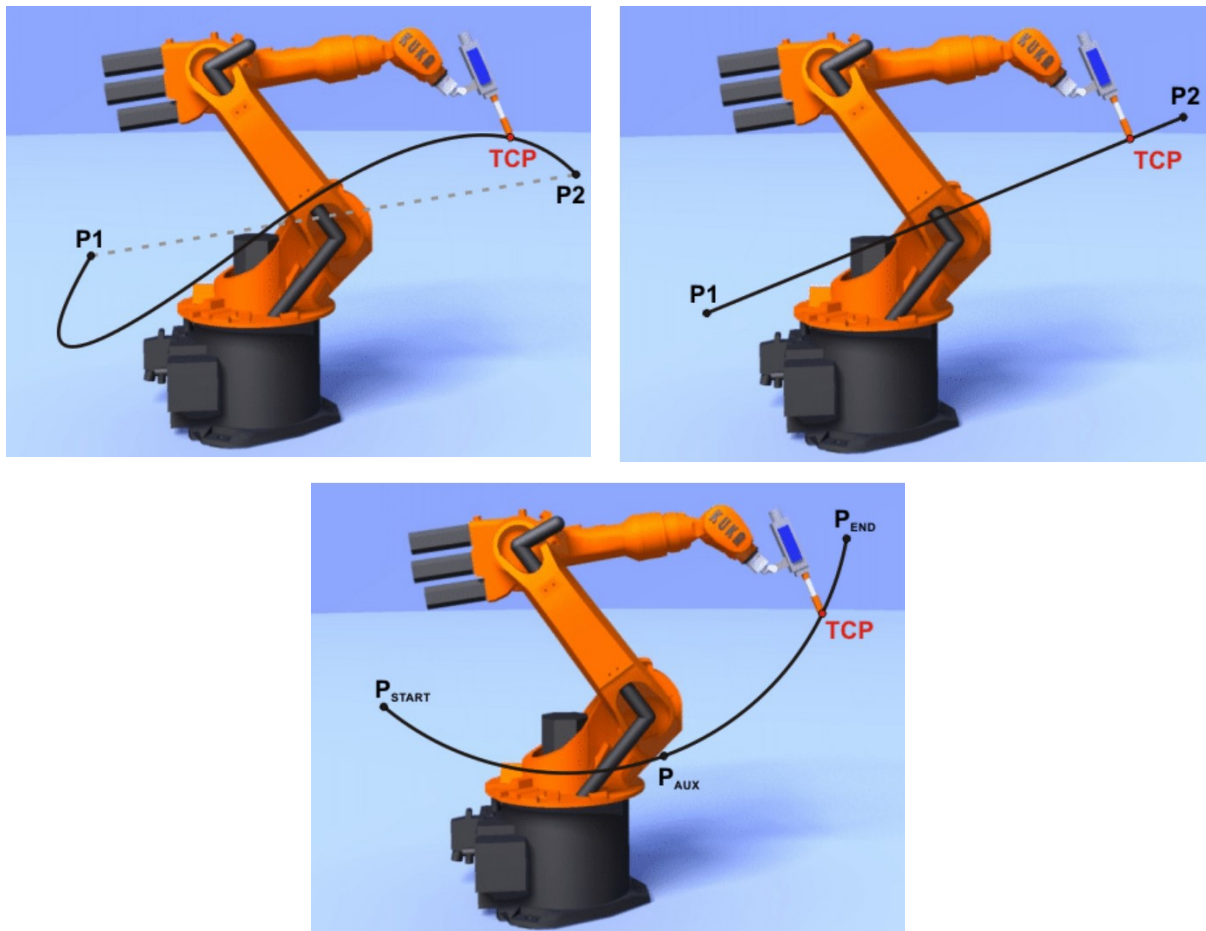


Abbildung 15: Darstellung der Bahnen für die Bewegungsarten PTP (l.o.), LIN (r.o.) und CIRC (unten) [4].

3. Anbindungsmöglichkeiten

In diesem Abschnitt werden die Verschiedenen Anbindungsmöglichkeiten an die Robotersteuerung vorgestellt und deren Vor- und Nachteile sowie Anforderungen bei der Implementierung genannt. Der Anschluss der Bussysteme erfolgt wie schon in Abschnitt 2.4 erwähnt über eine der 6 internen Ethernet-basierten Schnittstellen der Steuerung. Da im Labor nur zwei Geräte über einen kurzen Anschluss verbunden sind, wird in den folgenden Abschnitten nicht auf die möglichen Bustopologien eingegangen. In Abschnitt 3.1 werden die von der Steuerung unterstützten Feldbussysteme verglichen, in Abschnitt 3.2 werden weitere, auf TCP/IP oder auch auf *User Datagram Protocol* (UDP) basierte Anbindungen erläutert und in Abschnitt 3.3 schon früher implementierte Anbindungen an einen KUKA-Roboter vorgestellt.

3.1. Vergleich der Feldbusse:

PROFINET IO

PROFINET *Input Output* (IO) ist ein offener Standard für Industrial-Ethernet und wurde 2001 als Nachfolger von PROFIBUS entwickelt [15]. Die Hauptmerkmale sind dezentrale Peripherie, Echtzeitfähigkeit und ein modularer Aufbau der Kommunikation. Über PROFINET werden neben zyklischen Steuerungsdaten auch Konfigurations- und Diagnosedaten gesendet. Je nachdem welche Daten gesendet werden, wird entweder das TCP/IP/UDP-Protokoll oder ein spezielles Protokoll verwendet, welches direkt auf der Netzebene des *Open Systems Interconnection* (OSI)-Modells arbeitet und somit wenig Overhead hat und für die Übertragung von Echtzeitdaten geeignet ist. Der Aufbau so eines Echtzeitframes ist in Abbildung 16 dargestellt. Dadurch können sich Zykluszeiten von bis zu 250 μ s ergeben und durch Ethernet ist eine Datenmenge von 1440 Bytes pro Nachricht möglich.

Die Kommunikation erfolgt im Producer-Consumer-Modell und es existieren verschiedene Konformitätsklassen, von denen auf der KRC4 nur die Klasse A möglich ist. Diese bietet nur die grundlegende synchrone und asynchrone Kommunikation, die aber für den Laboraufbau reichen sollte.

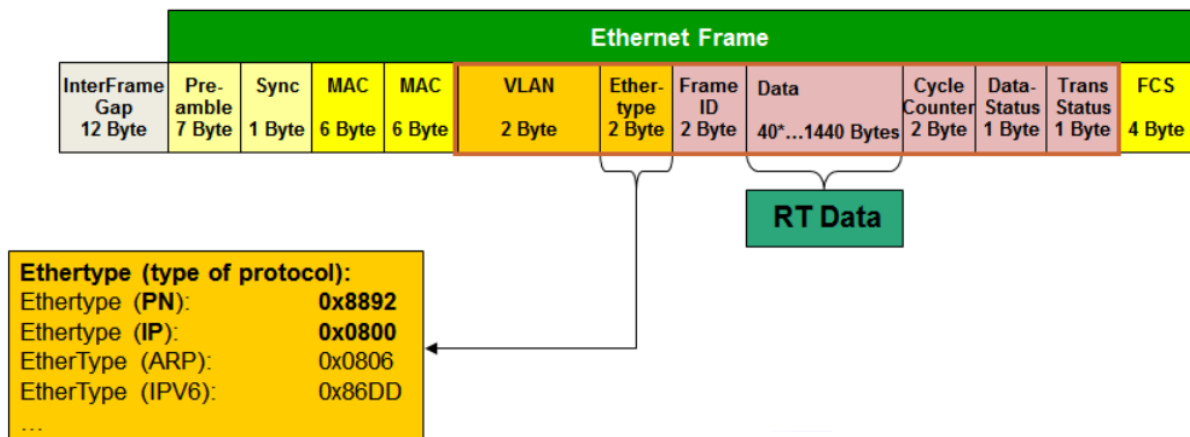


Abbildung 16: Aufbau eines Profinet-Frames bei der Echtzeit- Übertragung [16].

Für die KRC4-Steuerung erfolgt die Konfiguration in WorkVisual. Dafür ist das zusätzliche Paket KUKA.PROFINET und eine Objektbeschreibung in Form einer *General Station Description Markup Language* (GSDML)-Datei notwendig [17]. In WorkVisual werden dann die Ein- und Ausgänge des PROFINET-Busses entweder auf die E/A-Variablen der Steuerung oder auf andere Feldbussysteme gemappt. Dies geht aber nur bitweise und außerdem ist je nach verwendetem externen Gerät die Bitzahl beschränkt, weshalb die Übertragung von komplexeren Variablen schwierig ist. Außerdem ist die als kleinstmögliche Zykluszeit nur 1 ms wählbar. Die Anbindung an MATLAB ist wegen des speziellen Protokolls bei zyklischer Kommunikation ohne weiteres nicht möglich, weshalb noch entweder zusätzliche Kopplungsmodule oder eine SPS zwischen KRC4 und PC benötigt wird, welche dann TCP/IP oder UDP-Pakete vom PC in das PROFINET-Format umsetzen.

EtherNet/IP

Das EtherNet *Industrial Protocol* (Ethernet/IP) wurde von Allen-Bradley entwickelt und in 2001 veröffentlicht [18]. Es ist ein offener Standard, welcher von der *Open DeviceNet Vendor Association* (ODVA), einen Zusammenschluss mehrerer technischer Unternehmen, verwaltet. Die Kommunikation erfolgt in über TCP/IP oder UDP, in der Applikationsschicht aber wird zusätzlich das *Common Industrial Protocol* (CIP) verwendet [19]. Damit werden Geräte in verschiedene Typen unterteilt, die festgelegte Funktionen und bestimmte Daten im richtigen Format bereitstellen müssen. Bei CIP wird dies durch sogenannte Objekte realisiert, die mit einander interagieren, wie in Abbildung 17 gezeigt.

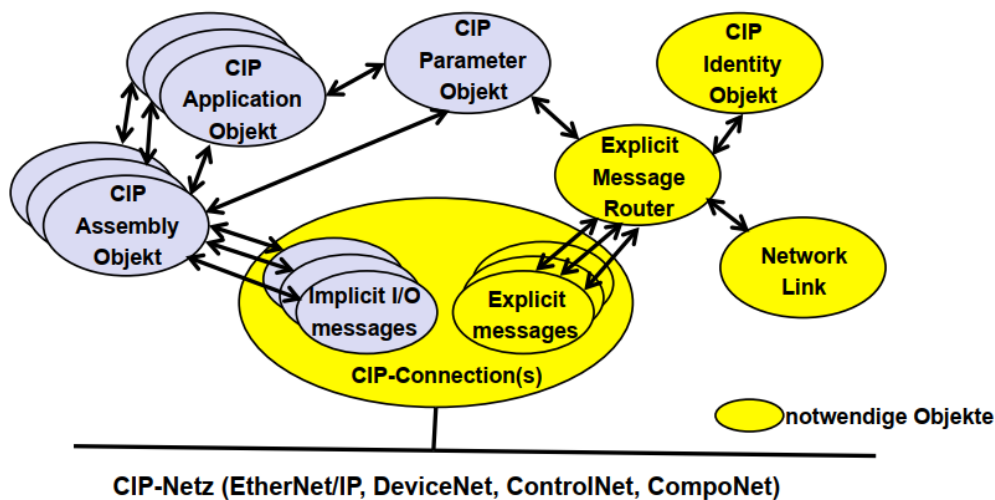


Abbildung 17: Darstellung der Verknüpfungen von CIP-Objekten [18].

Wie bei PROFINET können auch entweder zyklische Nutzdaten mit dem *implicit messaging* des CIP oder azyklische Parameterdaten mit *explicit messaging* gesendet werden. Es wird im Producer-Consumer-Modell kommuniziert, wo sich typische Zykluszeiten von einigen Millisekunden ergeben. EtherNet/IP kann aber keine harten Echtzeitanforderungen erfüllen, weshalb es i.d.R. zwischen Feld- und Netzwerkebene verwendet wird.

Die Konfiguration erfolgt ähnlich wie bei PROFINET in WorkVisual [20]. Dafür ist eine *electronic data sheet* (EDS)-Datei des externen Geräts, aber kein Optionspaket nötig. Das Mapping in WorkVisual erfolgt byteweise, wodurch mehr Information pro Frame übertragen werden kann.

PROFIBUS

Der PROFIBUS-Standard ist der Vorgänger zu PROFINET und wurde 1989 von einem Verbund verschiedener Unternehmen entwickelt mit dem Ziel, eine einheitliche Kommunikationsschnittstelle zwischen verschiedenen Herstellergeräten zu realisieren [21]. Von PROFIBUS existieren drei verschiedene Varianten, wobei die KRC4 nur die Variante PROFIBUS dezentrale Peripherie (DP) unterstützt, wo eine zentrale Steuerung mit mehreren dezentralen Feldgeräten kommuniziert. Die Kommunikation erfolgt im Gegensatz zu PROFINET im Master-Slave Verfahren mit einer maximalen Rate von 12 MBit/s. Außerdem ist ein Datenpaket auf maximal 244 Bytes beschränkt, während bei PROFINET 1440 Bytes möglich sind [22]. Vor Beginn der Kommunikation überträgt der Master Konfigurationsdaten und eine Identnummer an die Slaves und nur wenn diese mit den Daten der Slaves übereinstimmen, der Slave also für den

sendenden Master konfiguriert ist, wird eine zyklische Verbindung gestartet. Dies und die Verwendung von dedizierten Feldgeräten ermöglichen Echtzeitfähigkeit. Der Anschluss bei Profibus-DP erfolgt meistens über eine serielle RS-485 Schnittstelle mit einem D-Sub Stecker.

Die KRC4-Steuerung hat keine direkte Anbindung für PROFIBUS, weshalb zusätzliche EtherCAT-Kopplungsmodule nötig sind. Die Konfiguration vom Modul und PROFIBUS erfolgt aber ebenfalls in WorkVisual [23]. Es existieren separate Module, je nachdem ob die KR C4 als Master oder Slave verbunden werden soll. In WorkVisual wird dann eine *General Station Description* (GSD) Datei für das externe Gerät und eine *Extensible Markup Language* (XML)-Datei für den Buskoppler geladen und die Frame-Bytes wieder gemappt.

DeviceNet

Der DeviceNet-Standard ist das *Controller Area Network* (CAN)-basierte Äquivalent zu Ethernet/IP [24]. Es wurde ebenfalls von Allen-Bradley entwickelt und von der ODVA verwaltet. Das CIP wird ebenfalls verwendet, jedoch wird anstelle von TCP/IP oder UDP der serielle CAN-Bus benutzt, welcher eine Übertragungsrate von bis zu 500 kBit/s ermöglicht. Die Kommunikation kann als Master-Slave oder Peer-to-Peer realisiert werden. Dabei haben Geräte in einem Netzwerk die gleichen Berechtigungen, sodass sie Daten suchen und auch bereitstellen können. Aufgrund des CAN-Busses ist ein Datenpaket aber nur 8 Byte groß. Im Gegensatz zu EtherNet/IP ist aber die Einhaltung von harten Echtzeitanforderungen möglich, vorausgesetzt das die dahinterliegende Hardware dies auch unterstützt.

Zum Anschluss eines CAN-Busses an die KRC4-Steuerung ist eine RS485-Schnittstelle erforderlich. Wie bei PROFIBUS sind deshalb auch hier wieder EtherCAT-Kopplungsmodule nötig. Die Konfiguration der Geräte erfolgt in WorkVisual, worin für den Buskoppler wieder eine XML-Konfigurationsdatei benötigt wird [25]. Es werden dann Parameter gewählt, Ein- und Ausgänge gemappt und schließlich das WorkVisual-Projekt auf die Steuerung geladen.

EtherCAT

Ethernet for Control Automation Technology (EtherCAT) wurde von der Firma Beckhoff Automation entwickelt und bietet ein Ethernet-basiertes, echtzeitfähiges Feldbussystem, welches vor allem auf schnelle Zykluszeiten ausgelegt ist [26]. Feldgeräte in diesem System kommunizieren per Master-Slave Verfahren, jedoch verarbeiten die Slaves schon eine Nach-

richt, während diese noch versendet wird, anstatt den Empfang des kompletten Frames abzuwarten. Die Slaves können dann auch schon Daten in diesen Frame schreiben, bevor dieser weiterversendet wird. In

Abbildung 18 ist dieser Prozess gezeigt. Dies sorgt für eine schnelle Kommunikation mit Zykluszeiten von 150 μ s bei kompletter Verwendung der 1536 Bytes Nutzdaten und Jitter von weniger als 1 μ s. Es werden aber auf der Slave-Seite spezielle Hardwareschnittstellen benötigt, die die Verarbeitung der Nachrichten zur Laufzeit ermöglichen. Als Master kann aber jedes Ethernet-fähiges Gerät mit ausreichend schnellem Prozessor benutzt werden, wobei harte Echtzeitanforderungen nur mit einem deterministischen Betriebssystem erfüllbar sind. Zudem wird ein eigenes Kommunikationsprotokoll verwendet, welches direkt auf den Ethernet-Standard aufbaut und die Schichten 3 und 4 des OSI Modells erfasst. Dabei wird möglichst wenig Overhead erzeugt, sodass eine Nutzdatenrate von über 90% vorliegt. Weiterhin ist mit EtherCAT wie schon vorhin vorgestellt die Anbindung anderer Bussysteme wie CAN oder PROFIBUS möglich.

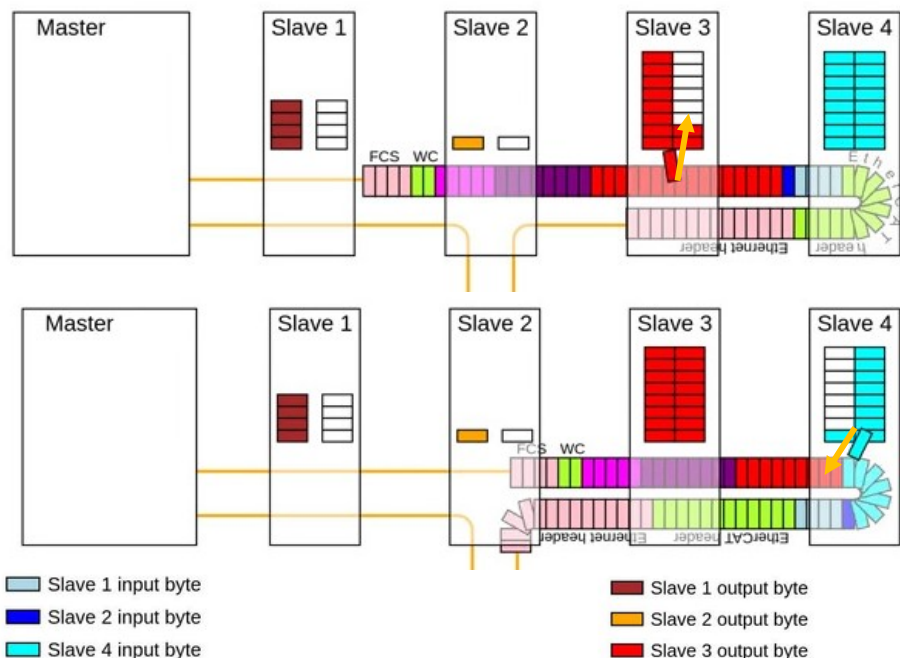


Abbildung 18: (Oben) Ein Slave liest Daten aus einem EtherCAT-Frame. (Unten) Ein anderer Slave schreibt während der Laufzeit Daten in den Frame [27].

Die Inbetriebnahme von EtherCAT auf der KR C4 läuft wie in den vorherigen Abschnitten auch über WorkVisual, jedoch ist noch ein zusätzliches Slave-Modul als Hardware notwendig [28]. Bei den Parametern in WorkVisual ist zudem nur eine Mindestzykluszeit von 1 ms wählbar.

Vergleich

In

Tabelle 4 werden die vorgestellten Bussysteme verglichen. Bei PROFIBUS wurde die Zykluszeit anhand der Formel in [29] geschätzt. Bei DeviceNet wurden keine Informationen zu gängigen Zykluszeiten gefunden und es wurde deshalb wie folgt geschätzt. Es wird von mindestens zwei Nachrichten pro Zyklus ausgegangen, die jeweils 108 Bit groß sind. Bei einer Übertragungsrate von 500 kBit/s käme eine Zykluszeit von ca. 440 μ s heraus, wobei noch Hardwareverzögerungen berücksichtigt werden müssen.

	PROFINET	EtherNet/IP	PROFIBUS	DeviceNet	EtherCAT
Min. Zykluszeit	250 μ s	Einige ms	355 μ s	440 μ s	150 μ s
Paketgröße	1500 Bytes	1500 Bytes	244 Bytes	8 Bytes	1486 Bytes
Anschluss	Ethernet	Ethernet	Seriell (RS485)	CAN	Ethernet
Echtzeitfähig	Ja	Nur weich	Ja	Nur weich	Ja
Kommunikationsart	Producer-Consumer	Producer-Consumer	Master -slave	Master -slave	Master -slave
Zusätzliche Hardware nötig	PCI-Karte für PC: (z.B. CFX 50-RE)	keine	Kopplungs-module: - Beckhoff EL6731 - Kunbus PR 100116	Kopplungs-module: - Beckhoff EL6752 - 1784-U2DN	Slave-modul: - Beckhoff EK1100
Optionspakete	KUKA.PROFINET	-	-	-	-
Kosten Labor-setup	2950 €	-	ca. 1115 €	ca. 1330 €	ca. 130 €

Tabelle 4: Vergleich der mit der KR C4 Compact kompatiblen Feldbussysteme.

Insgesamt eignet sich eine Anbindung im Labor mit den vorgestellten Feldbussen wenig, da die Vorteile dieser Busse nicht voll ausgenutzt werden können. Diese bieten zwar eine Möglichkeit zur schnellen, robusten und meist echtzeitfähigen Kommunikation zwischen vielen

verschiedenen Endgeräten, im Labor aber befinden sich jedoch nur zwei, von denen der Labor-PC durch das Windows-Betriebssystem nicht hart echtzeitfähig ist. Die Möglichkeit der schnellen Zykluszeiten wird zudem durch eine Aktualisierungszeit der Steuerungsinternen Variablen von 12 ms ausgeglichen. Außer bei EtherNet/IP fallen zudem bei allen Systemen zusätzliche Kosten an, da der Labor-PC oder die Steuerung die Verarbeitung der Feldbuspakete nicht unterstützen. Dafür wurden in der Tabelle 4 Beispiele für konkrete Hardwarekomponenten gegeben und deren Kosten recherchiert [30] [31] [32] [33] [34] [35]. Der Punkt 3.1 aus der Anforderungstabelle könnte also voll und Punkt 3.2 teilweise erfüllt werden, jedoch wird Punkt 3.4 nicht erfüllt. Eine Anbindung über EtherNet/IP wäre zwar denkbar, jedoch müsste in MATLAB eine Nachricht im CIP-Format erstellt werden, wozu aber keine genauere Information gefunden wurde. Deshalb werden in den nächsten Abschnitten weitere Anbindungsmöglichkeiten präsentiert, die auf einer normalen TCP/IP oder UDP-Verbindung aufbauen und deshalb keine weitere Hardware erfordern.

3.2. Weitere Anbindungen über TCP/IP oder UDP

OPC

OPC steht für *Open Platform Communications* und ist eine standardisierte Software-Schnittstelle, mit der Anwendungen von unterschiedlichen Herstellern Daten austauschen können [36]. Die Kommunikation basiert auf einem Client-Server-Modell, wo jedes Gerät bestimmte Klassen und Methoden der OPC-Spezifikation zu erfüllen hat, mit welchen Daten angefragt und versendet werden. Damit ist eine einheitliche Schnittstelle garantiert. An den Ablauf der Kommunikation selbst werden in den Spezifikationen keine Forderungen gestellt, weshalb die Latenzzeit und Echtzeitfähigkeit einer OPC-Anwendung von der Hardware abhängt, auf der diese läuft. In Abbildung 19 ist ein Schaubild zur OPC-Kommunikation gegeben. Die Kommunikation über OPC ist vor allem für den Austausch größerer Datenmengen zwischen mehreren verschiedenen Anwendungen mit unterschiedlichen Kommunikationsprotokollen geeignet.

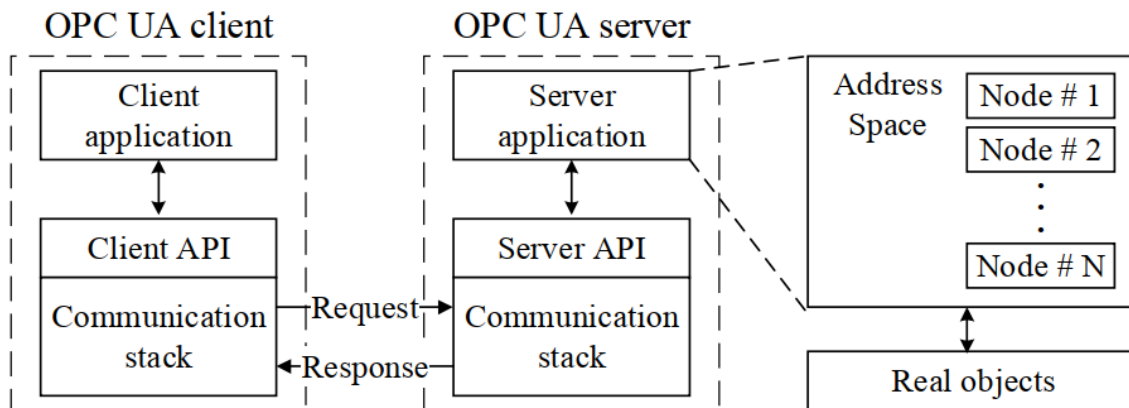


Abbildung 19: Darstellung der Kommunikation eines OPC UA Clients und Servers [37].

Mit dem Optionspaket KUKA.OPC UA ist die Erstellung eines OPC-Servers auf dem SmartPAD möglich [38]. Mit einem bereitgestellten Demo-OPC-Client oder der OPC-Toolbox in MATLAB lässt sich eine Verbindung zum Server herstellen. Jedoch sind die austauschbaren Daten fest definiert und nicht frei wählbar. Zudem ist nicht voraussagbar, welche Zykluszeit bei der Kommunikation zu erwarten ist, da größere Datenmengen in kleinere Pakete zerteilt und nacheinander versendet werden. Es können somit die Punkte 3.1, 3.2, 3.4 und womöglich weitere Anforderungen aus Tabelle 1 wohl nicht erfüllt werden.

RSI

Das Optionspaket KUKA.RobotSensorInterface (RSI) bietet eine Schnittstelle für Sensoren, um Messwerte und Daten mit der Steuerung auszutauschen [39]. Im Gegensatz zu den anderen Anbindungsoptionen, wo nur Variablenwerte ausgetauscht werden, greift RSI direkt auf die Bahnplanung der Steuerung zu und ermöglicht so Bewegungskorrekturen anhand von Sensorwerten. Um festzulegen, welche Daten gesendet und empfangen werden sollen, stehen Schnittstellen mit gewissen Funktionen zum Lesen und Schreiben von Daten zur Verfügung. Diese sogenannten RSI-Objekte können miteinander verknüpft werden, um so Signalflüsse, auch RSI-Kontext genannt, zu erzeugen. Die Signale können die Systemvariablen \$IN/\$OUT oder die vom RSI eigens bereitgestellten zusätzlichen E/As sein. Mithilfe von Buskopplern und IO-Boards lassen sich so verschiedene Bussysteme wie EtherCAT oder PROFINET mit RSI koppeln. Mit dem entsprechenden RSI-Objekt lässt sich auch direkt eine Verbindung über UDP aufbauen. Dabei ist der Zugriff auf alle vorhandenen Variablen möglich, es

muss aber mithilfe einer XML-Datei zusätzlich festgelegt werden, welche Daten in welchem Format kommuniziert werden. In Abbildung 20 ist ein Beispiel für ein RSI-Kontext mit einem Ethernet-Objekt gezeigt. Es können dann z.B. über Objekt 1 oder 2 eine bestimmte Variable oder ein Funktionsergebnis dem Ethernet-Objekt zum Senden übergeben werden und darüber empfangene Daten dann über Objekt 4 oder 5 weiterverarbeitet werden.

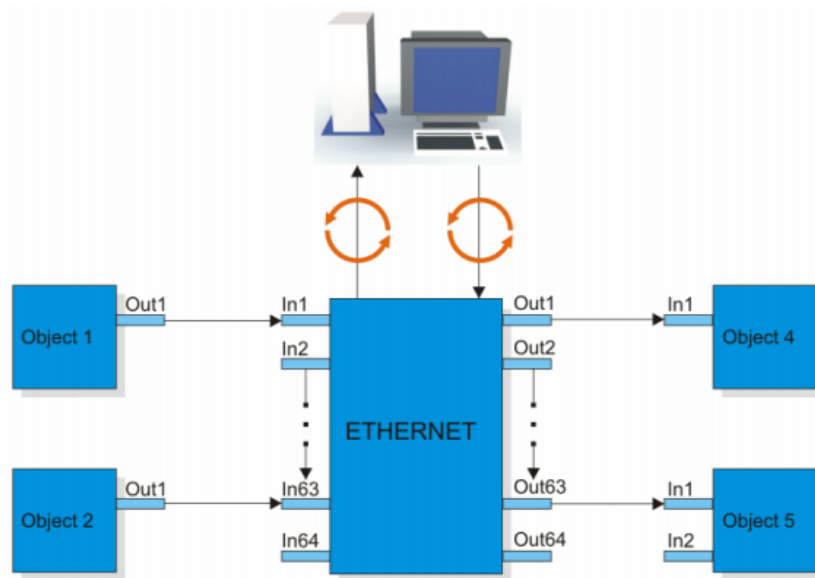


Abbildung 20: Beispiel für ein RSI-Kontext für die Kommunikation über Ethernet [39].

Über ein KRL-Programm wird dann der RSI-Kontext geladen und das RSI gestartet. Es werden dann in einem Zyklus von 12 ms anhand des RSI-Kontextes Daten gesendet. Der Sensor muss in dieser Zeitspanne antworten, denn wenn eine maximale Anzahl von Antworten nicht empfangen worden sind, bricht die Steuerung die Kommunikation ab. Es wird somit eine Echtzeitfähigkeit des Sensors vorausgesetzt, oder zumindest eine ausreichend schnelle Antwortzeit.

Als Korrekturdaten lassen sich entweder Achswinkel oder kartesische Positionen übermitteln. Die Korrekturposition bezieht sich dabei auf den TCP des Roboters. Es lässt sich auswählen, ob die Korrektur absolut oder relativ vorgenommen wird. Bei absoluter Korrektur werden die Sensorwerte stets auf die eigentlich programmierte Bahn draufaddiert und bei relativer Korrektur auf die aktuelle Position des Roboters. In Abbildung 21 ist die absolute Korrektur gezeigt. Es ist zu beachten, dass nur eine begrenzte Abweichung von einer programmierten Bahn möglich ist. Als Alternative ist eine Sensorgeführte Bewegung möglich,

wo die Bahn des Roboters allein durch die gesendeten Sensorwerte vorgegeben wird, wie in Abbildung 21 zu sehen. Auch hier sind absolute oder relative Korrekturangaben möglich.

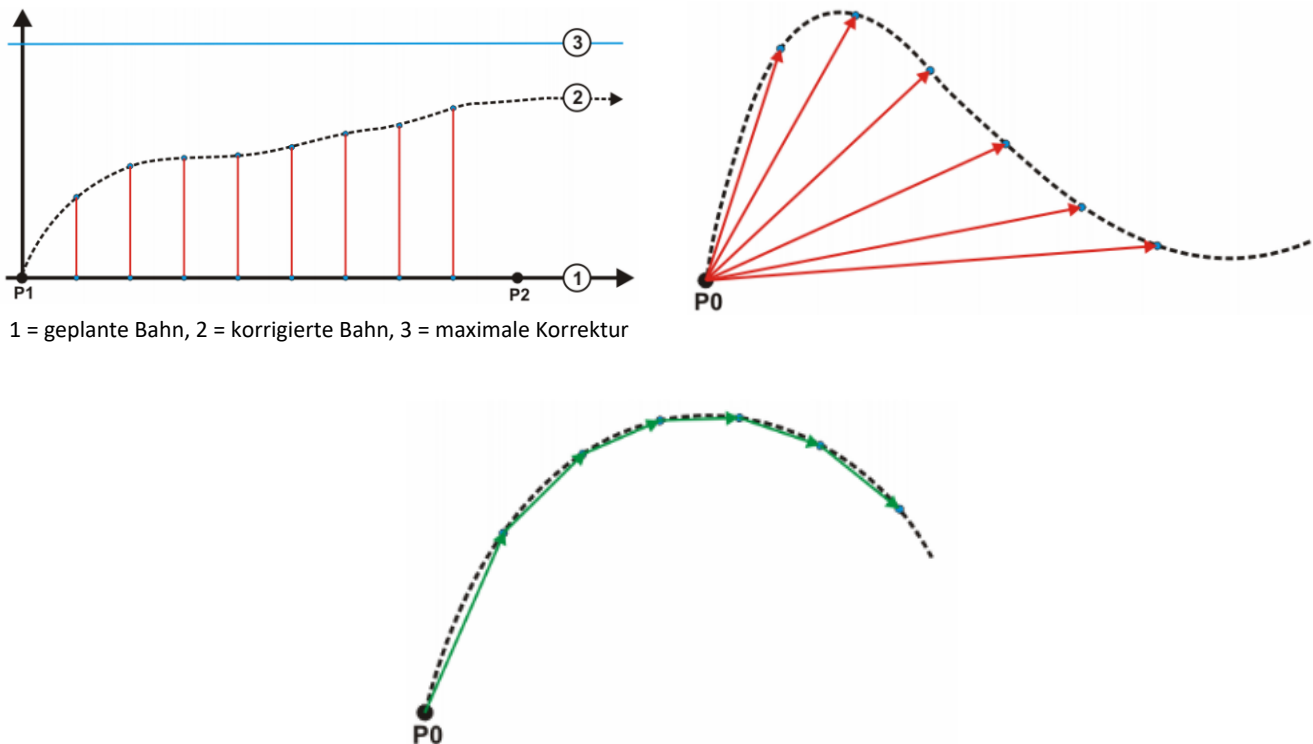


Abbildung 21: (L. o.) Darstellung einer Absoluten Bahnkorrektur. (R. o.) Darstellung der absoluten sensorgeführten Bewegung. (Unten) Darstellung der relativen sensorgeführten Bewegung [39].

Ethernet KRL:

Mit dem Optionspaket KUKA.Ethernet KRL ist ein allgemeiner Datenaustausch auf Basis von TCP/IP oder UDP möglich [40]. Es lassen sich dabei beliebige Datenstrukturen übertragen, die vorher durch eine XML-Datei definiert werden. Durch diese Konfiguration werden für jede Struktur Datenspeicher erzeugt, die als Buffer beim Senden und Empfangen dienen, wie in Abbildung 22 gezeigt. Daten können entweder als rohe Binärdaten oder als XML-Strukturen kommuniziert werden. Der Zugriff von der Steuerung erfolgt in einem KRL-Programm durch entsprechende *get/send* Befehle. Dadurch liegt eine Kapselung der Ethernet-Kommunikation von dem eigentlichen Programmablauf vor, wodurch ein robustes Verhalten vorliegt und Punkt 3.1 aus Tabelle 1 erfüllt wird. Es gibt dann aber auch keine Echtzeitgarantie, da durch die Zwischenspeicherung der Daten eine nicht konstante Verzögerung entsteht. Der zusätzlich notwendige Kauf von KUKA.Ethernet KRL und die XML-Konfiguration erfüllen ebenfalls nicht Punkt 3.4. Durch die *get/send* Befehle ist das Optionspaket eher für

bedingte azyklische Kommunikation geeignet, die sich anhand des KRL-Programms steuern lässt.

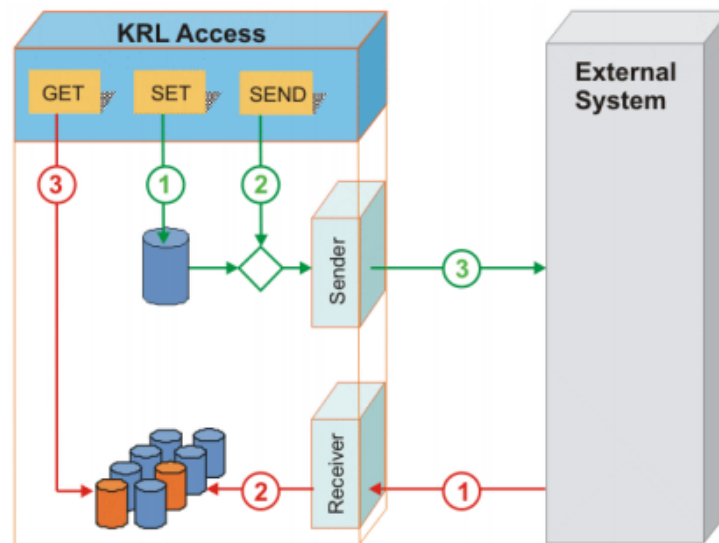


Abbildung 22: Schema der Kommunikation des KUKA.Ethernet KRL Optionspaketes [40].

KukavarProxy

KukavarProxy (KVP) ist eine Server-Anwendung, welche den Zugriff auf interne Variablen der Steuerung erlaubt. Sie wurde von der Firma IMTS S.r.L. entwickelt und 2019 der Quellcode veröffentlicht [41]. Mit dem Server kann über TCP/IP eine Verbindung mit 10 Clients gleichzeitig aufgebaut werden. Die Anwendung selbst wird auf dem embedded-Windows Betriebssystem der Steuerung gestartet und verwendet dann das *CrossComm* Interface. Mithilfe damit können Variablenwerte zwischen dem Echtzeit-System *VxWorks* und der Windows-Version auf der Steuerung ausgetauscht werden. Wie die *CrossComm*-Schnittstelle genau funktioniert, ist im Rahmen dieser Thesis nicht bekannt, da keine öffentlich zugängliche Dokumentation gefunden wurde.

Die Interaktion mit KukavarProxy läuft über das Senden von Lese- und Schreibanfragen für bestimmte Variablenwerte ab. Die Anfragen sind Byte-Arrays mit einem festen Format, wie in Abbildung 23 dargestellt. Auf eine Lese- oder Schreibanfrage wird eine Antwortnachricht zurückgesendet mit dem aktuellen Wert der Zielvariable. Der Aufbau ist ebenfalls in Abbildung 23 gezeigt. Da es für die später vorgestellte Implementierung von Bedeutung ist, wird im Folgenden der Aufbau dieser Anfragen erklärt.

Byte	1	2	3	4	5	6	7	8 - n
Feld	Nachricht-ID		Länge -3		r/w-Flag	Länge-Var. Name		Variablenname (ASCII)
Bsp.	00	01	00	09	00	00	06	66 83 80 86 65 82

Byte	1	2	3	4	5	6	7	8 - n	n+1	n+2	N+m
Feld	ID		Länge -3		Flag	Länge Name		Name (ASCII)	Länge Wert		Wert (ASCII)
Bsp.	00	01	00	09	00	00	06	66 83 80 86 65 82	00	01	49

Byte	1	2	3	4	5	6	7	8 - n	n + 3
Feld	ID		Länge -3		Flag	Länge Wert		Wert (ASCII)	Kontrollbytes
Bsp.	00	01	00	08	00/01	00	06	49	00 01 01

Abbildung 23: (Oben) Aufbau einer Leseanfrage an den KVP-Server für die Variable BSPVAR. (Mitte) Aufbau einer Schreibanfrage für BSPVAR auf den Wert 1. (Unten) Aufbau einer Antwort auf eine der beiden oberen Nachrichten.

Die ersten beiden Bytes bilden die ID der Nachricht als uint16-Wert, welche im Wertebereich [1, 99] frei wählbar ist und zur Zuordnung von Antworten zu den jeweiligen Nachrichten dient. Die nächsten beiden Bytes speichern ebenfalls im uint16-Format die Anzahl der folgenden Bytes, welche zu der aktuellen Nachricht gehören. Byte 5 bildet einen Flag zur Unterscheidung zwischen Lese- und Schreibanfrage, wobei der Wert 0 für Lesen und 1 für Schreiben steht. Byte 6 und 7 sind die Länge des Namens der Zielvariable, also die Anzahl der belegten Bytes für die ASCII-Zeichen des Namens, und ab Byte 8 sind diese ASCII-Werte gespeichert. Bei Schreibanfragen folgen danach noch zwei Bytes für die Länge des zu schreibenden Wertes und schließlich der Wert selbst. Es ist zu beachten, dass der Wert ebenfalls im ASCII-Format angegeben werden muss, wie in Abbildung 23 demonstriert.

Da die Server-Anwendung auf dem Windows-Betriebssystem ausgeführt wird und über TCP/IP kommuniziert, ist kein Determinismus gegeben. Bei ausreichend schneller Kommunikation ist es aber trotzdem möglich, zumindest weiche Echtzeitanforderungen zu erfüllen. In [42] wurde ein Laboraufbau erstellt, wo mit KukavarProxy zyklisch Ist- und Soll-Werte versendet wurden. In Abbildung 24 sind die Latenzzeiten ab dem Senden einer Lese- und Schreibanfrage bis zum Erhalten der Antwortnachricht gezeigt. Trotz der ab und zu auftretenden Peaks betrug die durchschnittliche Latenzzeit 4,27 ms, was für die Kommunikation mit der KUKA-Steuerung ausreicht, da ein Variablenwert nur alle 12 ms aktualisiert wird. Es

ist außerdem auch möglich, mehrere Anfragen in einem Frame an den Server zu schicken, worauf dieser die Antworten auch alle in einem Frame packt. Dies hat laut [42] keinen Einfluss auf die Latenzzeit eines Frames, wodurch weniger Anfragen bei einem Programm gestellt werden müssen und so die Ausführung verbessert werden kann.

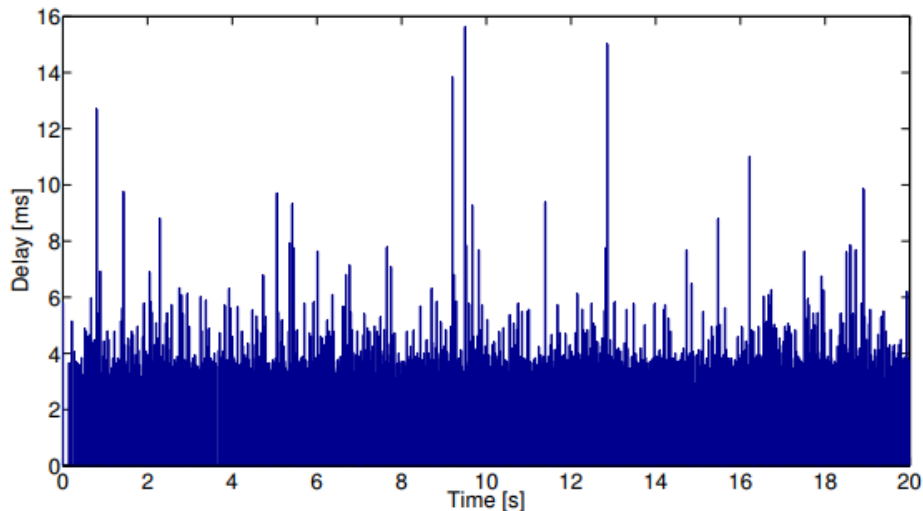


Abbildung 24: Latenzzeiten zwischen senden einer Anfrage und Empfangen einer Antwort mit KukavarProxy [42].

3.3. Existierende Anbindungsimplementierungen

Es wurde schon zu früheren Zeitpunkten versucht, eine Verbindung mit einer KUKA-Steuerung und einem externen PC, bzw. MATLAB herzustellen. Im folgenden Abschnitt werden zwei verschiedene Implementierungen und ihre Funktionsweise und Möglichkeiten gezeigt.

RoBO-2L

RoBO-2L ist eine open-source MATLAB Toolbox, welche an der Hochschule Bochum entwickelt wurde [43]. Sie stellt MATLAB-Funktionen wie die Generierung von Trajektorien und eine inverse Kinematikberechnung zur Verfügung, mit der sich ein KUKA KR6-R900 SIXX steuern lässt. Die Berechnung verwendet jedoch einen geometrischen Ansatz, der aktuell nur die kinematische Kette des KR6-R900 betrachtet. Die Toolbox verwendet zur Kommunikation mit der Steuerung das RSI, indem mit Hilfe einer C#-Bibliothek ein Serverthread erstellt wird, der zyklisch Soll-Geschwindigkeiten sendet und Ist-Achswerte und -Positionen empfängt. Die Sende- und Empfangswerte bilden Attribute einer Serverklasse und die Erstellung des Threads eine Methode. Es stehen dann weitere Methoden zum Ändern der Attribu-

te bereit, die über MATLAB aufgerufen werden können, da MATLAB auch die Ausführung von mit .NET erstellten Code unterstützt. Somit wird eine rechtzeitige Antwort auf die RSI-Nachrichten der Steuerung und gleichzeitig das Setzen eines neuen Sollwertes zum unabhängigen Zeitpunkt ermöglicht.

Solch eine Kommunikation wurde schon in einer früheren Toolbox implementiert, jedoch war diese nur für die KRC2-Steuerung von KUKA ausgelegt [44]. RoBO-2L hat diese Anbindung auf die neuere KRC4 erweitert. Im Moment werden nur Funktionen für lineare und kreisförmige, bzw. elliptische Bewegungen angeboten. Es werden dabei bei einer vorgegebenen Position und Bahngeschwindigkeit die Geschwindigkeiten der einzelnen Achsen berechnet, die an die Steuerung gesendet werden. Zudem steht eine GUI zur Verfügung, mit der sich die aktuelle Position anzeigen lässt und eine Wunschposition angegeben werden kann. Da aber RoBO-2L noch weiterentwickelt wird, kann sich die Funktionsweise und der Umfang der Funktionen zukünftig ändern. So soll in Zukunft auch eine PTP-Bewegung möglich sein.

JOpenShowVar

JOpenShowVar ist eine quelloffene, in Java programmierte Schnittstelle, welche neben Lese- und Schreibfunktionen von KUKA-Variablen auch erweiterte Funktionen wie eine externe Bahnplanung und eine GUI bietet [42]. Die Kommunikation basiert auf dem KukavarProxy-Server, der dann entweder auf einer KRC4- oder KRC2-Steuerung läuft. Mithilfe einer Client-Klasse *CrossCommClient* wird eine Verbindung aufgebaut und über Methoden Lese- und Schreibanfragen geschickt. Es werden die Java-Bibliotheken *net.Socket*, *io.BufferedInputStream* und *io.BufferedOutputStream* verwendet. Um den Manipulator zu bewegen, wird in einem KRL-Programm eine Schleife mit einem Bewegungsbefehl ausgeführt, dessen Zielposition eine von JOpenShowVar geschriebene Variable ist.

JOpenShowVar wurde hinsichtlich einer flexiblen Verwendung mit verschiedenen benutzerdefinierten Programmen entwickelt. Es wird in [42] z.B. die Möglichkeit der Implementierung einer externen Anwendung zur Bahnberechnung beschrieben. Wie in Abbildung 25 gezeigt, könnte zyklisch die Ist-Winkelstellung der Achsen abgefragt werden und anhand einer kartesischen Sollposition und einer inversen benutzerdefinierten Kinematikberechnung die Sollwinkel der Achsen berechnet und über den KVP-Server versendet werden. Allerdings muss so ein Anwenderprogramm entweder in Java programmiert sein oder eine Java-kompatible

Schnittstelle haben. Die inverse Kinematik ist zudem bis jetzt nur auf den Kuka KR6 R900 SIXX beschränkt.

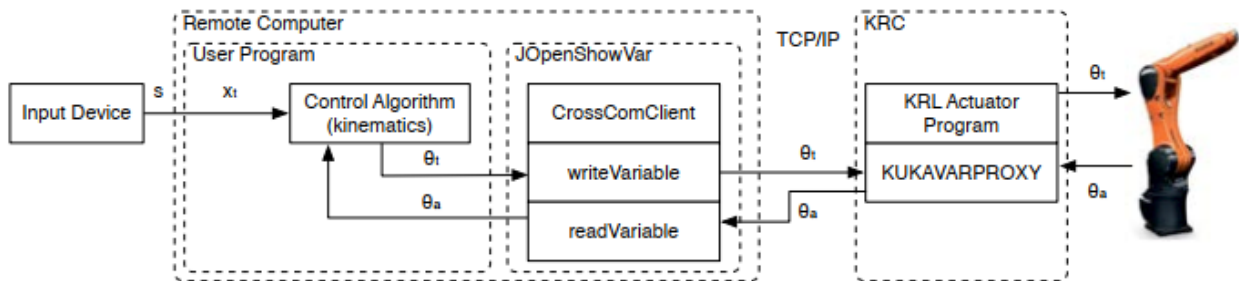


Abbildung 25: Schema der Verwendung einer externen Bahnplanung mit JOpenShowVar [42].

4. Eigene Implementierung

Die vorgestellte Toolbox RoBO-2L bietet zwar eine Möglichkeit, einen KUKA Roboter über MATLAB zu steuern, jedoch ist dafür ein kostenpflichtiges Optionspaket nötig. Außerdem ist eine inverse Kinematik für den im Labor verwendeten KR3 AGILUS nicht vorhanden. Das Programm JOpenShowVar bietet zwar eine kostenlose Alternative, jedoch läuft die Steuerung und Konfiguration über eine eigene GUI, womit es keine direkte Möglichkeit gibt, MATLAB einzubinden. Auf Basis der Kommunikation mit KukavarProxy wurden deshalb in dieser Thesis Programme in MATLAB zur Steuerung eines KUKA-Roboters entwickelt, welche in den folgenden Abschnitten vorgestellt wird. Es besteht die Möglichkeit, analog zu einem KRL-Skript Wegpunkte einer Bahn an die Steuerung zu senden, die diese dann abfährt. Die Steuerung des Roboters ist somit recht einfach und deshalb Anfängern zugänglich. Das dafür zuständige Programm in MATLAB empfängt und speichert zyklisch die Ist-Position des Roboters, sodass dann eine Visualisierung der gefahrenen Bahn möglich ist. Weiterhin wurde auch eine Möglichkeit zur manuellen Bewegung des Manipulators mit Tastatur oder einer Spacemacs implementiert, mit der auch Wegpunkte gespeichert werden können. Mit einer zusätzlich implementierten Klasse sind zudem Hilfsmethoden bereitgestellt, die sich in eigenen Anwendungen frei verwenden lassen. In Abbildung 26 ist ein Schaubild zur Interaktion der einzelnen Teilprogramme gegeben.

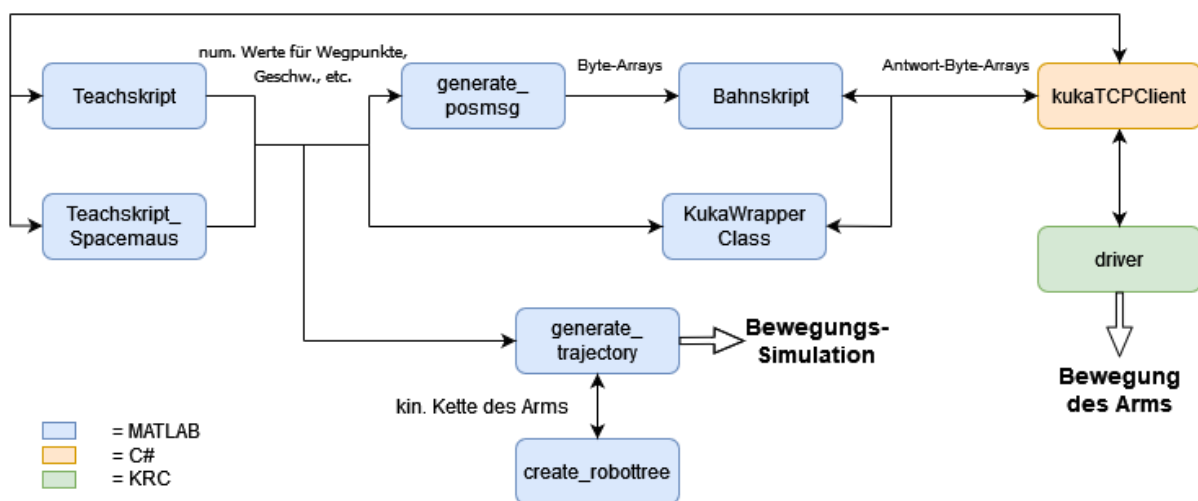


Abbildung 26: Schaubild für die Interaktion zwischen den verschiedenen Teilprogrammen.

4.1. Bahnfahrt

In diesem Abschnitt wird die Funktionsweise der MATLAB-Funktion *Bahnskript.m* vorgestellt, mit dem eine vordefinierte Bahn gefahren wird. Die Funktion interagiert mit einem KRL-Skript, welches parallel auf der Steuerung läuft und in Abschnitt 4.2 erläutert wird. Die Bahn wird durch Wegpunkte im Format der POS-Datenstruktur in KRL und durch weitere Parameter wie Bewegungsart, Geschwindigkeit und Zustand des Greifers beschrieben, welche in Tabelle 5 gezeigt sind. In Abbildung 27 ist ein Flussdiagramm für den Programmablauf gezeigt. Die Wegpunkte der Bahn und die zugehörigen Parameter werden nacheinander abwechselnd entweder einem Variablensatz 1 oder 2 zugeordnet. Das KRL-Skript arbeitet in einer Schleife abwechselnd Bewegungsbefehle ab, die entweder die ersten oder die zweiten Variablensätze als Argumente haben. Ist nun die Bewegung mit dem Satz 1 beendet, wird die Bewegung mit dem Satz 2 gestartet und gleichzeitig die Variablen des Bewegungssatzes 1 mit den nächsten Wegpunkten und Parametern überschrieben, die nach der aktuellen Bewegung folgen werden. Durch diese Arbeitsweise ist das Programm nicht abhängig von einer Echtzeit-Übertragung der Daten, was die Programmausführung robust macht. Latenzspitzen und langsame Sende- und Empfangsmethoden haben somit keinen Einfluss auf die Bewegung des Roboters, wohl allerdings auf die Aufnahme des Ist-Positionsverlaufs. Es wurden daher in Abschnitt 4.6 verschiedene Sende- und Empfangsmethoden in MATLAB auf ihre Laufzeit getestet. Dabei werden KRL-Variablen gelesen und geschrieben, die für den Bewegungsablauf zuständig sind. Zur besseren Verständlichkeit sind im Bericht alle KRL-Variablen in Großbuchstaben geschrieben.

KRL-Variable	MAT_POS	MOVEMODE	VEL	GRP	HP_POS	FINISH
Format	1x8 DOUBLE	1x1 INT	1x1 INT	1x1 INT	1x8 DOUBLE	1x1 BOOL
Wertebereich	-	[1, 4]	[1, 100]	[1, 2]	-	-
Bedeutung	Zielkoordinaten der aktuellen Bewegung	Wahl der Bewegungsart (PTP,LIN,...)	Relative Geschwindigkeit in % zu 1 m/s	Stellung des Greifers	Koordinaten Des Zwischenpunkts für eine Kreisbewegung	Angabe, ob der Zielpunkt erreicht wurde

Tabelle 5: Zu einem Bewegungsdatensatz zugehörige Variablen.

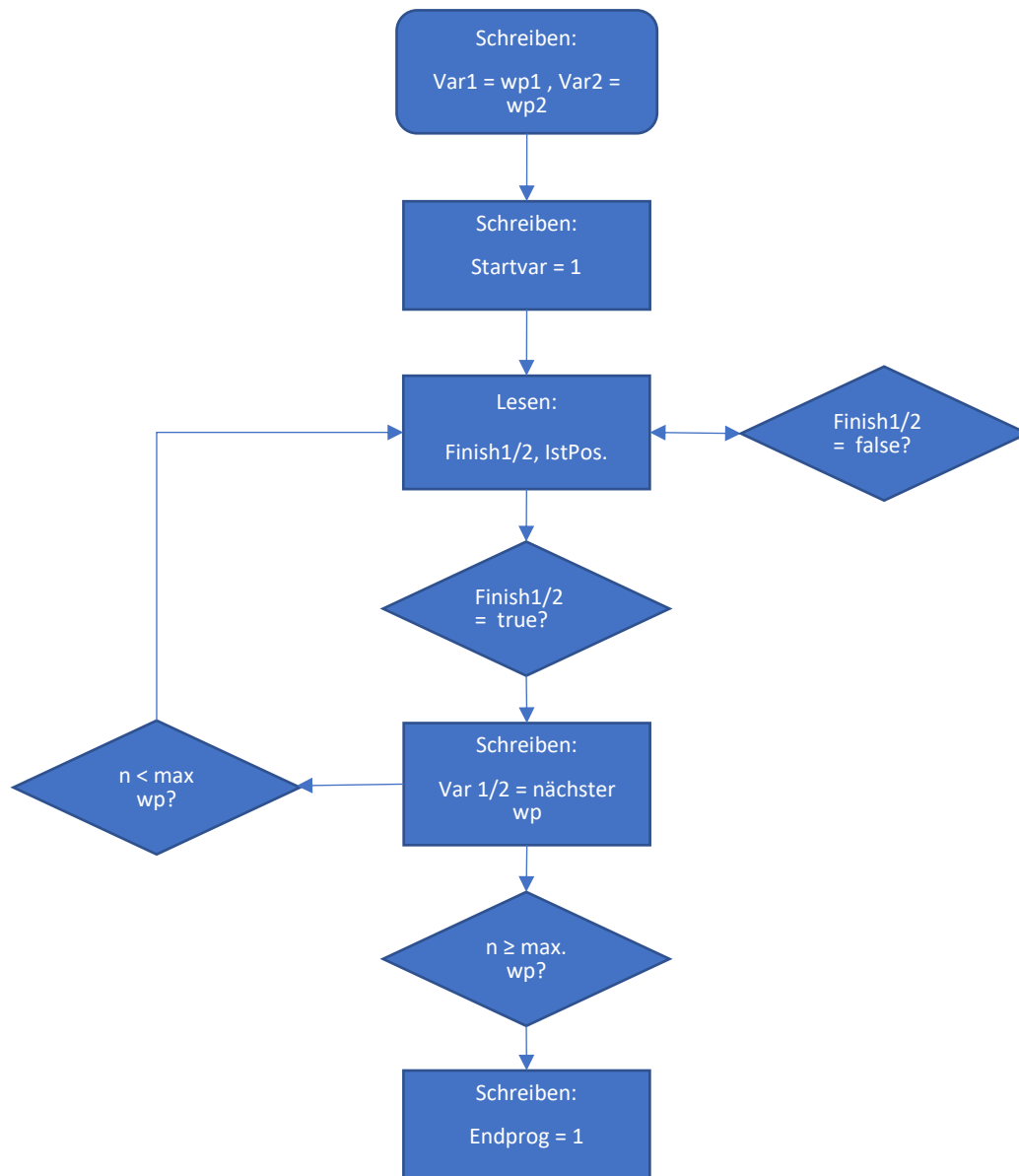


Abbildung 27: Flussdiagramm für das Abfahren einer Bahn.

Die Funktion erwartet als Argumente die Arrays mit den KVP-Nachrichten der Bewegungsparameter, welche mit der Funktion *generate_posmsg.m* erstellt werden. Die Funktion wird in Abschnitt 4.5 erklärt. Am Anfang werden dann alle notwendigen Variablen initialisiert und Arrays präallokiert, damit im späteren Schleifendurchlauf die Zykluszeit nicht durch Allokationen verlangsamt wird. Nach dem Initialisieren wird die .NET-assemblierte *Dynamic Link Library* (DLL) *kukaTCPClient* in MATLAB geladen, welche die Klasse *KukaClientClass* definiert und Sende- und Empfangsmethoden für eine TCP/IP-Verbindung bereitstellt. Es wird dann

eine Instanz von *KukaClientClass* erstellt und die ersten beiden Wegpunkte mit ihren Parametern sowie weitere Variablenwerte an die Steuerung gesendet. Zwischen den Schreib Anfragen wird noch 10 ms gewartet, damit auch die Antwortnachrichten von KVP beim Input-Buffer von *KukaClientClass* ankommen. Es wurde nämlich bei mehreren Schreiben Anfragen in einem Frame beobachtet, dass obwohl der KVP-Server alle Antworten nach kurzer Zeit sendet, diese nicht alle gleichzeitig im Buffer aufgenommen werden. Das zusätzliche Warten macht die Kommunikation robuster und eine längere Dauer ist bei der Initialisierung nicht relevant. Um den Buffer dann zu leeren, wird dieser zum Schluss ausgelesen. In Abbildung 27 ist der Kommunikationsverlauf für die Initialisierung gezeigt.

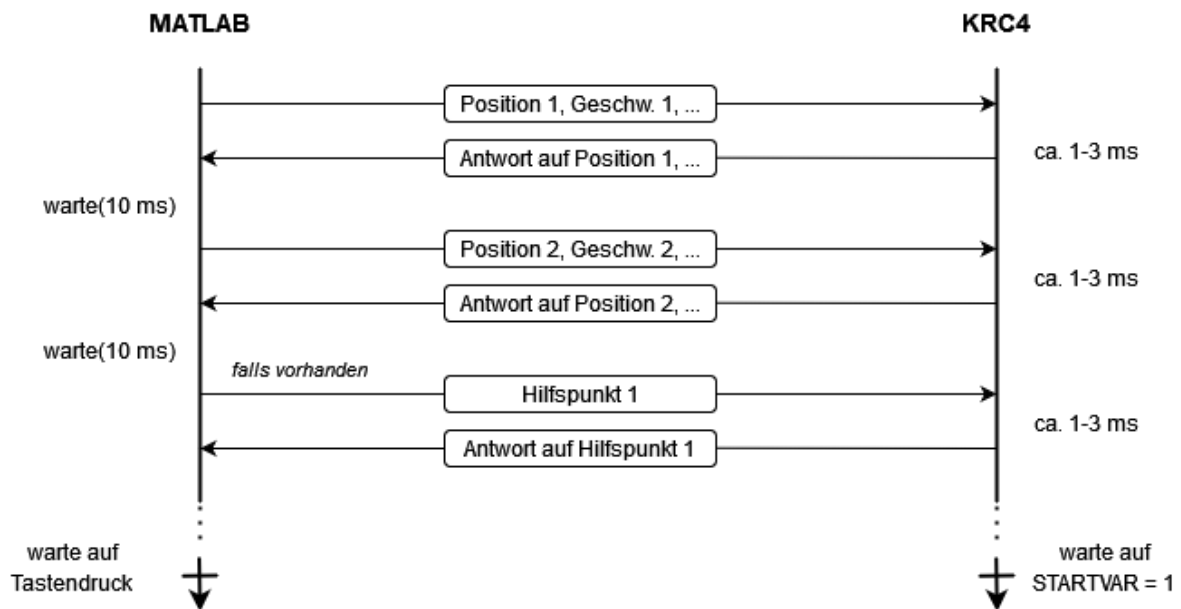


Abbildung 28: Darstellung des Kommunikationsverlaufs beim Initialisierungsvorgang in Bahnskript.m.

Das Programm wird danach pausiert und auf eine beliebige Tasteneingabe zum Fortsetzen gewartet, es ist jedoch zu beachten, dass nach 30s der KVP-Server automatisch die Verbindung beendet, sodass dann das Skript abgebrochen und nochmal neu gestartet werden muss. Nach dem Fortsetzen wird eine Schreib Anfrage der Variable STARTVAR auf 1 gesendet. Damit fängt das KRL-Programm mit der Bearbeitung der Schleife an, in welcher abwechselnd zuerst die Position 1 und danach die Position 2 angefahren wird.

In der Schleife wird entweder die KRL-Variablen FINISH 1/2 gepollt oder die nächsten Wegpunktwerte gesendet. In Abbildung 29 ist der Kommunikationsablauf gezeigt. Beim Abfragen wird zudem noch gleichzeitig eine Leseanfrage auf die Ist-Position des Roboterarms gesendet. Ist nun z.B. FINISH 1 wahr, ist die Teilbewegung mit den Werten der Variablen 1 beendet und es werden wie bei der Initialisierung neue Werte für Position und Parameter auf den Variablensatz 1 geschrieben. Es wird auch wieder eine feste Zeit gewartet damit alle Antworten ankommen und diese dann auf einmal ausgelesen. Im Polling-Teil wird dann FINISH 2 abgefragt. Um eine möglichst schnelle Zykluszeit zu erreichen, wird hier nur gewartet, bis die erste Antwort, welche der Wert von FINISH ist, im Input-Buffer ankommt. Beim Auswerten der Nachricht wird dann auch die Nachricht mit der Ist-Position im Buffer verfügbar. Wegen diesem Vorgang ist deshalb wichtig, dass der Input-Buffer der TCP/IP-Verbindung am Anfang eines Polling-Zyklus stets leer ist, da sonst das Programm bei einem nicht leeren Buffer weiterläuft, obwohl die Lese-Antwort auf FINISH noch nicht angekommen ist.

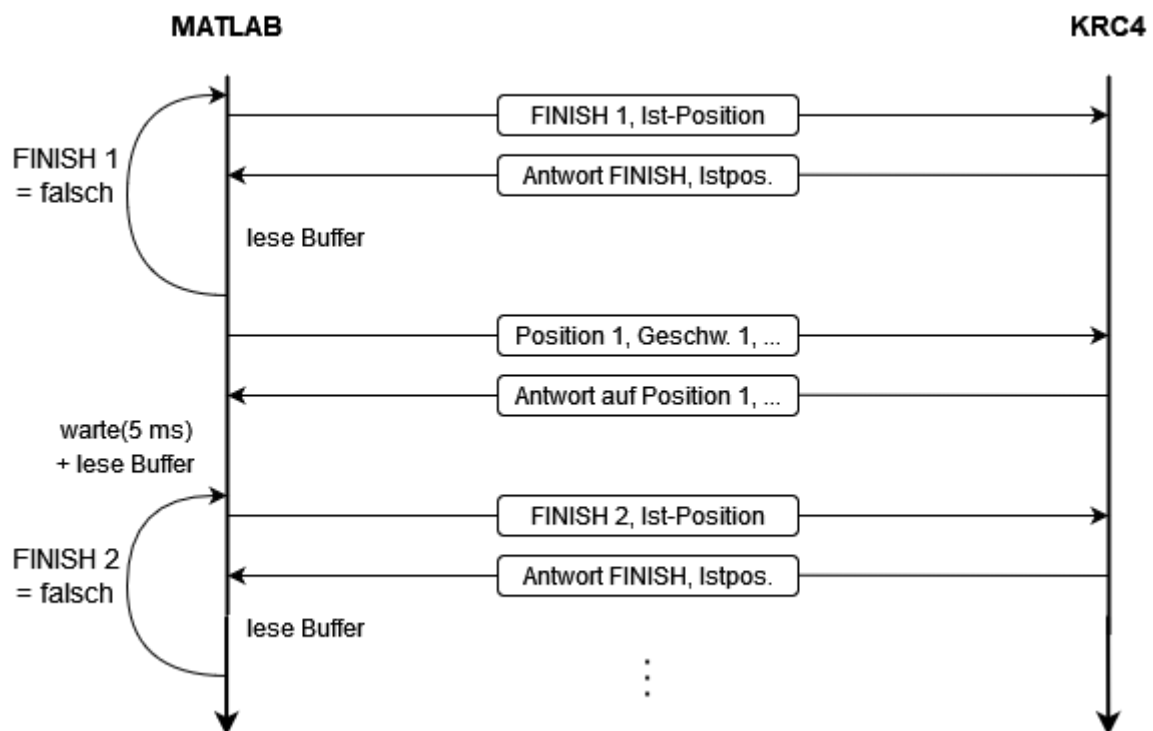


Abbildung 29: Darstellung des Kommunikationsverlaufs beim Programmablauf in Bahnskript.m

Weiterhin müssen noch aus der restlichen Antwortnachricht die Zahlenwerte für die Ist-Position extrahiert werden, welche diese als Kette von ASCII-Werten liefert. Die Schwierigkeit beim Auslesen liegt in der verschiedenen Anzahl der Zeichen, die ein Eintrag für eine Koordinate haben kann. So gehören bei $Y = 5.0$ nur drei Zeichen, aber bei $Y = -100.0$ sechs Zeichen zu dem Eintrag. Deshalb werden in dem uint8-Array der Nachricht die Positionen der ASCII-Werte für die Zeichen „X, Y, Z, A, B, C, S“ und „T“ mithilfe der MATLAB-Funktion *find(array==ASCII-wert)* gesucht. Die um zwei Stellen versetzten Indexe bilden den Startindex des Eintrags. Nach einem Eintrag folgt entweder ein Komma oder eine geschweifte Klammer am Ende der Nachricht. Es wird deshalb wieder mit *find* die Indexpositionen dieser Zeichen gesucht, welche dann um eins nach links versetzt den End-Index eines Eintrags bilden. In Abbildung 30 ist dieser Vorgang nochmal veranschaulicht. Mit dem Start- und End-Index werden nun die Zeichen von einem Eintrag ausgewählt und zuerst mit *char()* von ASCII-Dezimalwerten zu *char*-Zeichen konvertiert. Danach werden die Zeichen mit *str2double()* in Fließkommazahlen umgewandelt und schließlich in einem 1x8 Array ausgegeben.

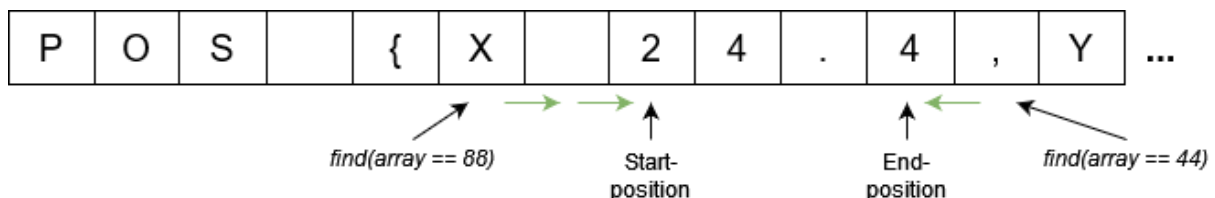


Abbildung 30: Veranschaulichung des Vorgangs zum Auswählen eines X-Koordinatenwertes von 24.4 mm.

Zum Schluss wird die Dauer des aktuellen Polling-Durchlaufs gemessen und falls diese kleiner als 12 ms ist, wird die restliche Zeit gewartet, da die Variablen in der Steuerung nur nach 12 ms aktualisiert werden. Das reduziert den Datenverkehr und verringert somit die Wahrscheinlichkeit von Latenz-Peaks. Wenn der Zähler für die Teilbewegungen sein Maximum erreicht hat, wird die KRL-Variable ENDPORG auf wahr gesetzt damit die Bahnfahrt nach der nächsten Bewegung beendet. Es wird im MATLAB-Workspace eine Matrix mit den aufgenommenen Ist-Positionen pro Polling-Zyklus abgelegt sowie zwei Grafiken mit dem Ist-Positionsverlauf und den Zykluszeiten beim Polling ausgegeben.

4.2. KRL-Skript Teil 1

Wo vorhin schon erwähnt, dient zum eigentlichen Ausführen der Bewegungsbefehle das KRL-Skript *driver*, welches von Variablen gesteuert wird, die von MATLAB-Funktionen überschrieben werden. Es umfasst die Programmteile für die Bahnfahrt, manuelle Bewegung und Zielpunktbewegung, welche über die Variable PROGVAR gewählt werden. In diesem Abschnitt wird der Teil für die Bahnfahrt vorgestellt und in Abschnitt 4.3 der Teil für die manuelle und Zielpunkt-Fahrt. Das Skript wird während dem Arbeiten an der Roboterzelle durchgehend ausgeführt und es ist so möglich zwischen den Programmteilen nur über MATLAB zu wählen. Zu Beginn von *driver* wird die manuelle Bewegung gewählt und eine Satzkoinzidenzfahrt (SAK) ausgeführt. Dabei wird mit reduzierter Geschwindigkeit und durchgedrückter Starttaste am SmartPAD zur Ausgangsposition des Arms gefahren. Dies ist am Anfang jedes KRL-Skriptes notwendig, damit der Arm seine aktuelle Position findet. Der POV-Override wird dabei temporär auf 100% gesetzt und dann wieder zurück auf den vorher eingestellten Wert, sobald die SAK-Fahrt beendet wurde. In Quelltext 2 ist die Struktur von *driver* gezeigt.

```
2      INI
3      PROGVAR = 2
4      ;SAK-Fahrt mit 100% Geschw.
5      temp = $OV_PRO
6      $OV_PRO = 100
7      SPTP XHOME CONT Vel=50 % PDAT1 Tool[1] Base[0]
8      $OV_PRO = temp
9
10     LOOP
11         SWITCH PROGVAR
12         CASE 1
13             Skript-Teil...
14
76        CASE 2
15             manuelle Bewegung...
16
123       CASE 3
17             Zielpunkt-Bewegung...
18
145       ENDSWITCH
146     ENDLOOP
```

Quelltext 2: Initialisierung und Endlosschleife des KRL-Skripts *driver.src*.

Die steuerungsrelevanten Variablen wurden davor global deklariert und außerdem wurde der Vorlauf für die Bahnberechnung ausgeschaltet, da dies zu einer fehlerhaften Ausführung führte. Das Programm pausiert dann, bis STARTVAR vom MATLAB-Programm auf wahr gesetzt wird. In Abbildung 31 ist das Ablaufdiagramm vom KRL-Programm für die Bahnfahrt

gezeigt. Über eine Steuerungsvariable wird dann ausgewählt, ob der Datensatz 1 oder 2 evaluiert wird. Es werden dann vor der Bewegung die FINISH-Variablen entsprechend gesetzt, sodass in *Bahnskript.m* der nächste Datensatz gesendet und auf die aktuelle Bewegung gewartet wird. Danach wird aus einen von vier möglichen Bewegungsaktionen gewählt, nämlich die vorhin vorgestellte PTP-, Linear- und Kreisbewegung und zusätzlich das Warten auf der vorhin erreichten Position. Die Bewegungsparameter werden dann aus den Variablen MAT_POS1, VEL1, usw. übernommen.

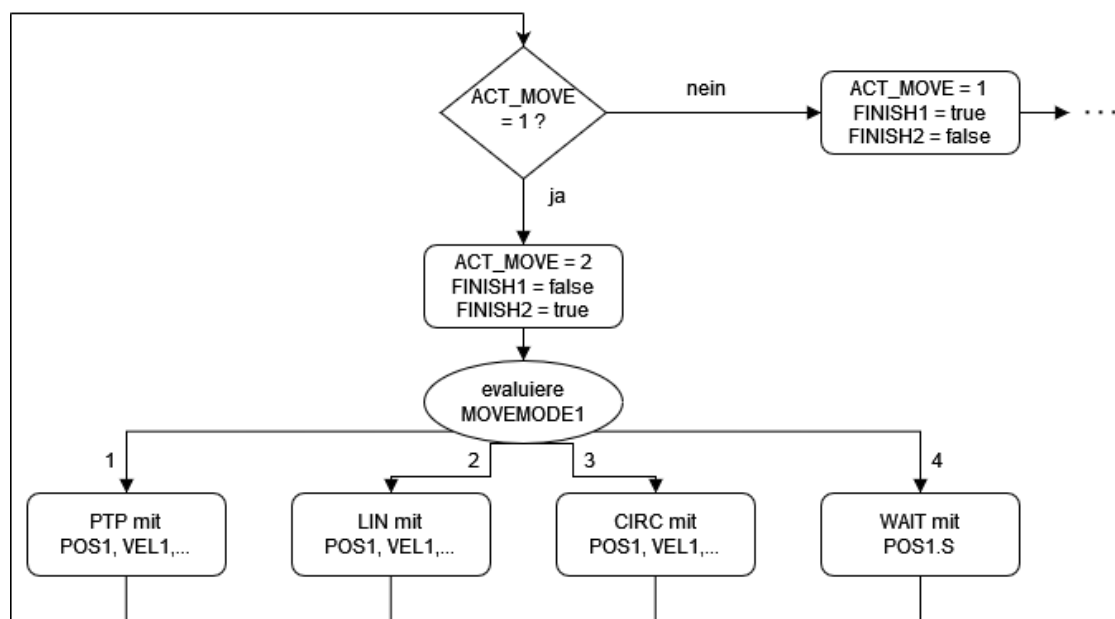


Abbildung 31: Ablaufdiagramm des KRL-Programms driver während einer Bahnfahrt.

Ist nun die Bewegung beendet, werden im nächsten Schleifendurchlauf die vorher beschriebenen Schritte für die jeweils anderen Parameter wiederholt. Um den Programmablauf zu beenden, muss dann ENDPORG auf wahr gesetzt werden. Es wird dann wieder auf den manuellen Bewegungsmodus gewechselt.

4.3. KRL-Skript Teil 2

Wie schon vorher erwähnt, bietet das KRL-Skript *driver* die Möglichkeit, den Roboterarm durch eine externe Peripherie direkt zu Bewegen. Es wurden dafür zwei MATLAB-Skripte zur

Steuerung mit einer Tastatur und mit einer Space-Maus geschrieben, die in Abschnitt 4.4 vorgestellt werden. In Abbildung 32 ist das Ablaufdiagramm für die manuelle Bewegung gezeigt. Damit die Schleife durchlaufen wird, muss die Variable ACTFLAG auf wahr gesetzt sein, ansonsten wird darauf gewartet. So ist eine bessere Kontrolle des Programmablaufs möglich und es ist auf dem SmartPAD ersichtlich, in welchem Codeabschnitt sich das KRL-Programm gerade befindet. Die Steuerung erfolgt in kartesischen Koordinaten und für die Bewegung wird der Befehl PTP_REL für eine Relativbewegung verwendet. Der Zielpunkt-Parameter wird dabei relativ zur aktuellen Ist-Position des Endeffektors interpretiert und die Bewegung erfolgt entlang der Ausrichtung des Basiskoordinatensystems. Für die Werte 1-6 von KEYFLAG wird die X-, Y- und Z-Achse jeweils positiv oder negativ in 2mm-Schritten entlanggefahren. Mit den Werten 7-12 von KEYFLAG wird die Rotation um die Achsen gesteuert. Die Abstufung erfolgt in Schritten von 0.5°. Zudem ist für eine gleichmäßige Bewegung der Vorlauf auf 1 gesetzt und das Überschleifen aktiviert, wodurch die interne Bahnplanung der Steuerung stets einen Bewegungsbefehl im Voraus berücksichtigt. Dadurch führt der Manipulator aber mindestens zwei Relativbewegungen aus, wodurch sich eine Präzision von 4mm ergibt.

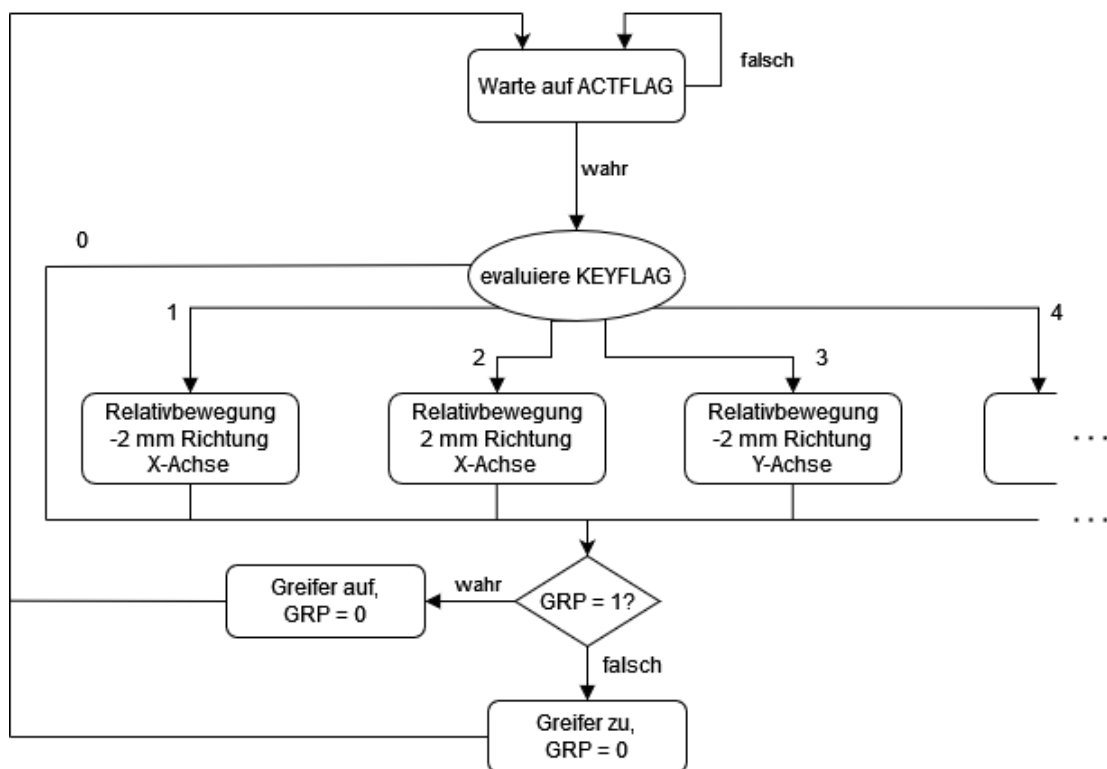


Abbildung 32: Ablaufdiagramm für die manuelle Bewegung in driver.

Es ist damit auch möglich, den Greifer manuell zu steuern. Dafür werden in jedem Schleifendurchlauf zusätzlich die Variable GRPFLAG geprüft. Diese ist standardmäßig auf 0 gesetzt und muss zum Öffnen oder Schließen des Greifers auf 1 oder 2 gesetzt werden. Beim Setzen von GRPFLAG wird der Greifer dann betätigt, 0,5s gewartet und GRPFLAG wieder auf 0 zurückgesetzt.

Eine weitere Möglichkeit zur manuellen Steuerung ist die Vorgabe eines konkreten Zielpunktes, welcher angefahren werden soll. Für einen PROGVAR-Wert von 3 wird wie in [42] zyklisch ein einziger Bewegungsbefehl abgearbeitet mit den Wegpunktdatensatz 1 als Parameter. Wenn der Zielpunkt erreicht wurde, dann wird in den folgenden Schleifendurchläufen der Bewegungsbefehl einfach übersprungen. In der Schleife kann auch wieder mit GRPFLAG der Greifer gesteuert werden. Diesmal wird aber der Wert nur auf 1 gesetzt, um den Greiferstatus zu ändern. Der Codeausschnitt für die Zielpunktbewegung ist in Quelltext 3 gezeigt. Das Setzen des Zielpunktes und der weiteren Parameter ist über die Methoden der Klasse *KukaWrapperClass* möglich, welche in Abschnitt 4.4 vorgestellt wird. Dadurch wird mehr Handlungsraum geboten, die Bewegung abhängig vom selbst erstellten Code zu beeinflussen, jedoch ist dies stark von der Latenz der Verbindung abhängig.

```
130  WHILE  ENDPROG == FALSE
131      SPTP MAT_POS1 Vel=50 % PDAT1 Tool[1] Base[0]
132      IF GRPFLAG == 1 THEN
133          IF GRP1 == 1 THEN
134              GRPg_SetStateAndCheck(1,2,0.5,0)
135              GRP1 = 2
136          ELSE
137              GRPg_SetStateAndCheck(1,1,0.5,0)
138              GRP1 = 1
139          ENDIF
140      GRPFLAG = 0
141  ENDIF
142  ENDWHILE
```

Quelltext 3: Codeabschnitt in driver.src für die Zielpunktbewegung.

4.4. Manuelle Bewegung

Mit den MATLAB-Skripts *Teachskript.m* und *Teachskript_Spacemaus.m* kann der Roboterarm über MATLAB manuell gesteuert und die Ist-Position des Endeffektors als Wegpunkte gespeichert werden. *Teachskript.m* verwendet dafür die Tastatur und funktioniert über ein *figure*-Objekt, welches bei Tastatureingaben eine zugewiesene Callback-Funktion *KeyressCb*

auslöst. Dies ist im Moment die einzige Möglichkeit in MATLAB, Tastatureingaben eventbasiert zu erkennen und zu verarbeiten. In dieser Funktion wird geprüft, welche Tasten gedrückt wurden und anhand dessen werden dann die richtigen Werte für KEYFLAG an die Steuerung gesendet. Eine Tabelle mit den Tastenkombinationen, Werten für KEYFLAG und den Bewegungsrichtungen ist mit

Tabelle 6 gegeben.

KEYFLAG-Wert	1/2	3/4	5/6	7/8	9/10	11/12
Tasten	Ctrl + ↑ / ↓	← / →	↑ / ↓	Shift + ← / →	Shift + ↑ / ↓	Alt + ↑ / ↓
Bewegung	+ / - X-Achse	+ / - Y-Achse	+ / - Z-Achse	+ / - Z-Drehung	+ / - Y-Drehung	+ / - X-Drehung

Tabelle 6: Zugewiesene Tasten und Bewegungsrichtungen für die KEYFLAG-Werte.

Ist nun eine der gültigen Tastenkombinationen gedrückt worden, wird an den KVP-Server die entsprechenden Werte für KEYFLAG und ACTFLAG gesendet, damit das KRL-Skript die Bewegung auch ausführt. Da aber der Callback bei gedrückter Taste ständig auslöst, setzt *KeyPressCb* einen internen Hilfsflag beim erstmaligen Senden und kann auch nur senden, wenn dieser Flag nicht gesetzt ist. Nach dem Prinzip einer binären Semaphore setzt dann der Callback für das Loslassen der Taste diesen Flag zurück, sodass dieser beim Drücken einer anderen Taste wieder gesendet werden kann, und setzt zudem KEYFLAG und ACTFLAG auf ihre Ursprungswerte zurück. Dadurch wird der Datenverkehr reduziert, was das Risiko für Latenzspitzen und sonstige Fehler reduziert. In Abbildung 33 ist ein Schaubild für den Ablauf gezeigt.

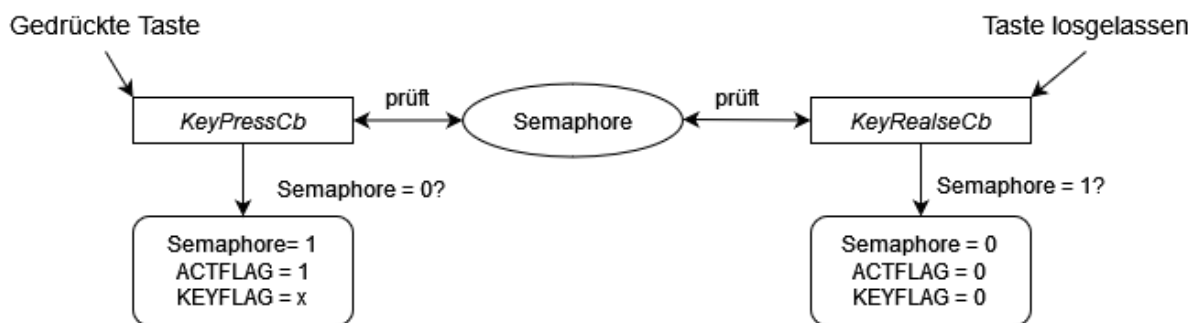


Abbildung 33: Schaubild für das Funktionsprinzip von Teachskript.m.

Mit der Leertaste lässt sich auch die aktuelle Ist-Position des Endeffektors abfragen und zwischenspeichern. Dafür wird wie im MATLAB-Verbindungsskript die interne KRL-Variable für die Ist-Position gelesen und aus dem Antwort-Char-Array die Zahlenwerte extrahiert. Die Werte werden dann im *figure*-Objekt in einer Matrix zwischengespeichert. Wird zusätzlich zur Leertaste Ctrl gedrückt, wird die Ist-Position als Hilfspunkt für eine Kreisbewegung anstelle eines Wegpunktes gespeichert. Der Hilfspunkt bezieht sich dabei immer auf den nächsten aufzunehmenden Wegpunkt. D.h. wenn z.B. schon 4 Wegpunkte aufgenommen wurden, wird mit Ctrl+Leertaste der Hilfspunkt, auch beim mehrmaligen betätigen nur für Wegpunkt 5 gespeichert, bzw. aktualisiert. Wird ein Wegpunkt ohne vorher aufgenommenen Hilfspunkt gespeichert, dann wird als Hilfspunkt ein Null-Array übernommen.

Mit der Taste G kann der Greifer geöffnet, bzw. geschlossen werden. Über eine interne MATLAB-interne Variable *gripflag* wird geprüft, welchen Status der Greifer hat und die Variable GRPFLAG in *driver* geschrieben. Mit dem Wert 1 wird er geschlossen und mit 2 geöffnet. Bei der Aufnahme eines Wegpunktes wird mithilfe von *gripflag* ebenfalls der aktuelle Greiferstatus gespeichert. Um das MATLAB-Programm zu beenden, muss Escape gedrückt werden. Dadurch werden die aufgenommenen Wegpunkte, Hilfspunkte und Greiferstellungen unter den Namen *waypoints*, *hilfspoints* und *grpstat* im Workspace abgelegt und können dann weiterverwendet werden.

Das Skript *Teachskript_Spacemouse.m* verwendet die MATLAB-Toolbox *Simulink 3D animation* mit der die Werte einer über USB angeschlossenen Space-Maus abgefragt werden können. Im Labor wurde dafür ein Modell von 3Dconnexion verwendet, welche noch zusätzlich zwei Tasten bereitstellt, die ebenfalls mit der Toolbox gelesen werden können [45]. Der Anfang des Skripts ist mit *Teachskript.m* gleich, jedoch wird anstatt eines *figure*-Objekts ein *spacemouse*-Objekt initialisiert. Die Funktionen dafür sind jedoch nicht Event-basiert, weshalb der Hauptteil des Skripts eine while-Schleife ist, womit jeder Achswert der Maus zyklisch nacheinander abgefragt wird. Eine Darstellung des Programmablaufs ist in Abbildung 34 gezeigt. Da die beiden Spacemouse-Tasten anderweitig verwendet werden, besteht aber keine Möglichkeit, die Schleife über die Maus zu beenden und es muss deshalb das Programm über MATLAB zuerst pausiert und dann gestoppt werden.

Der Wert für eine Achse wird bei vollem Ausschlag der Maus um ca. 1.9/ms stetig auf bzw. runterintegriert, weshalb pro Zyklus die Differenz von aktuellem und letztem Wert betrachtet wird. Liegt dabei der Wert für eine Achse über dem Schwellwert *deadzone* und ist die MATLAB-Variable *keyflag* gleich null, dann wird *keyflag* auf den entsprechenden Wert der Achse und Bewegungsrichtung gesetzt und mit Nachrichten für KEYFLAG und ACTFLAG die Bewegung gestartet. Es ist aber zu beachten, dass die *keyflag*-Werte für in *Teachskript_Spacemaus.m* andere sind als in *Teachskript.m*, da die Schnittstellenfunktion der MATLAB-Toolbox die Achsen in einer anderen Reihenfolge ausgibt. In den folgenden Zyklen wird dann geprüft, ob der Achswert der aktuellen Bewegung den Schwellwert unterschreitet. Falls ja, wird *keyflag* zu null gesetzt und KEYFLAG und ACTVAR auf null bzw. *false* geschrieben, um die Bewegung zu beenden.

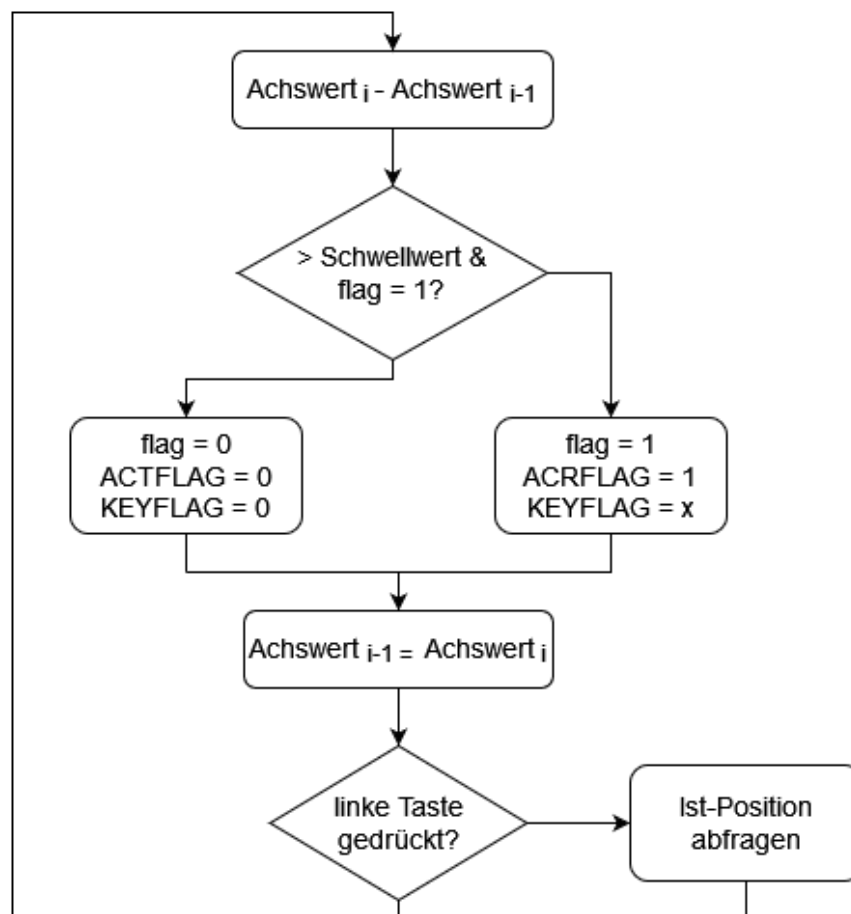


Abbildung 34: Ablaufdiagramm für Teachskript_Spacemaus.m

Mit der von Vorne betrachtet linken Taste wird wie mit der Leertaste in *Teachskript.m* ein Wegpunkt aus der aktuellen Armposition und der Greiferstatus gespeichert. Da die Prüfung der linken Taste in einer Schleife geschieht, wird mit einer Hilfsvariable *button_pressed* sichergestellt, dass pro Tastendruck der Codeteil für die Wegpunkterstellung nur einmal ausgeführt wird. Wird mit der linken auch die rechte Taste gedrückt, dann wird ein Hilfspunkt gespeichert.

Wie schon vorher erwähnt, bietet die Klasse *KukaWrapperClass.m* Methoden zur gezielten Bewegung des Roboterarms auf eine bestimmte Position. Die Codeimplementierung entspricht zum großen Teil der Implementierung in *Bahnskript.m*. Zur Initialisierung wird dem Konstruktor der Dateipfad der DLL-Datei *kukaTCPClient* übergeben. Mit der Methode *Connect()* wird eine Verbindung zur Steuerung aufgebaut und eine Instanz der DLL-Klasse übergeben. Da die Attribute der Klassen in MATLAB nur eingeschränkten Typen entsprechen dürfen, sind Objekte von externen Klassen wie *kukaTCPClient* als Attribut nicht möglich und deshalb dient die mit *Connect()* ausgegebene Instanz von *kukaTCPClient* als Argument für alle weiteren Methoden. Mit *sendTargetPosition(target, tcpObj)* wird dann die Zielposition gesendet, wobei sie mit *target* als Array mit double-Einträgen übergeben wird in der Methode dann zum uint8-Nachrichtenarray umgewandelt wird. Mit *getActPosition(tcpObj)* wird die Ist-Position abgefragt und ebenfalls als double-Array zurückgegeben. Es existieren noch weitere Methoden zum Lesen und Setzen der Geschwindigkeit und des Greifers, sowie zum Beenden der Verbindung.

4.5. Generieren der Nachrichten

Mit der MATLAB-Funktion *generate_posmsg.m* werden aus den numerischen Werten für die Wegpunktkoordinaten und weiteren Parametern Byte-Arrays mit Schreibnachrichten für den KVP-Server erzeugt. Als Eingabe werden Arrays und Matrizen der Wegpunktparameter außer FINISH in dem mit Tabelle 5 vorgestelltem Format erwartet. Am Anfang der Funktion werden die Argumente auf das gültige Format und Wertebereich geprüft. Für eine bessere Benutzerfreundlichkeit wird bei Unstimmigkeiten eine Fehlermeldung als Feedback ausgegeben.

Nach der Prüfung werden durch jeden Wegpunkt iterierend die Werte mithilfe von *num2str* und *int2str* in Char-Arrays umgewandelt und zwischengespeichert, wie in Quelltext 4 gezeigt. Da nicht für jeden Wegpunkt ein Hilfspunkt definiert sein muss, werden die Werte für die Hilfspunkte nur umgewandelt, wenn diese nicht aus Nullen bestehen. Falls ein Nullarray vorliegt und die Bewegungsart 3, also eine Kreisbahn, gewählt wurde, wird ein Fehler ausgegeben, da hierfür ein Hilfspunkt benötigt wird, ansonsten wird das Char-Array des dazugehörigen Wegpunktes übernommen, da die Nachricht für den Hilfspunkt dann sowieso nicht gesendet wird. Zum Schluss werden die Matrizen, bzw. Cell-Arrays mit den Schreibanfragen auf die KRL-Variablen für die Bahnfahrt erstellt. Ist der Index des aktuellen Wegpunktes ungerade, wird die Anfrage für die entsprechende Variable Nr.1 erstellt, bei geradem Index für die Variable Nr.2. Für die einzelnen Nachrichten werden die in

Tabelle 7 gezeigten IDs verwendet, um in Datenmonitoringprogrammen wie z.B. WireShark den Kommunikationsverlauf besser analysieren zu können.

Nachricht	Index
Movemod1/pos1/vel1/grp1	1/3/5/7
Movemod2/pos2/vel2/grp2	2/4/6/8
Read1/read2 (finish)	90/91
Readpos/readvel/readacc	74/75/76
Fin1/fin2	30/31
Start/endprog	10/20

Tabelle 7: Zuweisung von Nachrichten-IDs zu den jeweiligen Lese- und Schreibnachrichten der Bahnfahrt-Variablen.

Nach dem Durchlaufen aller Wegpunktswerte werden noch Schreibanfragen mit den gleichen Werten wie die letzte Nachricht, jedoch für die jeweils anderen Variablen erstellt. Wenn z.B. 5 Wegpunkte gegeben sind, dann sind die letzten Nachrichten Schreibanfragen auf MAT_POS1, MOVEMODE1, usw. Es werden dann aber noch Anfragen auf MAT_POS2, usw. mit denselben Werten wie aus den Anfragen Nr. 5 generiert, sodass insgesamt 6 Anfragen vorliegen. Der Grund dafür ist, dass beim Abarbeiten von *Bahnskript.m* nach dem Senden der Schreibanfragen Nr.5 kein Polling mehr auf eine FINISH-Variable und somit auch nicht auf die Ist-Position erfolgen würde. Mit Hinzufügen der 6. Anfragen, wird somit auch während der letzten Bewegung die Ist-Position aufgenommen. Die Funktion *generate_posmsg* gibt dann die uint8-Matrizen mit n Zeilen für die Funktion *Bahnskript.m* aus, wobei n für die Anzahl der Wegpunkte steht.

```

56     for i = 1:size(waypoints,1)
57         % Werte konvertieren und zwischenspeichern
58         modval = uint8(int2str(movemode(i)));
59         posval = uint8(['POS: X ', num2str(waypoints(i,1), '%.1f'), ...
60             velval = num2str(vel(i), '%3.1f');
61             :
70         % Byte-Arrays erstellen
71         if(mod(i,2))
72             modmsg(i,:) =
73                 uint8([0,11,0,3+9+3,1,0,9,uint8('MOVEMODE1'),0,1,modval]);
74             posmsg{i} =
75                 uint8([0,13,0,3+8+length(posval)+2,1,0,8,
76                     uint8('MAT_POS1'),0,length(posval),posval]);
77             :
81         else
82             modmsg(i,:) =
83                 uint8([0,22,0,3+9+3,1,0,9,uint8('MOVEMODE2'),0,1,modval]);
84             :
135     end
136     % Varargin prüfen
137     :

```

Quelltext 4: Codeteil in *generate_posmsg.m* für das Konvertieren der Bahnfahrt-Parameter in Byte-Array-Nachrichten.

Neben den vorher beschriebenen Argumenten kann *generate_posmsg* über *varargin* auch noch weitere optionale Argumente aufnehmen. Es ist damit dann möglich, ein Array zu übergeben, wo mit jedem Wegpunktparameter noch weitere Variablen gelesen oder geschrieben werden können. Es wird zuerst geprüft, ob auch nur ein zusätzliches Argument vorliegt und ob es ein ganzzahliges Array der Länge *n* ist. Anhand den Nummern in dem Array können dann in einer switch-case-Struktur zwischen verschiedenen benutzerdefinierten Nachrichten gewählt werden, die dann in einem Cell-Array ausgegeben werden. Die vorhandenen Nachrichten und Nummernzuweisungen lassen sich beliebig erweitern. So können während der Bahnfahrt z.B. Systemoptionen wie der Orientierungsmodus des Endeffektors geändert werden. Ist für eine Nummer kein Fall definiert, dann wird ein leeres Array ausgegeben.

4.6. Laufzeittest

Ein zentraler Teil der Anbindung zwischen der Steuerung und dem Labor-PC sind die verwendeten Lese- und Schreibmethoden der TCP/IP-Verbindung. In MATLAB lassen sich verschiedene Funktionen dafür einsetzen, weshalb diese auf ihre Ausführungszeit untersucht wurden. Die Ergebnisse davon werden in diesem Abschnitt vorgestellt. Es wurden hierfür im Polling-Teil von *Verbindungsskript* die Befehle *tic* und *toc* verwendet und für jeden Schlei-

fendurchlauf die Zeiten davon gespeichert. Der Overhead für das Ausführen von *tic/toc* liegt bei einigen μs , weshalb die Befehle sich kaum auf die Ergebnisse auswirken und mit *tic/toc* zudem das Erkennen von Peaks in einzelnen Zyklen möglich ist. Konkret wurden die MATLAB-internen Funktionen *fread*, *fwrite*, *read*, *write*, die Methoden der *java.net.Socket* Klasse und die Methoden der *NetworkStream*-Klasse des .NET-Frameworks untersucht.

Für die Versuche wurde eine Strecke von 4 Punkten, wie in Abbildung 35 gezeigt, mit PTP-Bewegungen und einer Geschwindigkeit von 50% bzw. 30% bei der zweiten Methode abgefahren. Nach jeder Fahrt wurden dann Graphen mit den Laufzeiten der einzelnen Abschnitte des Poll-Zyklus erstellt. Neben der Gesamtdauer wurden zudem die Dauer für das Versenden, das Warten auf lesbare Nachrichten und das Lesen selbst gemessen.

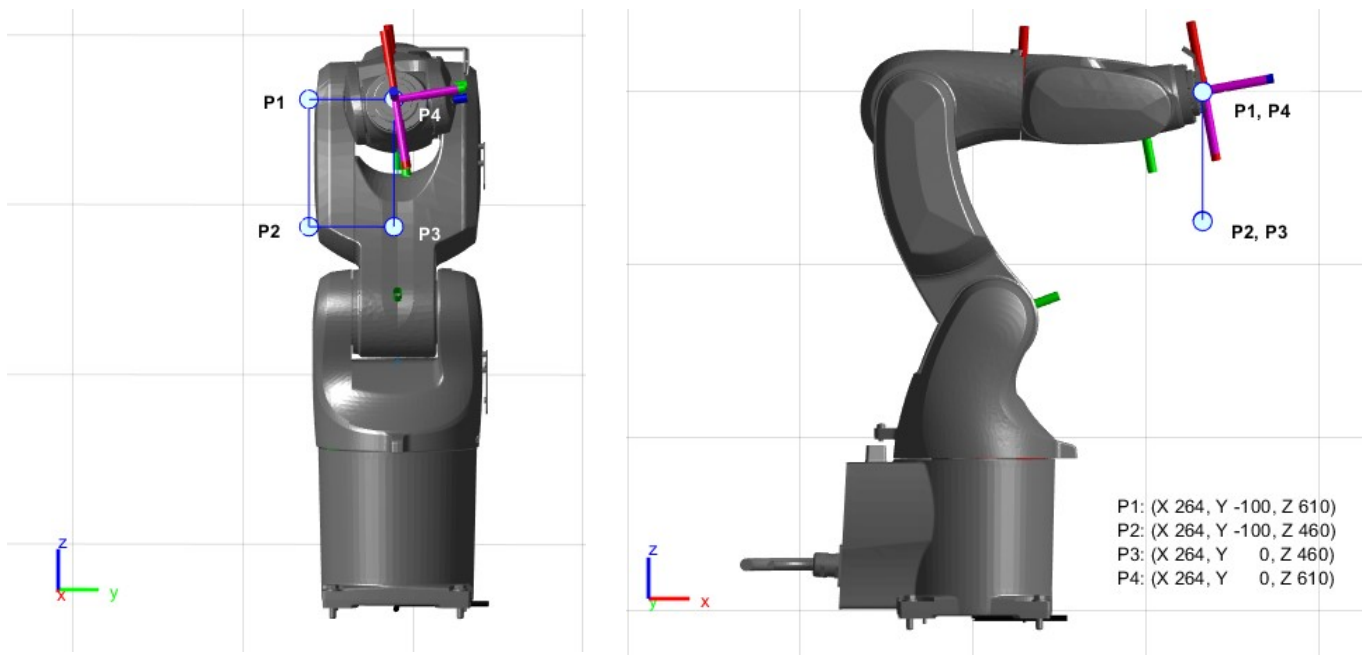


Abbildung 35: 3D-Modell des Manipulators mit eingezeichneten Wegpunkten der Testbahn.

Zuerst wurden die Funktionen *fread/fwrite*, welche von der *Instrument Control Toolbox* (ICT) bereitgestellt werden, und *read/write*, welche MATLAB-intern sind, untersucht. Beide funktionieren mithilfe von TCP/IP-Objekten, die I/O-Buffer und weitere Funktionalitäten bereitstellen. Bei *fread* muss beim Lesen die Anzahl der zu lesenden Bytes angegeben werden, da sonst der Befehl für einige Sekunden pausiert, was bei *read* nicht der Fall ist. Bei den Versuchen wurde zuerst nur die FINISH-Variable und danach zusätzlich die Ist-Position angefragt. Bei *fread* wurde nach jedem Zyklus der Input-Buffer geleert, da stets nur eine feste Anzahl

und somit nicht alle Bytes gelesen werden. Um einen möglichst schnellen Zyklus zu erreichen, wurde bei *fwrite* nach dem Senden mit *BytesAvailable()* gewartet, bis Nachrichten im Input-Buffer ankamen, bei *write* jedoch lief dabei die Funktion weiter, obwohl der Buffer beim Lesen leer war. Deshalb wurde eine bei *read/write* stattdessen eine feste Mindestzeit zwischen Senden und Lesen gewartet. In Abbildung 36 sind die Laufzeitmessungen graphisch dargestellt.

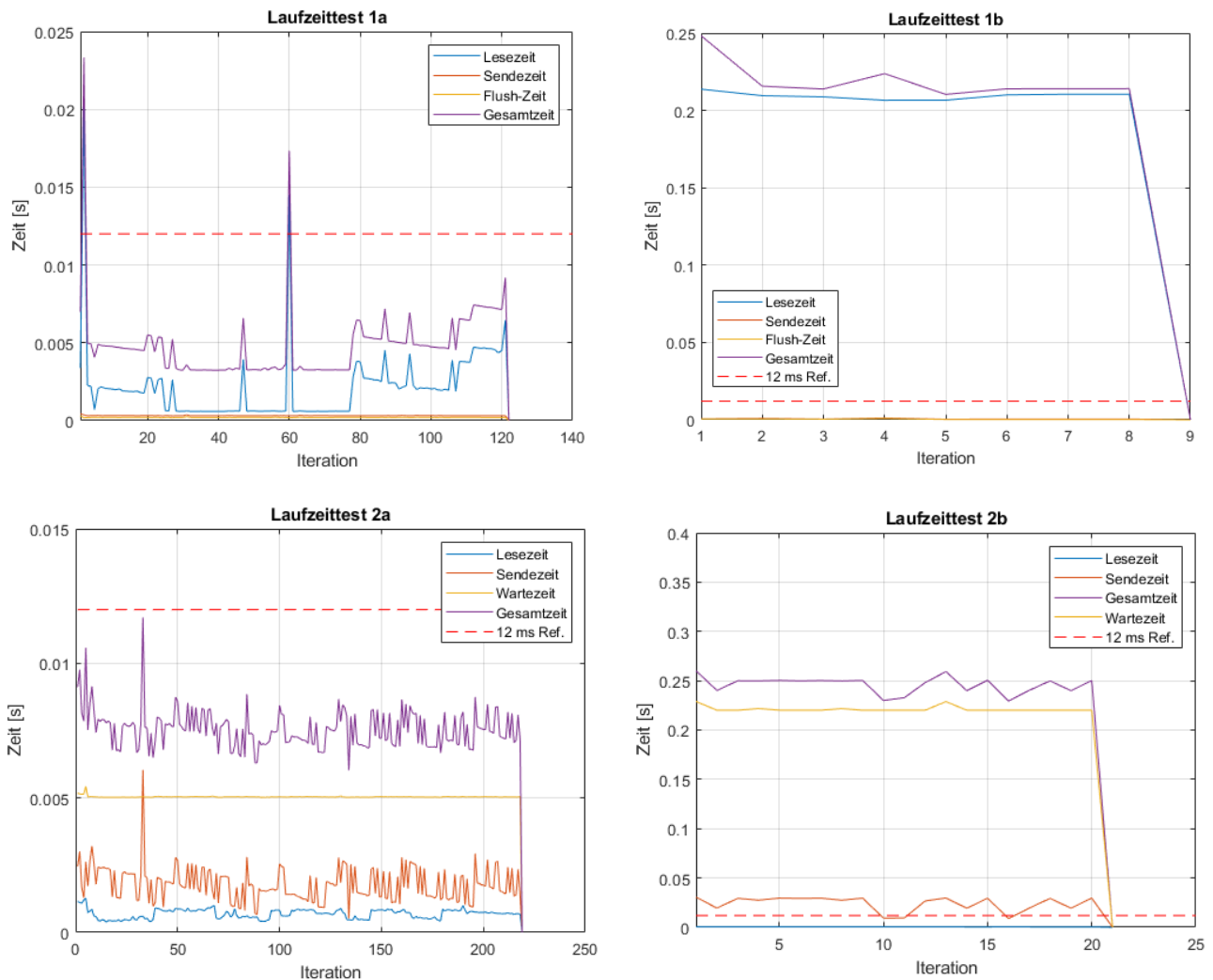


Abbildung 36: Zykluszeiten für jede Polling-Iteration während des Tests von *fread/fwrite* (1) und *read/write* (2). Es wurde im ersten Durchlauf nur FINISH (a) und danach zusätzlich die Ist-Position (b) angefragt.

Beim Graphen 1a wurde mit *fread/fwrite* und bei 2a mit *read/write* nur FINISH gepollt. Bei 1b wurde mit *fread/fwrite* und bei 2b mit *read/write* zusätzlich die Ist-Position angefragt. Beim Polling auf nur FINISH sind zwar einige Peaks zu erkennen, die Zeiten liegen aber im Schnitt unter 12 ms. Im zweiten Fall verlängert sich die Lesezeit von *fread* deutlich auf über 200 ms. Bei *read* musste ebenfalls die Wartezeit zwischen Senden und Lesen von 5 ms auf

mehr als 200 ms erhöht werden, da zum Lesezeitpunkt sonst nicht alle Bytes der Antwortnachrichten im Input-Buffer vorhanden waren. Es lässt sich vermuten, dass die interne Implementierung der TCP/IP-Objekte von MATLAB ein Grund für diese Problematik ist. Somit wird deutlich, dass die untersuchten Funktionen für die Lösungsimplementierung wenig geeignet sind, denn es ist damit keine sinnvolle Aufnahme der Ist-Position möglich und außerdem können bei schnelleren Roboterbewegungen als die Zykluszeit Fehler beim Bewegungsablauf auftreten.

MATLAB kann auch mit Java und .NET erstellte Klassen verwenden. Es wurde deshalb eine Kommunikation mit *java.net.Socket* und *System.Net.Sockets* der .NET-Bibliothek getestet. In MATLAB erstellte Instanzen davon liefern wiederum Instanzen von Input- und Output-Streams, über welche die eigentliche Kommunikation verläuft. In [46] wurden Hilfsfunktionen zum Verwenden der Java-Klasse geschrieben, welche für den Laufzeittest benutzt wurden. Zur Verwendung der .NET-Klasse wurde ähnlich wie in [43] eine DLL in C# erstellt und in MATLAB mit dem Befehl *NET.addAssembly* importiert. Bei Beiden wurde nach dem Senden auf Bytes im Buffer des Input-Streams gewartet und bei der Java-Klasse dann alle verfügbaren Bytes ausgelesen, während bei der .NET-Klasse stets 1500 Bytes gelesen wurden. Wieder wurde zuerst nur FINISH, danach auch die Ist-Position gepollt. Die Ergebnisse dieser Laufzeittests sind in Abbildung 37 graphisch gezeigt.

Im ersten Fall liegen Beide unter 12 ms, wobei bei der Java-Klasse die Wartezeit und bei der .NET-Klasse die Lesezeit deutlich schwankt. Wird jedoch die Ist-Position zusätzlich abgefragt, so steigt die Wartezeit bei der Java-Klasse, bis alle Antworten im Buffer angekommen sind, wie beim Durchlauf mit *read/write* deutlich auf über 200 ms an. Die Kommunikation über *System.Net.Sockets* ist jedoch nicht davon betroffen und die Ausführungszeiten der Lese-funktion ändern sich auch bei einer größeren Anzahl Bytes im Input-Buffer nicht. Somit ist die .NET-Klasse von den verglichenen Methoden am besten für die Implementierung im Labor geeignet. Es ist zwar eine Bahnfahrt auch mit den anderen Methoden möglich, durch das deutlich schnellere Auslesen des Antwort-Frames mit der Ist-Position sind die räumlichen Abstände der einzelnen Ist-Positionen viel geringer, wodurch ein erkennbarer Positionsverlauf entsteht. In Tabelle 8 sind nochmal die Werte der Laufzeitmessungen zusammengefasst.

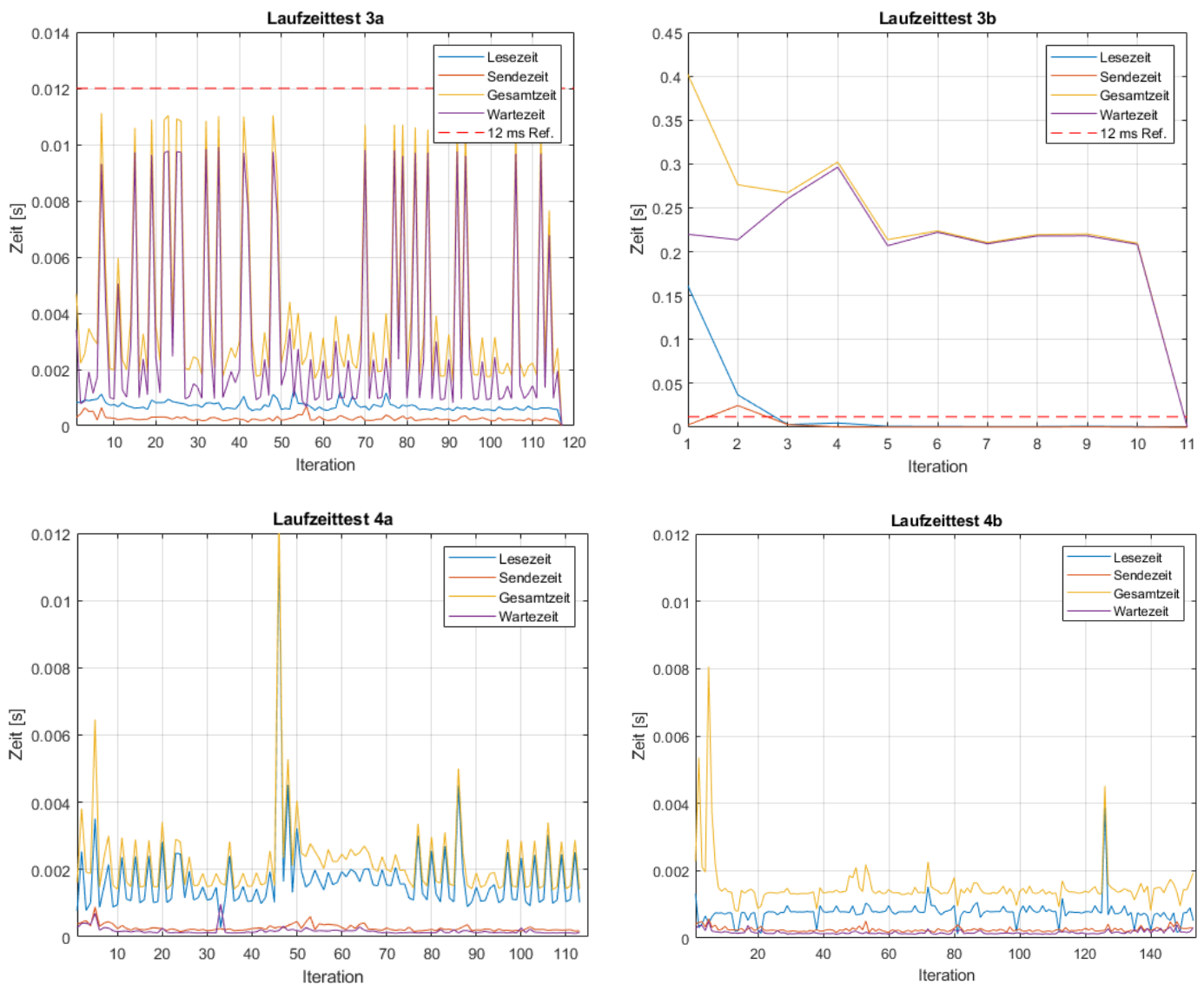


Abbildung 37: Zykluszeiten für jede Polling-Iteration während des Tests der Java-(3) und .NET-Klassen (4). Es wurde im ersten Durchlauf nur FINISH (a) und danach zusätzlich die Ist-Position (b) angefragt.

Gesamt-Zykluszeit Werte [ms]	<i>fread/fwrite</i>	<i>read/write</i>	Java-Klasse	.NET-Klasse
Durchschn. (Fall A)	4,8	7,5	4,1	2,2
Max. (Fall A)	23,3	11,7	11,1	12,5
Median (Fall A)	4,6	7,5	2,6	1,9
Durchschn. (Fall B)	219,2	210,6	231,4	1,5
Max. (Fall B)	248,3	229	402	8,1
Median (Fall B)	214	220	220,2	1,4

Tabelle 8: Ergebnisse der Laufzeitmessungen. Bei Fall A wurde nur FINISH, bei Fall B auch die Ist-Position angefragt.

5. Applikation

In diesem Kapitel zwei Beispielapplikationen vorgestellt, die mit den vorher beschriebenen Programmteilen erstellt wurden. Wie schon in Abschnitt 2.3 erwähnt, ergeben sich durch die Konstruktionsmöglichkeiten des Greifers nur eingeschränkte Anwendungsmöglichkeiten. Die erste Applikation ist daher das Stapeln von vier mit der Zelle mitgelieferten Würfel zu einem Turm in wird in Abschnitt 5.1 beschrieben. Später wurden die Kneifer des Greifers durch Zangenkonstruktionen ersetzt, welche mithilfe eines 3D-Druckers selbst erstellt wurden. Damit ist es dann auch möglich, eine kleine Flasche zu greifen. In Abschnitt 5.2 ermöglicht die zweite Applikation entsprechend das Einschenken eines Glases mit Wasser. In Abschnitt 5.3 wird auf die singulären Stellungen des Arms, auf was es bei der Bewegungsführung zu beachten gilt und auf die Bedeutung der Status- und Turn-Werte der Positionskoordinaten eingegangen.

5.1. Beispielapplikation 1

Bei dieser Applikation wurden vier Würfel mittig auf die Positionen der Löcher 4O, 6O, 4M und 6M des Arbeitsraumes der Schulungszelle platziert. Mit dem Skript *Bahnskript.m* wurden die in Abbildung 38 dargestellten Punkte angefahren und danach im Workspace *Appl_Wuerfel_Punkte.mat* gespeichert. Im Skript *Appl_Wuerfel.m* wird der Workspace dann geladen und zum Stapeln eines Würfels die in Abbildung 38 gezeigte Sequenz erstellt. Nach diesem Schema ließen sich noch weitere Würfel stapeln.

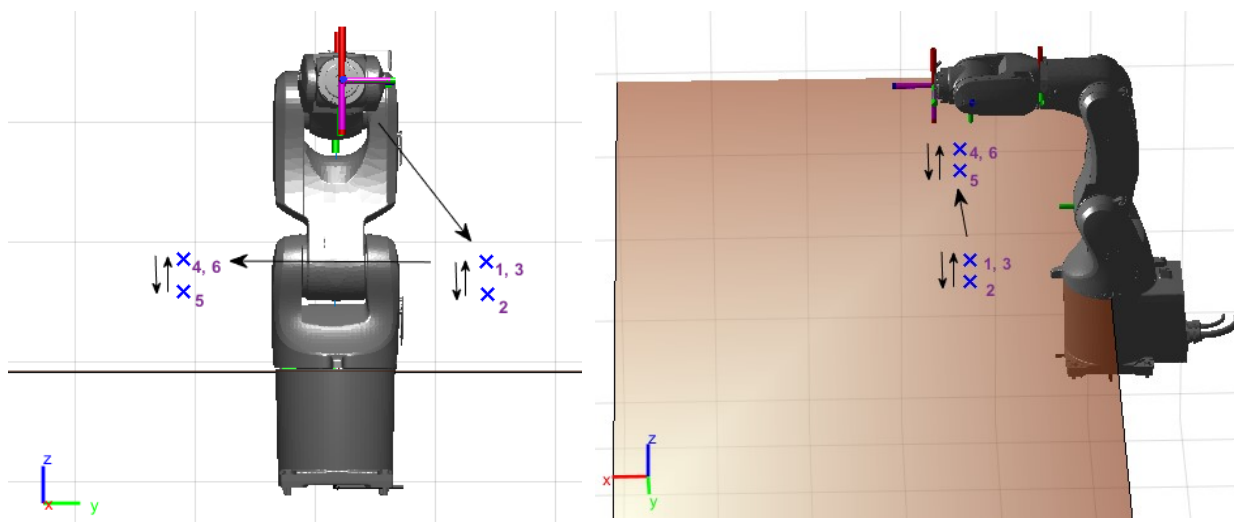


Abbildung 38: Darstellung der Sequenz der angefahrenen Punkte in der Beispielapplikation 1.

In Abbildung 39 ist der Ist-Positionsverlauf und die Zykluszeiten beim Polling während der Bewegungsausführung gezeigt. In diesem Fall treten in einigen Fällen Zeiten von deutlich über 12 ms auf, dessen Ursache aber nicht konkret festgestellt werden kann. Eine Vermutung ist das nicht deterministische Betriebssystem des Labor-PCs, wo bei längeren Programmlaufzeiten wohl früher oder später ein höherpriorisierter Task entweder MATLAB oder die TCP/IP-Übertragung stört. Da das Bahnfahrtprogramm jedoch mit Hinblick auf mögliche Latenzspitzen entwickelt wurde, ist die eigentliche Bewegung davon nicht beeinflusst.

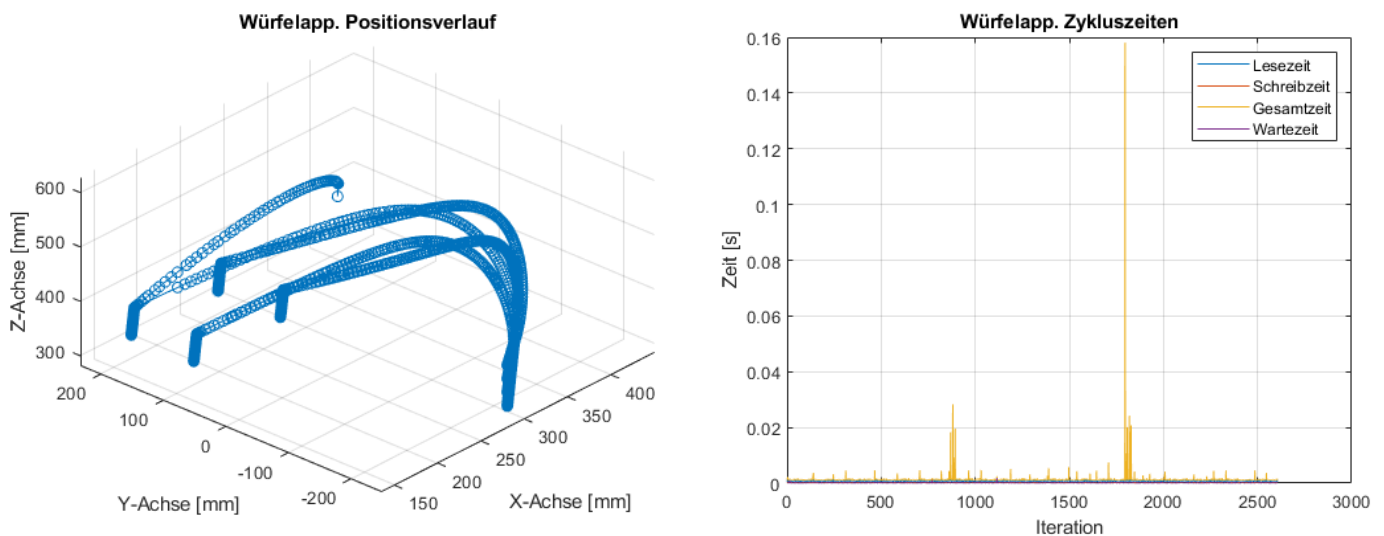


Abbildung 39: (Links) Ist-Positionsverlauf des Endeffektors während der Würfelapplikation. (Rechts) Zykluszeiten des Pollings während der Würfelapplikation.

5.2. Beispielapplikation 2

Bei dieser Applikation war das Ziel, das der Roboterarm eine Flasche greifen und ein Glas Wasser füllen kann. Eine Idee zum ermöglichen des Greifens war das Anbringen der Würfel an die Flasche, die der Arm dann über die Würfel bewegen konnte. Stattdessen wurden aber die Pinzetten an den Greiferbacken entfernt und eigens konstruierte Zangen befestigt, da somit die Flasche stabiler bewegt werden kann und keine speziell präparierte Flasche mit einem Würfel benötigt wird, was die Applikation flexibler macht. Die Zangen wurden mit FreeCAD entworfen und sind in Abbildung 40 zu sehen. Die Form der Zangen wurde dabei speziell auf die verwendete Flasche angepasst, welche an dem Greifpunkt einen Durchmesser von 4 cm hat und mit einem Flachbandgummi für eine höhere Griffbarkeit umwickelt ist. Weiterhin sind die Zangen von den Greiferbacken um 45° angewinkelt, damit beim Greifen

der Arm nicht in eine Singularität fährt. Die Dicke der Zangen beträgt 3 mm, da sie sonst zu biegsam sind und nicht genug Druck auf die Flasche ausüben. Zudem wurden beim Modellieren für eine bessere Rutschfestigkeit Rillen in die Innenseite eingefügt. Das Zangenmodell wurde dann in eine *Standard Triangle Language* (STL)-Datei exportiert und mit 3D-Drucker gedruckt. Als Material wurde Polyactid (PLA) genommen, da dies eine hohe Festigkeit hat, und das Formstück wurde zu 100% ausgefüllt, sodass keine Hohlräume in der Zange vorliegen.

Ebenso wurden für die Flasche und das Glas eine Halterung gedruckt, die an einem Unterlege-Tablett fixiert sind. Das Tablett wiederum ist an Metallstiften ausgerichtet, welche in den Löchern 1F, 4F, 6H und 6M stecken. Somit befinden sich Glas und Flasche stets an einer fixen Position und bei einem Auslaufen von Flüssigkeit werden die Komponenten im Inneren der Zelle nicht beschädigt. In Abbildung 40 ist der Aufbau in der Schulungszelle gezeigt.

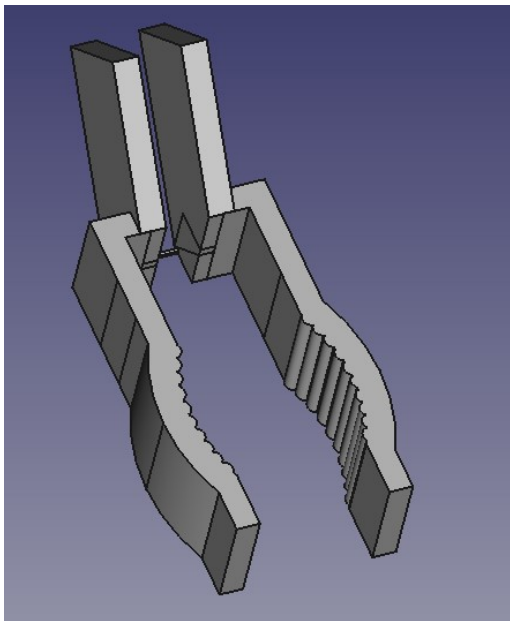


Abbildung 40: (Links) 3D-Modell der Greiferzangen. (Rechts) Aufbau im Arbeitsraum für die Flaschenapplikation.

In Abbildung 41 sind der Positionsverlauf und die Zykluszeiten für die Flaschenapplikation gezeigt. Es wurden zum Befüllen nacheinander die Punkte 1 bis 7 angefahren und danach die gleichen Punkte in umgekehrter Reihenfolge, um die Flasche abzustellen. Punkt 2 bis 4 dienen zum präzisen Anfahren der Zange an den Flaschenhals und zum Anheben aus der Halterung. Es werden dabei Linearbewegungen mit 0.3 bzw. 0.1 m/s ausgeführt. Die Flasche wird

über einen Umweg mit Punkt 5 auf Punkt 6 positioniert, da sonst die Achse 4 bis zu ihrem Endanschlag gedreht wird und das KRL-Programm dann stoppt. An Punkt 6 ist die Flasche dann um ca. 40° angewinkelt und mit dem Kopf über dem vorderen Glasrand ausgerichtet. Bei der Bewegung zu Punkt 7 wird sie dann langsam mit 0.08 m/s auf ca. 130° gedreht damit das Wasser nicht spritzt. Zudem wird die Flasche leicht zum Glas hin bewegt, da durch den sich verringernden Wasserdruck beim Ausschenken das Fließverhalten sich ändert. Beim Erreichen von Punkt 7 wird noch 2 Sekunden gewartet, damit die Flasche komplett geleert wird. Wie auch bei der ersten Applikation treten bei den Zykluszeiten zwei Latenzspitzen von bis zu 160 ms auf. Dies ist beeinflusst aber ebenfalls die Ausführung der Applikation nicht.

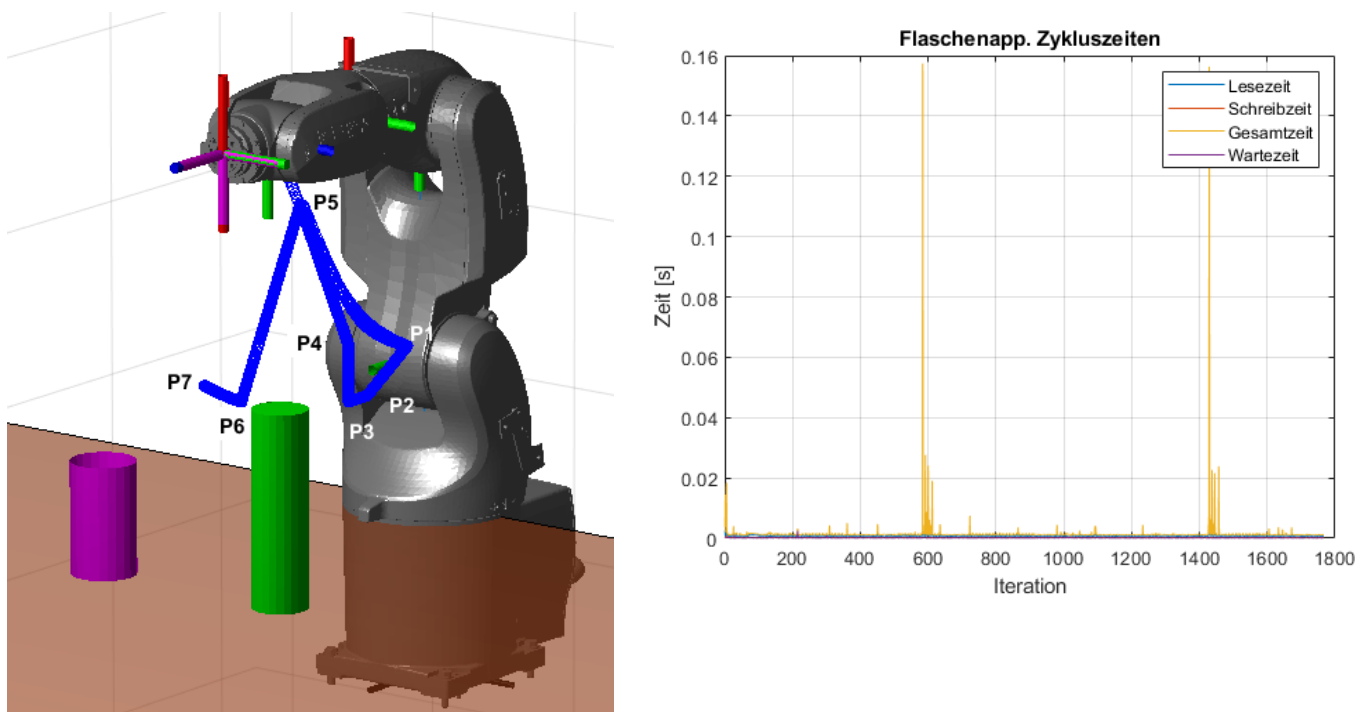


Abbildung 41: (Links) Modell des Laboraufbaus und Ist-Positionsverlaufs. (Rechts) Zykluszeiten während des Pollings bei Ausführung der Applikation.

5.3. Singularitäten

Durch die Konfiguration der sechs rotierenden Gelenke ergeben sich bei dem KR3 drei singuläre Stellungen, nämlich die Überkopf-, Strecklagen- und Handachsen-Singularität, welche in Abbildung 42 gezeigt sind [11]. Befindet sich der Manipulator in diesen Stellungen, so ergeben schon kleine Bewegungen des Endeffektors große Achswinkeländerungen. Das hat zur Folge, dass entweder die Geschwindigkeitsbegrenzung der Gelenkmotoren überschritten

oder der Endpunkt der Winkelbegrenzungen erreicht wird. Dadurch wird die Bewegung und ein eventuell laufendes KRL-Programm sofort gestoppt. Es ist deshalb wichtig, bei der Erstellung von Applikationen und bei der manuellen Bewegung darauf zu achten, dass die Singularstellungen nicht angefahren werden.

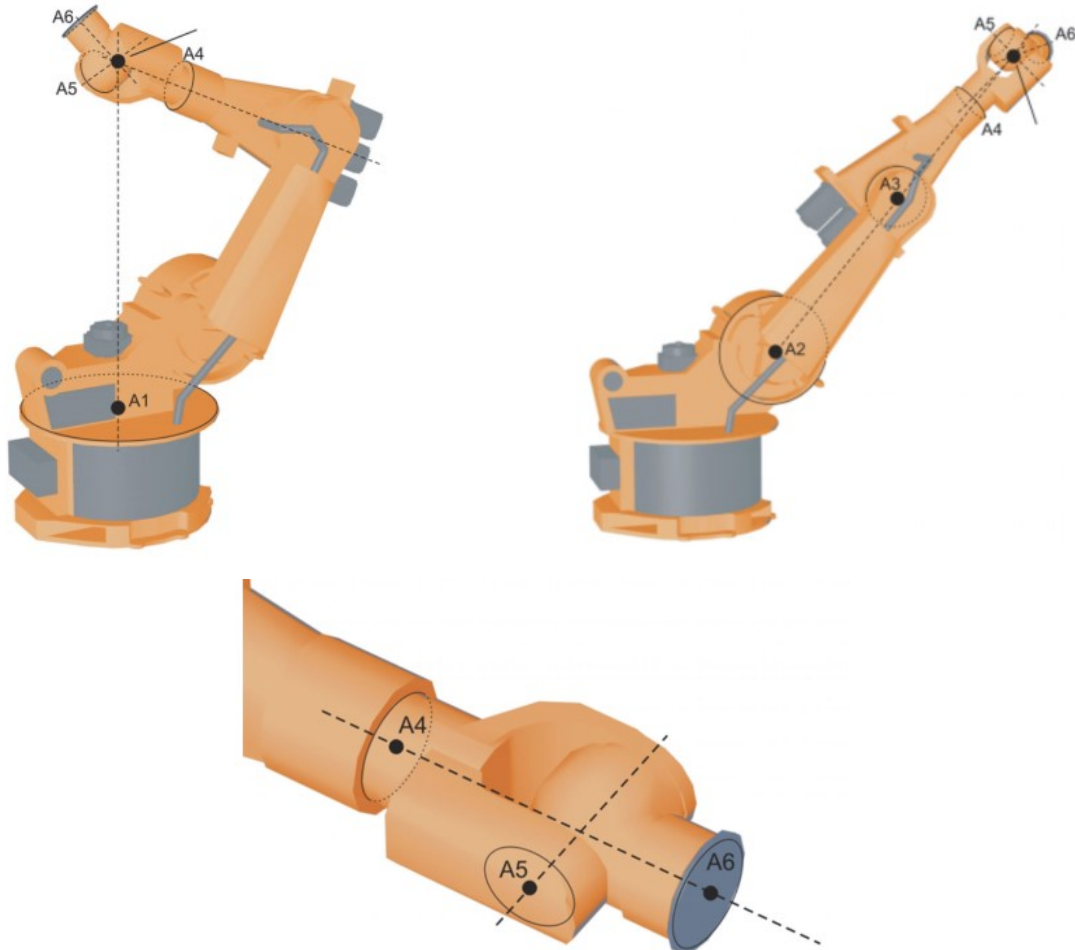


Abbildung 42: Manipulator in der Stellung der Überschlag- (o.l.), Strecklagen- (o.r.) und Handachsen-Singularität (unten) [11].

Durch den eingeschränkten Arbeitsbereich der Zelle ist ein Anfahren der ersten beiden Stellungen unwahrscheinlich, jedoch kann aus der Ausgangsstellung des Arms die Handachsen-Singularität leicht erreicht werden. Dabei liegen dann wie in Abbildung 42 unten zu sehen die Z-Achsen der Gelenke 4 und 6 parallel zueinander während die Achse 5 einen Wert nahe 0° hat. Dadurch ergeben sich unendlich viele mögliche Werte für die Gelenke 4 und 6, was die inverse Kinematik der Steuerung nicht auflösen kann. Es empfiehlt sich daher darauf zu achten, dass die Achse A5 stets angewinkelt ist. Bei kartesischen Bewegungen ist jedoch oft

nicht voraussehbar, welche Winkel die Achsen annehmen, weshalb sich die Vermeidung der Handachsen-Singularität als nicht trivial ergibt. Eine Möglichkeit, die auch die KUKA-Steuerung in der Betriebsart T1 verwendet, ist es die Geschwindigkeit der Achsen in der Nähe der Singularität zu reduzieren [47]. Dies ist jedoch nicht für AUT konfiguriert und daher muss der Anwender bei der Anwendungserstellung auf die richtige Wahl der Wegpunkte und der Geschwindigkeiten achten.

Weiterhin ist zu beachten, dass Singularitäten nur bei Bahnbewegungen wie LIN oder CIRC auftreten, da bei der PTP-Bewegung nicht kontinuierlich die inverse Kinematik berechnet werden muss. Es kann auch eine genauere Betrachtung der verschiedenen möglichen Achswinkelwerte für eine Position helfen, um bei der Bewegung zu dieser Position Singularitäten zu vermeiden. Wie schon in Abschnitt 2.6 erwähnt legen die Felder Status und Turn der POS-Struktur in KRL eine konkrete Achswinkelkonfiguration für einen Punkt fest. Weil die PTP-Bewegung die Strecke mit den kleinsten Winkeländerungen fährt und so eine Konfiguration schon indirekt gewählt wird, sind die Status- und Turn-Werte nur für die Bahnbewegungen relevant.

Diese sind als INT-Werte definiert, die aber wie in Abbildung 43 gezeigt jeweils einen binären Wert repräsentieren. Für Turn steht dabei jedes Bit für ein Gelenk beginnend bei Gelenk 1 beim untersten Bit. Ein Wert von 0 für ein Bit steht dann für einen positiven Wert des entsprechenden Gelenkwinkels und ein Wert von 1 für einen negativen Wert. Das Statusfeld hat nur drei Bits, welche wie folgt zugewiesen sind. Falls die Position der Achse 5 einen positiven X-Wert hat, befindet sich der Arm im Grundbereich und Bit 0 hat den Wert 0. Bei einem Wert von 1 befindet sich der Arm dann im Überkopfbereich, wo der Arm nach „hinten“ gelehnt ist. Wenn bei einer Bewegung beim Start- und Zielpunkt sich deren Status-Wert in Bit 0 unterscheiden, kann dies somit auf eine mögliche Überkopf-Singularität hindeuten. Bit 1 gibt an, ob Gelenk 3 einen gewissen Schwellwert überschreitet. Dieser Wert ist jedoch je nach Manipulator-Modell unterschiedlich und kann nur experimentell bestimmt werden. Bit 2 gibt anhand des Gelenks 5 an, ob der Endeffektor relativ zum Gelenk 5 gesehen über oder unter ihm liegt.

T:

Wert	Bit 5	Bit 4	Bit 3	Bit 2	Bit 1	Bit 0
0	$A6 \geq 0^\circ$	$A5 \geq 0^\circ$	$A4 \geq 0^\circ$	$A3 \geq 0^\circ$	$A2 \geq 0^\circ$	$A1 \geq 0^\circ$
1	$A6 < 0^\circ$	$A5 < 0^\circ$	$A4 < 0^\circ$	$A3 < 0^\circ$	$A2 < 0^\circ$	$A1 < 0^\circ$

S:

Wert	Bit 2	Bit 1	Bit 0
0	$0^\circ \leq A5 < 180^\circ$ $A5 < -180^\circ$	$A3 < \emptyset$ ($\emptyset$ hängt vom Robotertyp ab)	Grundbereich
1	$-180^\circ \leq A5 < 0^\circ$ $A5 \geq 180^\circ$	$A3 \geq \emptyset$ ($\emptyset$ hängt vom Robotertyp ab)	Überkopfbereich

Abbildung 43: Bedeutung der einzelnen Bits eines Status- und Turn Wertes [48].

6. Modell

Im Labor sind derzeit nur zwei Zellen mit einem Roboterarm verfügbar, was für den Lehrbetrieb hinderlich sein kann, da Studierende nur begrenzte Zeit damit arbeiten können. Es wurde deshalb neben den Steuerungsfunktionen für die Zellen selbst ein Programm geschrieben, welches den Arm in MATLAB modelliert und damit dann wie die reale Version Trajektorien abfährt. Somit ist es Studierenden möglich, sich auf eventuelle Aufgaben im Labor vorzubereiten und selbst erstellte Lösungsansätze vorher auszuprobieren. In Abschnitt 6.1 wurde zuerst die Kinematik des Manipulators betrachtet und Modellierungsmethoden in MATLAB untersucht. In Abschnitt 6.2 wird dann der Programmablauf beschrieben, welcher die Zwischenpunkte der Trajektorie erstellt anhand von gegebenen Wegpunkten.

6.1. Modellierung des KR3

Für die Modellierung eines Manipulators stehen in MATLAB zwei Toolboxen zur Verfügung, nämlich die *Robotic Systems Toolbox* (RST) und *Simscape Multibody* (SM). SM ist für die Modellierung und Simulation von mechanischen, dreidimensionalen Systemen in Simulink ausgelegt, während RST verschiedene Funktionen für die Erstellung von Robotermodellen allgemein bietet. Es wurden beide Toolboxen für die Modellerstellung verwendet.

Der KR3 besteht aus 6 rotierenden Gelenken, die sich nur um eine Achse drehen. Die Gelenkkonfiguration entspricht den Denavit-Hartenberg-Parametern aus

Tabelle 9.

Achse	θ	d	A	α
0	-	183 mm	-	180°
1	q_1	-162 mm	20 mm	90°
2	q_2	-	260 mm	-
3	$q_3 - 90^\circ$	-	20 mm	90°
4	q_4	-138 mm	-	-90°
5	q_5	-	-	-90°
6	q_6	-	-	180°

Tabelle 9: Verwendete Denavit-Hartenberg-Parameter für die Modellierung des KR3.

Die Achse 0 beschreibt eine zusätzliche feste Transformation von dem Fußpunkt des Roboters zum ersten Starren Körper. Damit die Gelenkkoordinatensysteme aber auch im Gelenk selbst liegen, sind noch bei der Modellierung im Programm ein Offset für Achse 3, 4 und 5 mitberücksichtigt.

Für die Modellerstellung in RST ist das Skript *create_robottree.m* zuständig. Es werden mit den Befehlen *RigidBody(name)* und *Joint(name,'revolute')* starre Körper und einachsige Drehgelenke, welche bei einer Sollwinkelvorgabe sich um ihre z-Achse drehen, erstellt. Ein Gelenk verbindet zwei starre Körper und mit *setFixedTransform(Joint,tform)* wird mithilfe einer homogenen Transformationsmatrix *tform* die relative Positionsänderung des Gelenks in Bezug zum vorherigen Gelenk angegeben. Die Länge der starren Körper ergibt sich dann aus den Positionen der Gelenke. Weiterhin lassen sich Grenzwerte für die Gelenkbewegungen festlegen, wofür die Werte aus Abbildung 5 übernommen wurden, damit bei der Berechnung der inversen Kinematik später keine ungültigen Bewegungen zustande kommen. Die Gelenke werden dann dem jeweils nachfolgenden starren Körper zugewiesen und diese dann dem *rigidbodytree*-Objekt zugeordnet, welches schließlich die kinematische Kette darstellt. Mit Quelltext 5 ist dieser Prozess für einen starren Körper gezeigt.

```
10 %Basis
11 body1 = robotics.RigidBody('body1');
12 jnt1 = robotics.Joint('jnt1','revolute');
13 jnt1.HomePosition = 0;
14 jnt1.PositionLimits = [-170/180*pi , 170/180*pi];
15 tform = [RotX(pi), [0;0;0.183]; 0,0,0,1];
16 setFixedTransform(jnt1,tform);
17 body1.Joint = jnt1;
18
19 %kin. Kette erstellen und body1 einfügen
20 robot = robotics.RigidBodyTree;
21 addVisual(robot.Base,"Mesh",strcat(mainpath,'\CAD-Dateien\
baseMesh.stl'));
22 addVisual(body1,"Mesh",strcat(mainpath,'\CAD-Dateien\
body1Mesh.stl'),tform);
23 addBody(robot,body1,'base')
```

Quelltext 5: Erstellung eines starren Körpers und Gelenkes, welche zur kinematischen Kette 'robot' hinzugefügt werden.

Es ist auch möglich, den starren Körpern eine 3D-Visualisierung mit einer STL-Datei zuzuordnen. Dafür wurde in [49] ein *Computer-aided Design* (CAD)-Modell des KR3 als *Standard for*

the Exchange of Product Model Data (STP)-Datei genommen und mit dem Programm Free-CAD die Teilkomponenten für jeden starren Körper separat extrahiert und daraus ein 3D-Mesh generiert, welches als STL-Datei gespeichert wurde. In Abbildung 44 ist die komplette kinematische Kette ohne und mit Visualisierung in der Standardstellung der Gelenke gezeigt. Diese betragen von Gelenk 1 bis 6 $[0, 110, -110, 0, 0, 0]$ in Grad.

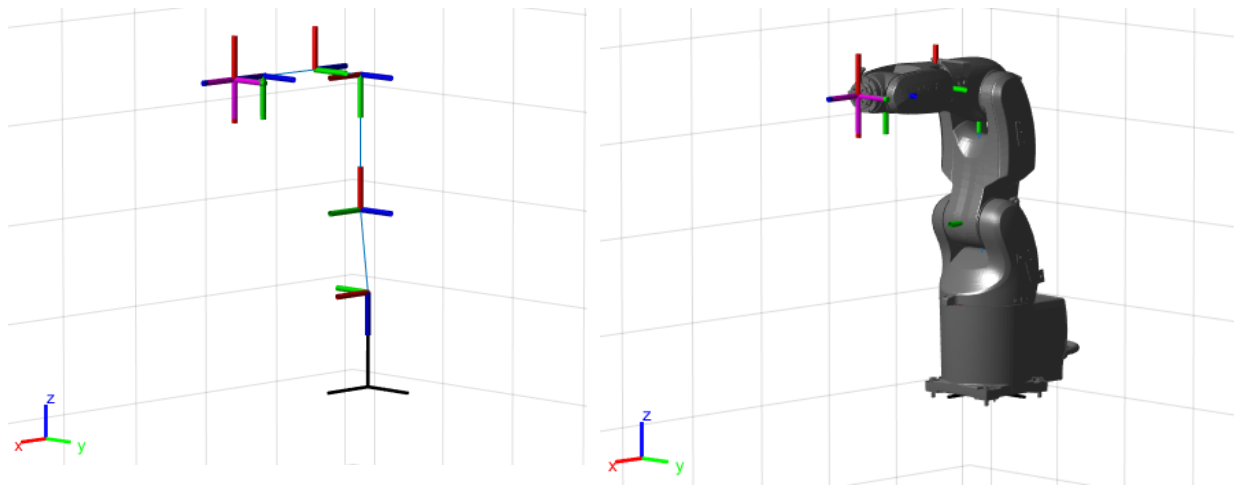


Abbildung 44: Darstellung der mit RST erstellten kinematischen Kette ohne (links) und mit (rechts) Visualisierung.

Die Modellerstellung in SM funktioniert ähnlich wie in RST mit der Verknüpfung von starren Körpern und Gelenken, jedoch beziehen sich alle Elemente auf ein Basis-Koordinatensystem. Von diesem lassen sich mit Transformationsblöcken nun neue Referenzsysteme definieren, an denen die Körper und Gelenke angehängt werden können. Die Drehung eines Gelenkes wird dann als Drehung des Folgesystems um die z-Achse des vorherigen Systems realisiert. Starre Körper sind in SM an sich unabhängige Objekte mit einem Referenzpunkt, welcher an ein bestehendes Koordinatensystem fest angehängt werden kann. Bei einer Bewegung des Systems bewegt sich der Körper entsprechend mit, sodass die relative Position des Körpers zum System gleich bleibt. Es können entweder Körper in einer gegebenen GUI selbst modelliert werden oder als CAD-Dateien im STP-Format importiert werden, was auch für den KR3 gemacht wurde. Da die einzelnen CAD-Dateien aus einer Datei extrahiert wurden, wo der Referenzpunkt im Fußpunkt des Roboters lag, haben die in Simulink importierten Körper ebenfalls den gleichen Referenzpunkt. Deshalb muss vor dem Einfügen eines Körpers an ein Bezugssystem der Referenzpunkt mit einem Transformationsblock so verschoben werden, dass

der Körper alle gemachten Transformationen vom Anfangs- bis zum aktuellen Bezugssystem berücksichtigt. In Abbildung 45 ist ein Ausschnitt aus dem Simulink-Programm mit den vorher beschriebenen Komponenten und das fertige Modell im Simulationsablauf mit allen verwendeten Koordinatensystemen und markiertem absolutem Bezugssystem gezeigt. Es wurde das Modell in [50] als Vorlage verwendet, wo bei den Transformationsblöcken die Werte entsprechend für den KR3 geändert und eine zusätzliche Ausgabe der Endeffektor-Orientierung erstellt wurde. Das angepasste Modell ist als *KR3_Modell.slx* gespeichert.

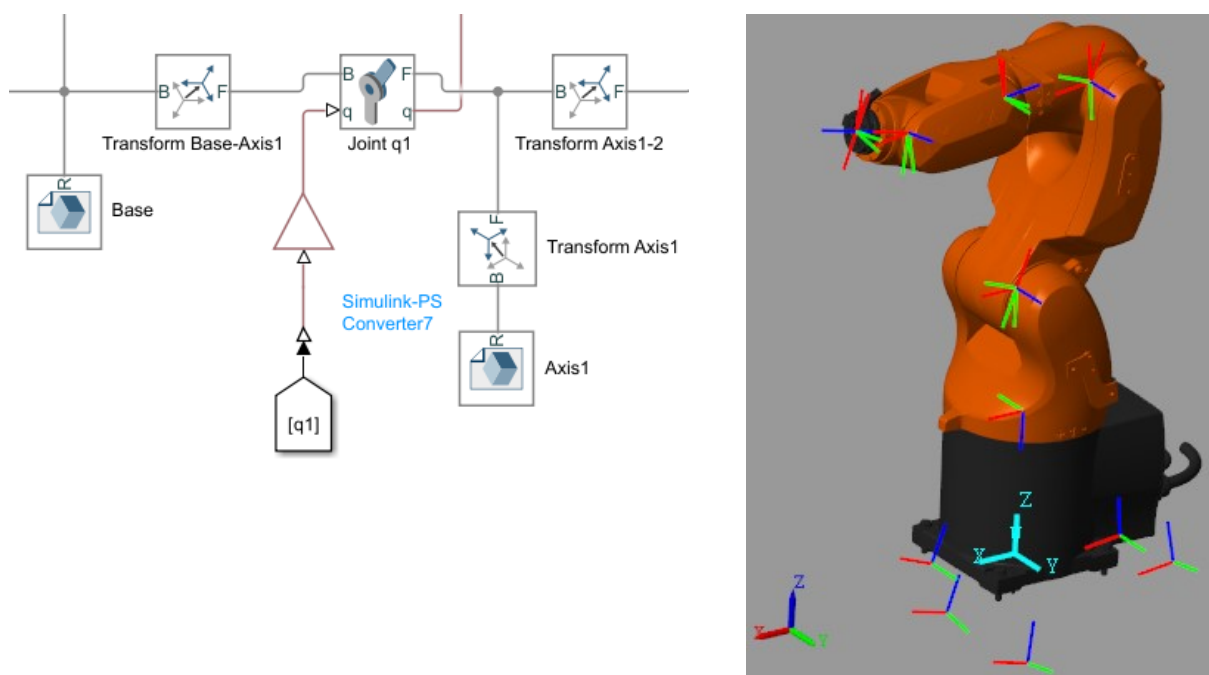


Abbildung 45: (Links) Ausschnitt aus dem Modellierungsteil des Simulink-Modells. (Rechts) Darstellung des Modells mit allen verwendeten Koordinatensystemen.

In beiden Toolboxen ist zusätzlich die Berechnung der Dynamik des Arms möglich. So können in Simulink dann anstatt Sollwinkeln auch den Gelenken Drehmomente übergeben werden. Dafür ist die Angabe von Massen und Trägheitsmomenten notwendig. Für die starren Körper in SM können anhand der Form der CAD-Teilmodelle und einer Dichteangabe, die Werte automatisch berechnet werden. In diesem Fall wurde ein Wert von 1110 kg/m^3 gewählt, damit annähernd eine Gesamtmasse von $26,5 \text{ kg}$ für alle Teile entsteht. Für die Körper des

rigidbodytree-Objekts wurden im Skript die Werte für eventuell spätere Anwendungen eingetragen, im Moment aber werden sie nicht benötigt

6.2. Erstellung der Trajektorie

Mit der Funktion *generate_trajectory.m* wird anhand der Weg- und Hilfspunkte, welche mit *Teachskript.m* und *Teachskript_Spacemaus.m* gespeichert werden können, und den dazugehörigen Bewegungsarten und Geschwindigkeiten eine Trajektorie erstellt und die Bewegung in einem 3D-Graph angezeigt. Die Trajektorie besteht aus dem Ist-Positionsverlauf des Endeffektors und aus den zur Position dazugehörigen Achswinkeln. Dazu wird anhand der in Abschnitt 6.1 modellierten kinematischen Kette ein inverser Kinematik-Solver mit dem RST-Befehl *ik = InverseKinematics(RigidBodyTree', robot)* erzeugt. Dieser arbeitet numerisch mit dem Broyden-Fletcher-Goldfarb-Shanno (BFGS) Verfahren mit maximal 200 Iterationen. Es werden dann alle Wegpunkte durchiteriert und anhand der Bewegungsart die Zwischenpunkte erstellt.

Bei der Bewegungsart PTP werden zunächst für jeweils den Start- und Zielpunkt einer Teilstrecke der Trajektorie die inverse Kinematik berechnet. Das *ik*-Objekt nimmt dabei als Argument die Bezeichnung des Endeffektors, die homogene Transformationsmatrix des Punktes und jeweils ein 1x6 Array mit Gewichten und Startwerten. Danach wird anhand der Geschwindigkeit und des Abstandes zwischen Start- und Zielpunkt die notwendige Anzahl an Zwischenpunkten n für den aktuellen Teilabschnitt i mit folgender Formel berechnet.

$$n_i = \frac{|\vec{wp}_{i+1} - \vec{wp}_i|}{v_{\max} * v_{\text{rel},i}} * \text{FPS} \quad (4)$$

$\vec{wp}$ sind die vorgegebenen Wegpunkte und v_{rel} die relative Geschwindigkeit bezüglich $v_{\max}$. Diese ist im Skript 1 m/s, da bei größeren Werten die Bewegung zu schnell angezeigt und keine nachvollziehbare Trajektorie generiert wird. Außerdem ist der Abstand $|\vec{wp}_{i+1} - \vec{wp}_i|$ nur bei Linearbewegungen exakt und führt deshalb zu einer etwas schnelleren PTP-Bewegung als erwartet. Die aus dem Wegpunktabstand und Geschwindigkeit berechnete Dauer der Teilstrecke wird aktuell mit einer Bildfrequenz (FPS) von 24 multipliziert. Mit dem RST-Befehl *trapveltraj* wird dann zwischen der Start- und Zielachskonfiguration eine Trajek-

torie mit n Zwischenwerten und trapezförmigem Geschwindigkeitsprofil erzeugt. Der Anteil für die Beschleunigung und Bremsung beträgt dabei jeweils 10% von der Gesamtdauer der Teilbewegung. Da keine genauen Daten zum Geschwindigkeitsverhalten des Manipulators während einer Bewegung gefunden wurden, sind diese Einstellung jedoch nur eine Schätzung.

Um die Trajektorie auch zeichnen zu können, wird für jeden Zwischenwert die Position und Orientierung des Endeffektors mit dem Befehl *getTransform* und der kinematischen Kette berechnet. Die Orientierung wird für alle Bewegungen als Quaternion gespeichert, da der Befehl *plotTransforms*, mit dem für jeden Zwischenpunkt die Orientierung dargestellt werden kann, die Parameter in dieser Form erwartet. In Quelltext 6 ist der Codeausschnitt für eine PTP-Bewegung gezeigt.

```

56  %-----ptp-----
57  if(movemod(i) == 1)
58      axis_start = ik('body7',tformstart,weights,initialguess);
59      axis_target = ik('body7',tformtarget,weights,initialguess);
60      %Anzahl ZWP anhand Geschw. berechnen
61      dist = sqrt((waypoints(i+1,1)- waypoints(i,1))^2 +...
62                  (waypoints(i+1,2)- waypoints(i,2))^2 +...
63                  (waypoints(i+1,3)- waypoints(i,3))^2);
64      t      = dist/(movevel(i)/100*maxvel);
65      numSamples = round(t*fps);
66      [q,qd,~,tSamples,pp] = trapveltraj([axis_start,
67                                         axis_target],numSamples);
68      traj = [traj, q];
69      for k = 1:size(q,2)
70          hommat = getTransform(robot,q(:,k), 'body7', 'base');
71          tcppos = [tcppos, hommat(1:3,4)];
72          quaterns = quaternion(hommat(1:3,1:3), 'rotmat', 'point');
73          qtlist = [qtlist, quaterns.'];
74      end
75  %-----lin-----
76  elseif(movemod(i) == 2)
77      :

```

Quelltext 6: Ausschnitt aus *generate_trajectory.m*, der für die Generierung von Zwischenpunkten für eine PTP-Bewegung zuständig ist.

Bei einer Linearbewegung wird ähnlich vorgegangen, jedoch werden zuerst die Zwischenpunkte als Positionen interpoliert und danach aus den Positionen mit dem IK-Solver die Achswinkel. Dies hat aber zur Folge, dass die Berechnung für den Teilabschnitt deutlich länger dauert, da der IK-Solver öfter aufgerufen wird.

Die Trajektorienbildung für eine Kreisbewegung ist komplizierter, denn es sind dafür nur drei Punkte im dreidimensionalen Raum gegeben, nämlich der Startpunkt $\overrightarrow{SP}$, der Hilfspunkt $\overrightarrow{HP}$ und der Zielpunkt $\overrightarrow{ZP}$ einer Teilbewegung. Zuerst wird daher der Normalenvektor $\vec{n}$ der Ebene, auf der die drei Punkte liegen, berechnet. Dafür wird das Kreuzprodukt der beiden Hilfsvektoren $\overrightarrow{w_1} = \overrightarrow{HP} - \overrightarrow{SP}$ und $\overrightarrow{w_2} = \overrightarrow{ZP} - \overrightarrow{SP}$, wie in Formel 5 gezeigt, gebildet.

$$\vec{n} = \frac{\overrightarrow{w_1}}{|\overrightarrow{w_1}|} \times \frac{\overrightarrow{w_2}}{|\overrightarrow{w_2}|} \quad (5)$$

Danach wird der Mittelpunkt des Kreises $\overrightarrow{MP}$ bestimmt, auf denen die drei Punkte liegen. Da der Abstand vom Mittelpunkt zu allen drei Punkten gleich ist, lassen sich folgende Gleichungen aufstellen.

$$\begin{aligned} |\overrightarrow{MP} - \overrightarrow{SP}| &= |\overrightarrow{MP} - \overrightarrow{HP}| \\ |\overrightarrow{MP} - \overrightarrow{ZP}| &= |\overrightarrow{MP} - \overrightarrow{SP}| \\ |\overrightarrow{MP} - \overrightarrow{HP}| &= |\overrightarrow{MP} - \overrightarrow{ZP}| \end{aligned} \quad (6)$$

Nach dem quadrieren auf beiden Seiten und ausmultiplizieren der Klammern kürzen sich die quadratischen Terme für x_{mp} , y_{mp} und z_{mp} weg. Es bleibt ein lineares Gleichungssystem, welches in Formel 7 gezeigt ist.

$$\begin{aligned} \text{I} \quad & x_{mp}(-2x_{sp} + x_{hp}) + y_{mp}(-2y_{sp} + y_{hp}) + z_{mp}(-2z_{sp} + z_{hp}) \\ & = x_{hp}^2 - x_{sp}^2 + y_{hp}^2 - y_{sp}^2 + z_{hp}^2 - z_{sp}^2 \\ \text{II} \quad & x_{mp}(-2x_{zp} + x_{sp}) + y_{mp}(-2y_{zp} + y_{sp}) + z_{mp}(-2z_{zp} + z_{sp}) \\ & = x_{sp}^2 - x_{zp}^2 + y_{sp}^2 - y_{zp}^2 + z_{sp}^2 - z_{zp}^2 \\ \text{III} \quad & x_{mp}(-2x_{hp} + x_{zp}) + y_{mp}(-2y_{hp} + y_{zp}) + z_{mp}(-2z_{hp} + z_{zp}) \\ & = x_{zp}^2 - x_{hp}^2 + y_{zp}^2 - y_{hp}^2 + z_{zp}^2 - z_{hp}^2 \end{aligned} \quad (7)$$

Falls eine Koordinate von zwei Punkten gleich ist, so ist dieses System aber unterbestimmt und es existieren unendlich viele Lösungen. Diese befinden sich dann auf einer Ebene, welche zwischen diesen beiden Punkten liegt und entlang der Achse mit dem gleichen Wert verläuft. Falls dies der Fall ist, wird im Code für die Koordinate des Mittelpunktes, für welche die Werte von zwei Punkten gleich sind, die Mitte zwischen diesem Wert und den Wert des anderen Punktes gewählt. In Abbildung 46 ist dies veranschaulicht. Es wird dann eine weitere

Zeile zum Gleichungssystem mit dem Wert hinzugefügt. Schließlich wird mit dem Befehl *lin-solve* das Gleichungssystem gelöst und der Mittelpunkt berechnet.

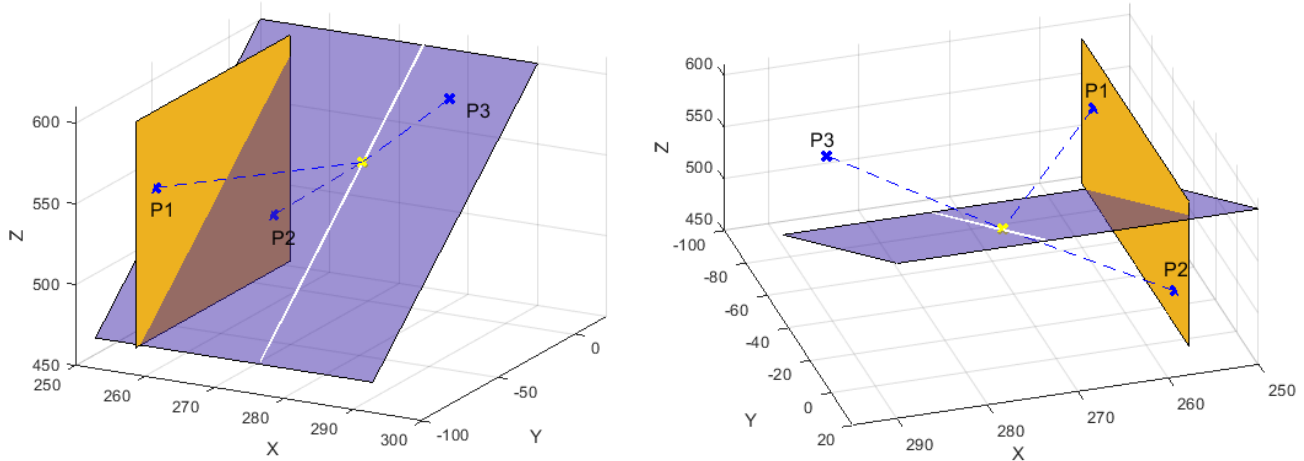


Abbildung 46: Darstellung des Lösungsbereichs für den Mittelpunkt des Kreises. Die orangene Ebene beinhaltet P1 und P2 und die violette Ebene enthält alle Punkte mit gleichem Abstand zu P1 und P2. Die weiße Linie zeigt die Lösungen für einen festgelegten X-Wert für den Mittelpunkt.

Für die Berechnung des Winkels zwischen Start- und Zielpunkt wird ein neues orthogonales Koordinatensystem aufgespannt mit den Vektoren $\vec{v}_1 = \overrightarrow{SP} - \overrightarrow{MP}$, $\vec{v}_2 = \vec{n} \times \vec{v}_1$ und $\overrightarrow{MP}$ als Ursprung. Es werden dann mit folgender Formel zuerst der Winkel zwischen $\overrightarrow{SP}$ und $\overrightarrow{HP}$ und analog zwischen $\overrightarrow{HP}$ und $\overrightarrow{ZP}$ berechnet. So wird der Winkel ω des Kreisbogens berechnet, der auch durch $\overrightarrow{HP}$ verläuft.

$$\omega = \tan^{-1} \left(\frac{|\overrightarrow{HPMP} \times \vec{v}_1|}{\overrightarrow{HPMP} * \vec{v}_1} \right) \quad (8)$$

Anhand des Winkels und des Kreisradius wird dann die Länge des Kreisbogens berechnet und mit der Geschwindigkeit dann mit Formel 4 die Anzahl der Zwischenpunkte. Damit wird dann zwischen 0 und ω interpoliert und die Winkel α_k gebildet. Schließlich werden die Zwischenpunkte wie folgt gebildet.

$$\overrightarrow{ZWP_k} = \overrightarrow{MP} + \cos(\alpha_k) \vec{v}_1 + \sin(\alpha_k) \vec{v}_2 \quad (9)$$

Wie für die Linearbewegung wird dann für jeden Zwischenpunkt die inverse Kinematik berechnet.

Am Ende des Codes gibt es die Möglichkeit, die berechnete Bahn animiert dazustellen. Es werden dabei für alle Zwischenpunkte der Trajektorie nacheinander in einem 3D-Graphen die Roboterstellung und die Position des aktuellen und der vorherigen Punkte gezeigt. In Abbildung 47 ist ein Ausschnitt aus solch einer Sequenz gezeigt. Die CAD-Objekte für die starren Körper werden aber nicht dargestellt, da dies ein höherer Rechenaufwand ist und die Animation dadurch verlangsamt wird. Das Simulink-Modell kann ebenfalls mit den Zwischenpunkten eine Bahnfahrt darstellen, jedoch ist es hier nicht möglich, die Abspielgeschwindigkeit zu beeinflussen und die schon abgefahrte Trajektorie zu zeichnen. Die Darstellung in Simulink bietet jedoch den Vorteil, dass die Bewegung zu verschiedenen Zeitpunkten pausierbar und die Animation beliebig oft wiederholbar ist.

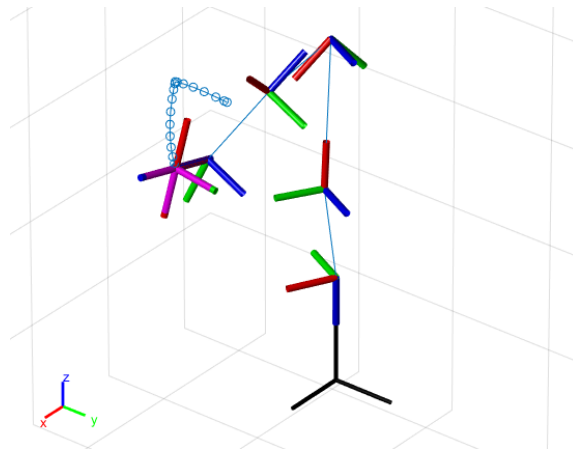


Abbildung 47: Ausschnitt aus der Animation der Bahnfahrt des Robotermodells.

7. Fazit

7.1. Bewertung

In dieser Masterthesis wurde eine Anbindung an eine Schulungszelle mit dem Roboterarm KR3 Agilus von KUKA über das Programm MATLAB implementiert, um damit den Bewegungsablauf des Arms extern steuern zu können. Es wurden dafür die die Komponenten der Zelle sowie deren Funktionsweise untersucht. Weiterhin wurden verschiedene Anbindungsmöglichkeiten vorgestellt und verglichen, um auf Basis darauf Werkzeuge in MATLAB zu erstellen, welche für das Erstellen von Applikationen mit dem Roboterarm genutzt werden können. Bei der Bearbeitung wurden die in Tabelle 1 genannten Anforderungen berücksichtigt. In Tabelle 10 sind diese Anforderungen nochmal bewertet.

Mit *Bahnskript.m* wurde Anforderung 1.1 und mit *Teachskript.m* und *Teachskript_Spacemaus.m* die Anforderungen 1.2 und 1.3 erfüllt. Mit *KukaWrapperClass.m* sind die Features 1.1-1.3 ebenfalls erfüllt. Es lässt sich nämlich mit *Bahnskript.m* mit dem Roboterarm eine Bahn fahren, dessen Wegpunkte mit den beiden anderen MATLAB-Skripten in den Workspace gespeichert werden können.

Mit der Verwendung der in Tabelle 5 gezeigten Variablen wurde für die Bahnbewegung auch Anforderung 1.4 erfüllt, jedoch lässt sich die Geschwindigkeit für die manuelle Bewegung nicht über MATLAB einstellen, sondern muss dann entsprechend über den POV-Modus auf dem SmartPAD geregelt werden. Bei allen Steuerungsprogrammen lässt sich zudem der Greifer über MATLAB betätigen, weshalb auch Anforderung 1.5 erfüllt wurde.

Bei der Bahnfahrt wird zudem der Ist-Positionsverlauf ausgegeben, welcher u.a. durch die schnelle Kommunikation über die DLL-Datei *KukaTCPClient.dll* möglich ist, jedoch wird nicht die Geschwindigkeit und Beschleunigung aufgenommen. Die Erweiterung von *Bahnskript.m* auf diese Features lässt sich jedoch analog zum Prinzip der Positionsanfrage durchführen, denn es muss dafür ebenfalls eine Systemvariable ausgelesen und von *uint8* in *char* konvertiert werden. Durch das alternierende Abfahren der Bahn über zwei Variablensätze bei der Bahnfahrt und den in Kapitel 4 vorgestellten weiteren Programmabläufen wurde ein robustes Programmverhalten ermöglicht. Wegen der Verwendung von MATLAB und dem KVP-Server auf einem Windows-Betriebssystem ist aber keine harte Echtzeit möglich, jedoch

können durch die Verwendung der DLL-Datei aber zumindest weiche Echtzeitanforderungen erfüllt werden. Durch die Eingabekontrolle bei *generate_posmsg.m* und einer Kommentierung der Codedateien wurde auf eine hohe Benutzerfreundlichkeit geachtet. Zudem ist die Installation des KVP-Servers wie in Anhang 2 dargestellt im Vergleich zu anderen Bussystemen recht einfach. Weiterhin wurde mit *create_robottree.m* und *generate_trajectory.m* eine Möglichkeit zur Visualisierung von Armbewegungen ermöglicht und somit Anforderung 4.1 erfüllt. Es ist zurzeit aber der Greifer nicht modelliert, wodurch Greifvorgänge deshalb nicht simuliert werden können.

Feature-Nr.	Beschreibung	Wichtigkeit	Status
1	Bewegungssteuerung		
1.1	Vorgabe von Sollpositionen, zu denen sich der Arm bewegt	++	erfüllt
1.2	Bewegungsführung von Hand	++	erfüllt
1.3	Speichern von Positionen & Bahnen	+	erfüllt
1.4	Anpassung von Bewegungen	+	teilweise erfüllt
1.5	Steuerung des Greifers	++	erfüllt
2	Rückgabewerte		
2.1	Rückgabe der Ist-Position	++	erfüllt
2.2	Rückgabe der Ist-Geschwindigkeit	-	nicht erfüllt
2.3	Ist-Beschleunigung	-	nicht erfüllt
3	Weitere Eigenschaften		
3.1	Robuste Ausführung	++	erfüllt
3.2	Echtzeitverhalten	+	teilweise erfüllt
3.3	Benutzerfreundlichkeit	+	erfüllt
3.4	Installationsaufwand	-	erfüllt
4	Modellanforderungen		
4.1	Darstellung der Armbewegung	+	erfüllt
4.2	Simulation des Greiferverhaltens	-	nicht erfüllt

Tabelle 10: Bewertete Anforderungen an die Lösungsimplementierung.

7.2. Ausblick

In Zukunft kann das Erstellen von weiteren Applikationen im Rahmen von Lehrveranstaltungen und Projekten stattfinden, womit Kenntnisse in der Robotik und der Umgang mit MATLAB vertieft werden können. Durch die Einschränkungen des Greifers ist bei Aufgabenstellungen an dem Roboterarm auch das Modellieren einer neuen Zange als Teil der Aufgabe denkbar. Somit können Studierende auch Kenntnisse im Konstruieren und 3D-Drucken erhal-

ten. Auch lassen sich die schon erstellten Applikationen erweitern. So kann z.B. bei der Flaschenapplikation ein Füllstandsensor integriert werden, um das Einschenken zu steuern. Über ein IO-Board an dem Anschluss X12 am KRC4 sind digitale Ein- und Ausgänge mit 24V-Pegel verfügbar, mit denen der Anschluss des Sensors denkbar wäre. Es müssten dann aber im WorkVisual-Projekt auf dem SmartPAD die Ein- und Ausgänge noch mit den Systemvariablen \$IN und \$OUT verknüpft werden, um die Sensorsignale auch in einem KRL-Programm zu verwenden.

Durch die Möglichkeit der Ausgabe der Ist-Position ist die Implementierung von Regelkreisen mit dem Roboterarm denkbar. Jedoch stellt sich hier wie vorhin beschrieben das Problem des Determinismus. Dies wäre z.B. mit QUARC zum Teil lösbar, welches ein Zusatzpaket für Simulink ist und eine Echtzeit-Ausführung von Modellen ermöglicht. Es müssten dann aber die Funktionalität der MATLAB-Skripte und -Funktionen in Simulink nachgestellt werden, jedoch ist hier zu beachten, dass bei der Verwendung von QUARC Simulink-Blöcke nur einmal in einer festen Zykluszeit ausgeführt werden. Da in den Skripten und Funktionen in MATLAB bedingte Wartezeiten enthalten sind, müssen eventuell komplett neue Ansätze für Simulink-Modelle erarbeitet werden.

Das in Kapitel 6 vorgestellte Modell hat derzeit keine Darstellung des Greifers. Es wäre somit denkbar, den am realen Roboterarm befestigten Greifer als CAD-Modell zu erstellen und in die kinematische Kette einzufügen. Die beiden für die 3D-Darstellung verwendeten Toolboxen bieten noch viele weitere Blöcke und Funktionen, mit denen u.a. die Behandlung von Kollisionen möglich ist. Ein weiteres mögliches Projekt in Zukunft wäre dann z.B. zu untersuchen, ob das Greifverhalten in dem Modell simuliert werden kann und es ggf. dann zu implementieren.

Literaturverzeichnis

- [1] C. Müller, „International Federation of Robotics IFR,“ 2020 September 24. [Online]. Available: https://ifr.org/downloads/press2018/Presentation_WR_2020.pdf. [Zugriff am 16 September 2021].
- [2] J. Mareczek, in *Grundlagen der Roboter-Manipulatoren – Band 1*, Berlin, Springer Vieweg, 2020, S. 10-11.
- [3] J. Hofschulte, *Folien zur Vorlesung Industrial Robots*, Hannover, 2019.
- [4] J.-H. Kim und M. Park, „Visualization of Concrete Slump Flow Using the Kinect Sensor,“ *Sensors*, S. 771, März 2018.
- [5] J. Kotlarski und T. Ortmaier, *Skript zur Vorlesung Robotik*, Hannover, 2013.
- [6] Kuka Deutschland GmbH, „KR3 AGILUS,“ 14 Februar 2019. [Online]. Available: https://xpert.kuka.com/service-express/portal/project1_p/document/kuka-project1_p-common_PB7966_de. [Zugriff am 19 September 2021].
- [7] KUKA Deutschland GmbH, „KR3 R540 Produkt-Information,“ 22 April 2016. [Online]. Available: https://xpert.kuka.com/service-express/portal/project1_p/document/kuka-project1_p-basic_AR12240_de. [Zugriff am 19 August 2021].
- [8] KUKA Deutschland GmbH, „ready2_educate Betriebsanleitung,“ 19 November 2020. [Online]. Available: https://xpert.kuka.com/service-express/portal/project1_p/document/kuka-project1_p-common_PB9389_de. [Zugriff am 24 August 2021].
- [9] KUKA Deutschland GmbH, „KR C4 Compact Spezifikation,“ 30 Juli 2020. [Online]. Available: https://xpert.kuka.com/service-express/portal/project1_p/document/kuka-project1_p-common_PB919_de. [Zugriff am 19 August 2021].
- [10] KUKA Deutschland GmbH, „ready2_educate Level 3,“ 2017.
- [11] KUKA Deutschland GmbH, „ready2_educate Level 1,“ 2019.
- [12] KUKA Deutschland GmbH, „VxWin White Paper JN 23058,“ April 2008. [Online]. Available: <https://www.yumpu.com/en/document/view/5333799/vxwin-white-paper-jn-23058-kuka>. [Zugriff am 19 August 2021].
- [13] KUKA Deutschland GmbH, „KUKA.WorkVisual,“ [Online]. Available: https://www.kuka.com/de-de/produkte-leistungen/robotersysteme/software/systemsoftware/kuka_systemsoftware/kuka_work-visual. [Zugriff am 16 September 2021].
- [14] KUKA Deutschland GmbH, „ready2_educate 2,“ 2019.
- [15] Profibus Nutzerorganisation e.V., „Profinet Systembeschreibung,“ 23 November 2018. [Online]. Available: <https://www.profibus.com/download/profinet-technology-and-application-system->

description. [Zugriff am 24 August 2021].

- [16] Profinet University, „Network Reference Model,“ 2021. [Online]. Available: <https://profinetuniversity.com/industrial-automation-ethernet/network-reference-model/>. [Zugriff am 24 August 2021].
- [17] KUKA Roboter GmbH, „KR C4 PROFINET 3.0,“ 18 Juli 2013. [Online]. Available: https://xpert.kuka.com/service-express/portal/project1_p/document/kuka-project1_p-common_PB3708_de. [Zugriff am 24 August 2021].
- [18] D. L. Stiegler, „Industrielle Bussysteme : EtherNet/IP,“ 2015. [Online]. Available: http://www.lehre.dhbw-stuttgart.de/~srupp/IBS/07_EtherNet-IP.pdf. [Zugriff am 24 August 2021].
- [19] C. J. Wells, „Common Industrial Protocol,“ 28 Januar 2009. [Online]. Available: <https://www.technologyuk.net/computing/computer-networks/industrial-networks/cip.shtml>. [Zugriff am 24 August 2021].
- [20] KUKA Roboter GmbH, „KR C4 EtherNet/IP 2.0,“ 25 Februar 2013. [Online]. Available: https://xpert.kuka.com/service-express/portal/project1_p/document/kuka-project1_p-common_PB1586_de. [Zugriff am 25 August 2021].
- [21] PROFIBUS Nutzerorganisation e.V., „PROFIBUS Systembeschreibung,“ April 2016. [Online]. Available: <https://www.profibus.com/download/profibus-technology-and-application-system-description>. [Zugriff am 24 August 2021].
- [22] Visaya, „Der Unterschied zwischen PROFIBUS- und PROFINET-Kommunikationsprotokollen,“ 23 Oktober 2019. [Online]. Available: <https://visaya.solutions/de/qa/difference-between-profibus-and-profinet>. [Zugriff am 24 August 2021].
- [23] KUKA Deutschland GmbH, „KR C4 PROFIBUS,“ 25 Januar 2019. [Online]. Available: https://xpert.kuka.com/service-express/portal/project1_p/document/kuka-project1_p-common_PB3708_de. [Zugriff am 2021 August 24].
- [24] Kunbus GmbH, „DeviceNet Grundlagen,“ [Online]. Available: <https://www.kunbus.de/devicenet-grundlagen.html>. [Zugriff am 25 August 2021].
- [25] KUKA Roboter GmbH, „KR C4 DeviceNet,“ 08 April 2015. [Online]. Available: https://xpert.kuka.com/service-express/portal/project1_p/document/kuka-project1_p-common_PB1586_de. [Zugriff am 25 August 2021].
- [26] EtherCAT Technology Group, „EtherCAT - Der Ethernet-Feldbus,“ [Online]. Available: <https://www.ethercat.org/de/technology.html>. [Zugriff am 25 August 2021].
- [27] T. Broling, „Wikipedia,“ 5 September 2007. [Online]. Available: <https://de.wikipedia.org/wiki/Datei:EthercatOperatingPrinciple.webm>. [Zugriff am 25 August 2021].
- [28] KUKA Deutschland GmbH, „KR C4 EtherCAT,“ 21 August 2018. [Online]. Available:

- https://xpert.kuka.com/service-express/portal/project1_p/document/kuka-project1_p-common_PB1586_de. [Zugriff am 25 August 2021].
- [29] P. M. Felser, „PROFIBUS Manual,“ [Online]. Available: <https://www.felser.ch/profibus-manual/schatzformel.html>. [Zugriff am 26 August 2021].
- [30] KUKA Deutschland GmbH, „KUKA.ProfiNet Controller/Device 3.2,“ [Online]. Available: <https://my.kuka.com/shop#/product/244984>. [Zugriff am 26 August 2021].
- [31] Mercateo Deutschland AG, „KL6731 Beckhoff Automation,“ [Online]. Available: https://www.mercateo.com/p/2064-BEC000000395/EL6731_PROFIBUS_Masterklemme.html. [Zugriff am 26 August 2021].
- [32] RS Components GmbH, „Kunbus PR100116 Netzwerkkarte,“ [Online]. Available: <https://de.rs-online.com/web/p/netzwerk-schnittstellenkarten/1825576/>.
- [33] Mercateo Deutschland AG, „Beckhoff EL6752 DeviceNet-Masterklemme,“ [Online]. Available: https://www.mercateo.com/p/1886-M0027128/Beckhoff_EL6752_DeviceNet_Masterklemme.html. [Zugriff am 26 August 2021].
- [34] WiAutomation, „1784-U2DN Allen-Bradley,“ [Online]. Available: <https://de.wiautomation.com/allen-bradley/general-automation/i-o/1784U2DN>. [Zugriff am 26 August 2021].
- [35] Mercateo Deutschland AG, „Beckhoff EK1100 EtherCAT Koppler,“ [Online]. Available: https://www.mercateo.com/p/1886-M0293921/Beckhoff_EK1100_EtherCAT_Koppler.html. [Zugriff am 26 August 2021].
- [36] inray Industriesoftware GmbH, „Was ist ein OPC Server?,“ [Online]. Available: <https://www.kepware-opcserver.de/opc-server/>. [Zugriff am 27 August 2021].
- [37] Z. Luo et al., „OPC UA-Based Smart Manufacturing: System Architecture, Implementation, and Execution,“ *5th International Conference on Enterprise Systems (ES)*, S. 281-286, 2017.
- [38] KUKA Deutschland GmbH, „KUKA.OPC UA 2.0,“ 11 Dezember 2018. [Online]. Available: https://xpert.kuka.com/service-express/portal/project1_p/document/kuka-project1_p-common_PB9970_de. [Zugriff am 27 August 2021].
- [39] KUKA Deutschland GmbH, „KUKA.RobotSensorInterface 4.0,“ 9 Juli 2018. [Online]. Available: https://xpert.kuka.com/service-express/portal/project1_p/document/kuka-project1_p-common_PB8012_de. [Zugriff am 27 August 2021].
- [40] KUKA Roboter GmbH, „KUKA.Ethernet KRL 3.0,“ 15 Januar 2018. [Online]. Available: https://xpert.kuka.com/service-express/portal/project1_p/document/kuka-project1_p-common_PB8066_de. [Zugriff am 27 August 2021].
- [41] M. Frago und L. du Peloux, „KukavarProxy,“ 6 Februar 2019. [Online]. Available: <https://github.com/lmtsrl/KUKAVARPROXY>. [Zugriff am 27 August 2021].

- [42] F. Sanfilippo et al., „Controlling Kuka Industrial Robots: Flexible Communication Interface JOpenShowVar,“ *IEEE Robotics Automation Magazine*, Bd. 22, Nr. 4, S. 96-109, 2015.
- [43] J. Golz, T. Würtz, D. Eickmann und R. Biesenbach, „RoBO-2L, a Matlab interface for extended offline programming of KUKA industrial robots,“ in *11th France-Japan 9th Europe-Asia Congress on Mechatronics (MECATRONICS) / 17th International Conference on Research and Education in Mechatronics (REM)*, Frankreich-Japan, 2016, S. 64-67.
- [44] F. Chinello, S. Scheggi, F. Morbidi und D. Prattichizzo, „KCT: a MATLAB toolbox for motion control of KUKA robot manipulators,“ in *2010 IEEE International Conference on Robotics and Automation*, 2010, S. 4603-4608.
- [45] Conrad, „3Dconnexion SpaceMouse,“ [Online]. Available: <https://www.conrad.de/de/p/3dconnexion-spacemouse-compact-kabelgebunden-3d-maus-optisch-schwarz-silber-1681800.html>. [Zugriff am 09 September 2021].
- [46] K. Bartlett, „jtcp(actionStr,varargin),“ MATLAB Central File Exchange, [Online]. Available: <https://de.mathworks.com/matlabcentral/fileexchange/24524-jtcp-actionstr-varargin>. [Zugriff am 21 September 2021].
- [47] A. Owen-Hill und Robotiq Inc., „3 types of robot singularities and how to avoid them,“ 02 März 2016. [Online]. Available: <https://robohub.org/3-types-of-robot-singularities-and-how-to-avoid-them/>. [Zugriff am 22 Oktober 2021].
- [48] G. Franz, „KRL Kuka Robot Language für KRC V5.3.5,“ SlidePlayer, [Online]. Available: <https://slideplayer.org/slide/215273/>. [Zugriff am 21 September 2021].
- [49] KUKA Deutschland GmbH, „Download Center,“ 16 Juli 2020. [Online]. Available: <https://www.kuka.com/de-de/services/downloads?terms=Language:de:1;Language:en:1;&q=KR3>. [Zugriff am 31 August 2021].
- [50] B. Notkin, „Пакет моделей к демонстрационному заданию,“ 30 Januar 2019. [Online]. Available: <https://de.mathworks.com/matlabcentral/fileexchange/70131>. [Zugriff am 3 September 2021].

Anhang 1: Ordnerstruktur der Abgabedatei

```
C:.\
|  Appl_Flasche.m
|  Appl_Flasche_Punkte.mat
|  Appl_Wuerfel.m
|  Appl_Wuerfel_Punkte.mat
|  Bahnskript.m
|  create_robottree.m
|  driver.dat
|  driver.src
|  generate_posmsg.m
|  generate_trajectory.m
|  KR3_Modell.slx
|  KukaWrapperClass.m
|  Teachskript.m
|  Teachskript_Spacemaus.m
|
+---CAD-Dateien
|   base.step
|   baseMesh.stl
|   body1.step
|   body1Mesh.stl
|   body2.step
|   body2Mesh.stl
|   body3.step
|   body3Mesh.stl
|   body4.step
|   body4Mesh.stl
|   body5.step
|   body5Mesh.stl
|   body6.step
|   body6Mesh.stl
|   GreiferzangeRund.FCStd
|   GreiferzangeRund.stl
|   Halter.FCStd
|   Halter_Flasche.stl
|   Halter_Glas.stl
|
\---kukaTCPClient
|   kukaClientClass.cs
|   kukaTCPClient.csproj
|   kukaTCPClient.sln
|
+---.vs
|   \---kukaTCPClient
|       \---v15
|           |   .suo
|           \---Server
|               \---sqlite3
|                   db.lock
|                   storage.ide
|                   storage.ide-shm
|                   storage.ide-wal
```

```

+---bin
|   \---Debug
|       kukaTCPClient.dll
|       kukaTCPClient.pdb
|
|
+---obj
|   \---Debug
|       | .NETFramework,Version=v4.5.AssemblyAttributes.cs
|       | DesignTimeResolveAssemblyReferencesInput.cache
|       | kukaTCPClient.csproj.AssemblyReference.cache
|       | kukaTCPClient.csproj.CoreCompileInputs.cache
|       | kukaTCPClient.csproj.FileListAbsolute.txt
|       | kukaTCPClient.dll
|       | kukaTCPClient.pdb
|       |
|       \---TempPE
\---Properties
    AssemblyInfo.cs

```

Anhang 2: Installation des KVP-Servers

1. Anmeldung als Administrator auf dem SmartPAD.
2. Navigation zu *Inbetriebnahme* → *Netzwerkconfiguration* im Hauptmenü des SmartPADs und Zuweisung der statischen IP-Adresse 10.2.4.240 mit der Subnetzmaske 255.255.255.0 (siehe Abbildung 48).
3. Im Menüpunkt *Erweitert* → *NAT* den Port 7000 hinzufügen und die Protokolle UDP/TCP zulassen.
4. USB-Stick mit den Dateien für den Proxyserver an SmartPAD anschließen, im Dateinavigator die Dateien *KukavarProxy.exe*, *csWSK32.ocx* und *CSWSK32.OCA* kopieren und dann im Pfad *C:/Users/kukazelle/Desktop* ablegen
5. Eine USB-Maus an das SmartPAD anschließen. Danach *Inbetriebnahme* → *Service* → *HMI minimieren* im SmartPAD-Hauptmenü wählen und per Doppelklick *KukavarProxy.exe* starten (siehe Abbildung 49).

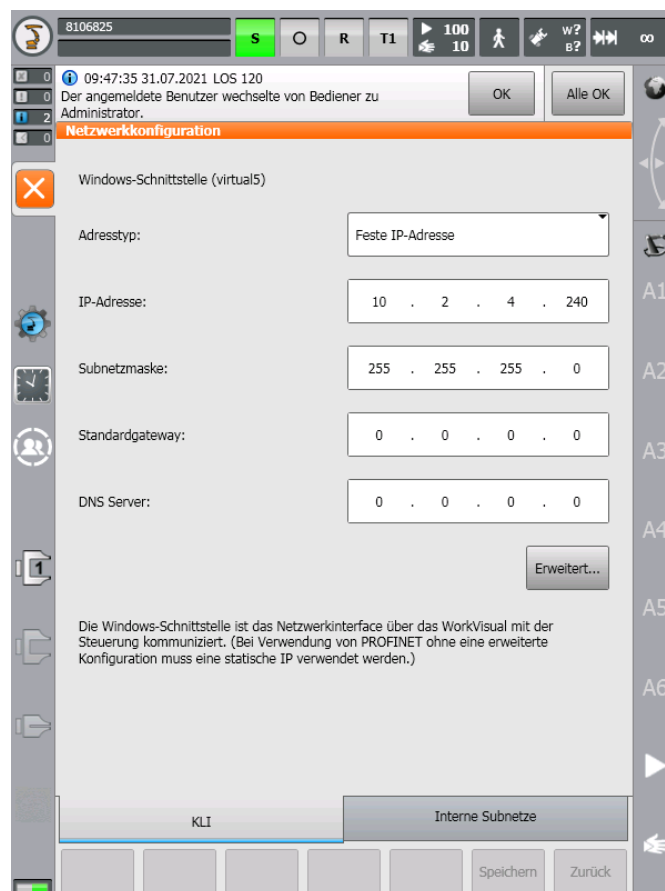


Abbildung 48: Screenshot des Netzwerkkonfigurationsmenüs mit fest zugewiesener IP-Adresse.

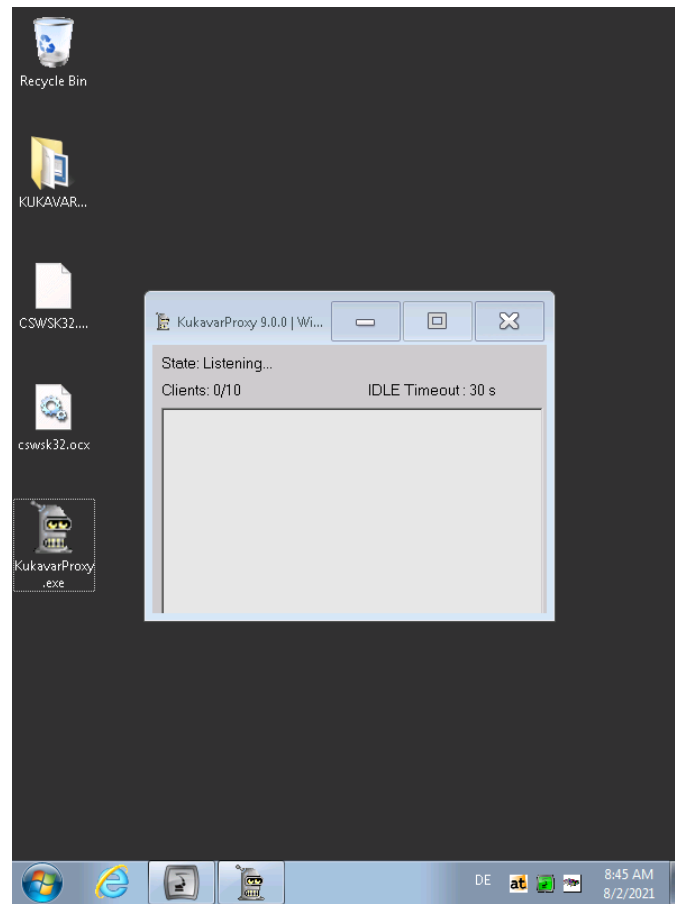


Abbildung 49: Desktop des Windows-Betriebssystems auf dem SmartPAD und laufender KVP-Server.